

1.算法面试

1.快速排序

基本思想：

通过一趟排序将待排记录分隔成独立的两部分，其中一部分记录的关键字均比另一部分的关键字小，则可分别对这两部分记录继续进行排序，以达到整个序列有序。

算法描述：

快速排序使用分治法来把一个串（list）分为两个子串（sub-lists）。

具体算法描述如下：

（1）从数列中挑出一个元素，称为“基准”（pivot）；

（2）重新排序数列，所有元素比基准值小的摆放在基准前面，所有元素比基准值大的摆在基准的后面（相同的数可以到任一边）。在这个分区退出之后，该基准就处于数列的中间位置。这个称为分区（partition）操作；

（3）递归地（recursive）把小于基准值元素的子数列和大于基准值元素的子数列排序。

代码实现：

```
function quickSort(arr, left, right) {  
    var len = arr.length,  
        partitionIndex,  
        left = typeof left !== 'number' ? 0 : left,  
        right = typeof right !== 'number' ? len - 1 : right;  
  
    if (left < right) {  
        partitionIndex = partition(arr, left, right);  
        quickSort(arr, left, partitionIndex-1);  
        quickSort(arr, partitionIndex+1, right);  
    }  
  
    return arr;  
}  
  
function partition(arr, left, right) {    // 分区操作  
    var pivot = left,                    // 设定基准值（pivot）  
        index = pivot + 1;  
    for (var i = index; i <= right; i++) {  
        if (arr[i] < arr[pivot]) {  
            swap(arr, i, index);  
            index++;  
        }  
    }
```

```
    }  
    swap(arr, pivot, index - 1);  
    return index-1;  
}
```

```
function swap(arr, i, j) {  
    var temp = arr[i];  
    arr[i] = arr[j];  
    arr[j] = temp;  
}
```

2.归并排序

归并排序是建立在归并操作上的一种有效的排序算法。该算法是采用分治法（**Divide and Conquer**）的一个非常典型的应用。将已有序的子序列合并，得到完全有序的序列；即先使每个子序列有序，再使子序列段间有序。若将两个有序表合并成一个有序表，称为 2-路归并。

算法描述：

- （1）把长度为 n 的输入序列分成两个长度为 $n/2$ 的子序列；
- （2）对这两个子序列分别采用归并排序；
- （3）将两个排序好的子序列合并成一个最终的排序序列。

代码实现：

```
function mergeSort(arr) {  
    var len = arr.length;  
    if (len < 2) {  
        return arr;  
    }  
    var middle = Math.floor(len / 2),  
        left = arr.slice(0, middle),  
        right = arr.slice(middle);  
    return merge(mergeSort(left), mergeSort(right));  
}
```

```
function merge(left, right) {  
    var result = [];  
  
    while (left.length>0 && right.length>0) {  
        if (left[0] <= right[0]) {  
            result.push(left.shift());  
        } else {
```

```

        result.push(right.shift());
    }
}

```

```

while (left.length)
    result.push(left.shift());

while (right.length)
    result.push(right.shift());

return result;
}

```

常见排序算法原理及代码实现参考如下地址：
<https://twocups.cn/index.php/2020/02/01/17/>

3.查找算法

七大查找算法：

1. 顺序查找、
2. 二分查找、
3. 插值查找、
4. 斐波那契查找、
5. 树表查找、
6. 分块查找、
7. 哈希查找，这些查找算法中二分查找是最容易考察的。

二分查找：

二分查找也称折半查找（Binary Search），它是一种效率较高的查找方法。但是，折半查找要求线性表必须采用顺序存储结构，而且表中元素按关键字有序排列，注意必须要是有序排列。

代码实现：

（1）使用递归

```

/**
 * 使用递归的二分查找
 *title :recursionBinarySearch
 *@param arr 有序数组
 *@param key 待查找关键字
 *@return 找到的位置
 */
public static int recursionBinarySearch(int[] arr,int key,int low,int high){

    if(key < arr[low] || key > arr[high] || low > high){
        return -1;
    }

    int middle = (low + high) / 2;           //初始中间位置

```

```

        if(arr[middle] > key){
            //比关键字大则关键字在左区域
            return recursionBinarySearch(arr, key, low, middle - 1);
        }else if(arr[middle] < key){
            //比关键字小则关键字在右区域
            return recursionBinarySearch(arr, key, middle + 1, high);
        }else {
            return middle;
        }
    }
}

```

(2) 不使用递归实现 (while 循环)

```

/**
 * 不使用递归的二分查找
 *title:commonBinarySearch
 *@param arr
 *@param key
 *@return 关键字位置
 */
public static int commonBinarySearch(int[] arr,int key){
    int low = 0;
    int high = arr.length - 1;
    int middle = 0;        //定义 middle

    if(key < arr[low] || key > arr[high] || low > high){
        return -1;
    }

    while(low <= high){
        middle = (low + high) / 2;
        if(arr[middle] > key){
            //比关键字大则关键字在左区域
            high = middle - 1;
        }else if(arr[middle] < key){
            //比关键字小则关键字在右区域
            low = middle + 1;
        }else{
            return middle;
        }
    }

    return -1;    //最后仍然没有找到，则返回-1
}

```

4.二叉树

定义：二叉树，是一种特殊的树，二叉树的任意一个节点的度都不大于 2，不包含度的节点称之为叶子。

遍历方式：二叉树的遍历方式有三种，中序遍历，先序遍历，后序遍历。

需求：将一个数组中的数以二叉树的存储结构存储，并遍历打印。

代码实现：

```
import java.util.ArrayList;
import java.util.List;
public class bintree {
    public bintree left;
    public bintree right;
    public bintree root;
//    数据域
    private Object data;
//    存节点
    public List<bintree> datas;

    public bintree(bintree left, bintree right, Object data){
        this.left=left;
        this.right=right;
        this.data=data;
    }
//    将初始的左右孩子值为空
    public bintree(Object data){
        this(null,null,data);
    }

    public bintree() {

    }

    public void creat(Object[] objs){
        datas=new ArrayList<bintree>();
        //        将一个数组的值依次转换为 Node 节点
        for(Object o:objs){
            datas.add(new bintree(o));
        }
//        第一个数为根节点
        root=datas.get(0);
//        建立二叉树
```

```

        for (int i = 0; i < objs.length/2; i++) {
//            左孩子
            datas.get(i).left=datas.get(i*2+1);
//            右孩子
            if(i*2+2<datas.size()){//避免偶数的时候 下标越界
                datas.get(i).right=datas.get(i*2+2);
            }

        }

    }

//先序遍历
public void preorder(bintree root){
    if(root!=null){
        System.out.println(root.data);
        preorder(root.left);
        preorder(root.right);
    }

}

//中序遍历
public void inorder(bintree root){
    if(root!=null){
        inorder(root.left);
        System.out.println(root.data);
        inorder(root.right);
    }

}

// 后序遍历
public void afterorder(bintree root){
    if(root!=null){
        System.out.println(root.data);
        afterorder(root.left);
        afterorder(root.right);
    }

}

public static void main(String[] args) {
    bintree bintree=new bintree();
    Object []a={2,4,5,7,1,6,12,32,51,22};
    bintree.creat(a);
}

```

```
        bintree.preorder(bintree.root);
    }
}
```

5.海量数据求 TOPN

上千万或上亿数据（有重复），统计其中出现次数最多的前 N 个数据。

参考答案：

上千万或上亿的数据，现在的机器的内存应该能存下。所以考虑采用 `hashmap`/搜索二叉树/红黑树等来进行统计次数。然后就可以取出前 N 个出现次数最多的数据。

6.海量数据去重

怎么在海量数据中找出重复次数最多的一个？说说你的思路

参考答案：

先做 `hash`，然后求模映射为小文件，求出每个小文件中重复次数最多的一个，并记录重复次数。然后再对上一步结果，求出的数据中重复次数最多的一个即可。

7.海量数据找不重复的整数

在 2.5 亿个整数中找出不重复的整数。谈谈你的思路

备注，内存不足以容纳这 2.5 亿个整数。

参考答案：

方案 1：采用 2-Bitmap（每个数分配 2bit，00 表示不存在，01 表示出现一次，10 表示多次，11 无意义）进行，需要内存并不多，还可以接受。然后扫描这 2.5 亿个整数，查看 Bitmap 中相对应位，如果是 00 变 01，01 变 10，10 保持不变。扫描完后，查看 bitmap，把对应位是 01 的整数输出即可。

方案 2：也可采用与第 1 题类似的方法，进行划分小文件的方法。然后在小文件中找出不重复的整数，并排序。然后再进行归并，注意去除重复的元素。

8.统计 query 频度并排序

有 10 个文件，每个文件 1G，每个文件的每一行存放的都是用户的 query，每个文件的 query 都可能重复，要求你按照 query 的频度排序。谈谈你的思路

参考答案：

方案 1：顺序读取 10 个文件，按照 `hash(query)` 的结果将 query 写入到另外 10 个文件(记为)

中。这样新生成的文件每个的大小大约也 1G(假设 hash 函数是随机的)。找一台内存存在 2G 左右的机器,依次对每个文件用 `hash_map(query, query_count)`来统计每个 query 出现的次数。利用快速/堆/归并排序按照出现次数进行排序。将排序好的 query 和对应的 query_count 输出到文件中。这样得到了 10 个排好序的文件。然后对这 10 个文件再进行归并排序(内排序与外排序相结合)。

方案 2: 一般 query 的总量是有限的,只是重复的次数比较多而已,可能对于所有的 query,一次性就可以加入到内存了。这样,我们就可以采用 trie 树/hash_map 等直接来统计每个 query 出现的次数,然后按出现次数做快速/堆/归并排序就可以了。

方案 3: 与方案 1 类似,但在做完 hash,分成多个文件后,可以交给多个文件来处理,采用分布式的架构来处理(比如 MapReduce、Spark),最后再进行合并。

9.求访问百度最多的 IP

现有一份海量日志数据保存在一个超级大的文件中,该文件无法直接读入内存,要求从超级文件中提取某天访问百度次数最多的那个 IP?谈谈你的思路

参考答案:

第一步:从这一天的日志数据中把访问百度的 IP 取出来,逐个写入到一个大文件中;
第二步:然后可以采用 `hash(IP)%1000` 映射的方法,将整个大文件映射为 1000 个小文件;
第三步:接着,找出每个小文件中出现频率最大的 IP(可以采用 `hash_map` 进行频率统计,然后再找出频率最大 IP)及相应的频率; 第四步:最后在这 1000 个最大的 IP 中,找出那个频率最大的 IP 即可。

10.求频数最高的 100 个单词

有一个 1G 大小的文件,文件里面每一行是一个词,词的大小不超过 16 字节,内存限制大小是 1M,要求返回频数最高的 100 个词,请列出具体实现步骤?谈谈你的思路

参考答案:

第一步:顺序读取文件,对于每个词 x,取 `hash(x)%5000`,然后按照该值分别映射到 5000 个小文件(记为 `f0,f1,...,f4999`)中,这样每个文件大概是 200k 左右大小。如果其中还有文件超过了 1M 大小,还可以按照类似的方法继续往下分,直到分解后得到文件的大小都不超过 1M 即可。

第二步:对与每个小文件,统计每个文件中出现的词以及相应的频率(可以采用 trie 树/hash_map 等),并取出出现频率最大的 100 个词(可以用含 100 个结点的最小堆),并将这 100 个词及相应的词频存入文件,这样又得到了 5000 个新的文件。

第三步:然后对这 5000 个新文件进行归并(类似与归并排序);

11.找出两个海量文件共同 url

给定 a、b 两个文件，各存储 50 亿个 url，每个 url 各占 64 字节，内存限制是 4G，让你找出 a、b 文件共同的 url？谈谈你的思路。

参考答案：

思路分析：每个 url 大小为 64bytes，那么可以估计每个文件的大小为 $50 \text{ 亿} \times 64\text{B} = 320\text{G}$ ，远远大于内存限制的 4G，所以不可能将其完全加载到内存中处理，可以采用分治的思想来解决。

实现步骤： 第一步：遍历文件 a，对每个 url 求取 $\text{hash}(\text{url})\%1000$ ，然后根据所取得的值将 url 分别存储到 1000 个小文件(记为 $a_0, a_1, \dots, a_{999}$ ，每个小文件约 300M)

第二步:遍历文件 b，采取和 a 相同的方式将 url 分别存储到 1000 个小文件(记为 $b_0, b_1, \dots, b_{999}$)
巧妙之处： 第一步和第二步处理后，所有可能相同的 url 都被保存在对应的小文件 ($a_0 \text{ vs } b_0, a_1 \text{ vs } b_1, \dots, a_{999} \text{ vs } b_{999}$) 中，不对应的小文件不可能有相同的 url。然后我们只需要求出这个 1000 对小文件中相同的 url 即可。

第三步：求每对小文件 a_i 和 b_i 中相同的 url 时，可以把 a_i 的 url 存储到 `hash_set/hash_map` 中。然后遍历 b_i 的每个 url，看其是否在刚才创建的 `hash_set` 中，如果是，那么就是共同的 url，存到文件里面就可以了。

12.统计海量单词词频并求 TOPN

假设你只有一台内存 2G 的笔记本，15 的四核 CPU 以及 4T 的硬盘，请设计一个程序，实现对 1T 英文数据进行词频分析，完成以下两个小题：

- 1) 求所有词的词频，把相应的值存入文件；
- 2) 找出这些词里出现频次最高的 100 个词

参考答案：

1) 将 1T 的数据使用哈希函数映射到 10000 个文件当中去，这样做可以将所有一样的数据映射到同一个文件当中（这个很重要）。依次将 10000 个文本文件读入内存，使用 `hashmap` 对每一个文本中每一个单词进行词频统计，将单词作为 `key`，单词出现次数作为 `value`，每读入一个单词，都查看其在 `hashmap` 中是否存在，存在则将 `value` 值加 1，不存在就将其加入 `hashmap`，并将 `value` 值置为 1，然后将结果写入一个文件即可。文件的每一行只存储一个单词以及这个单词出现的次数，并且二者之间用空格隔开。

2) 要求统计出现次数最高的 100 个单词。 实现原理：可以建立只有 100 个元素的小根堆

来实现目的。首先取出 100 个元素建立小根堆，继续从文件中取出元素与小根堆的堆顶元素进行比较，如果比堆顶元素大，就替换掉堆顶元素，并且重建为小根堆，依次做下去，最后堆中保留的 100 个元素就是 top100。实现方式：使用 hashmap 进行词频统计的时候，先用小根堆统计出每个文件中 Top100，然后 10000 个文件的 top100 汇总到一起后，再使用上述同样的方法得到最终的 top100。备注：这里为什么使用小根堆？大根堆排序出来的是从小到大，小根堆排序的出来的是从大到小。

2.Java 面试

1.面向对象基本特征

面向对象的基本特征是什么？怎么理解？

参考答案：

面向对象的基本特征是封装、继承、多态。

封装：

封装是把过程和数据包围起来，对数据的访问只能通过已定义的界面。面向对象计算始于这个基本概念，即现实世界可以被描绘成一系列完全自治、封装的对象，这些对象通过一个受保护的接口访问其他对象。

继承：

继承是一种联结类的层次模型，并且允许和鼓励类的重用，它提供了一种明确表述共性的方法。对象的一个新类可以从现有的类中派生，这个过程称为类继承。新类继承了原始类的特性，新类称为原始类的派生类（子类），而原始类称为新类的基类（父类）。派生类可以从它的基类那里继承方法和实例变量，并且类可以修改或增加新的方法使之更适合特殊的需要。

多态性：

多态性是指允许不同类的对象对同一消息作出响应。多态性包括参数化多态性和包含多态性。多态性语言具有灵活、抽象、行为共享、代码共享的优势，很好的解决了应用程序函数同名问题。

2.hashmap 实现原理

HashMap 实现原理是什么？

参考答案：

HashMap 基于 hash 原理，我们通过 put() 和 get() 方法储存和获取对象。当我们将键值对传递给 put() 方法时，它调用键对象的 hashCode() 方法来计算 hashcode，让后找到 bucket 位置来

储存值对象。当获取对象时，通过键对象的 `equals()` 方法找到正确的键值对，然后返回值对象。`Hash Map` 使用链表来解决碰撞问题，当发生碰撞了，对象将会储存在链表的下一个节点中。`Hash Map` 在每个链表节点中储存键值对对象。 当两个不同的键对象的 `hashcode` 相同时会发生什么？它们会储存在同一个 `bucket` 位置的链表中。键对象的 `equals()` 方法用来找到键值对。

3.事务的理解

简述一下事务，并且你对事务的理解？

参考答案：

1、简述事务的基本概念

在关系数据库中，一个事务可以是一条 `SQL` 语句，一组 `SQL` 语句或整个程序。

事务是恢复和并发控制的基本单位。

事务应该具有 4 个属性：

原子性、一致性、隔离性、持久性。这四个属性通常称为 **ACID** 特性。

原子性（**atomicity**）。一个事务是一个不可分割的工作单位，事务中包括的操作要么都做，要么都不做。

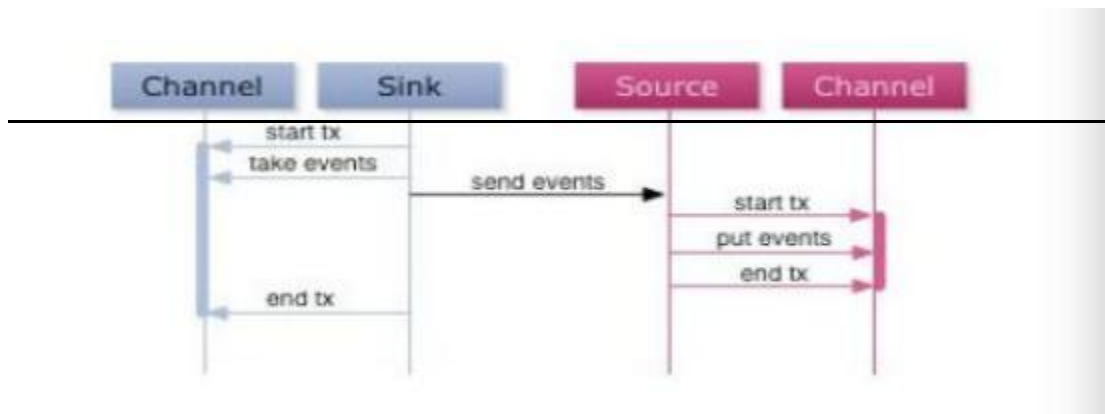
一致性（**consistency**）。事务必须是使数据库从一个一致性状态变到另一个一致性状态。一致性与原子性是密切相关的。

隔离性（**isolation**）。一个事务的执行不能被其他事务干扰。即一个事务内部的操作及使用的数据对并发的其他事务是隔离的，并发执行的各个事务之间不能互相干扰。

持久性（**durability**）。持久性也称永久性（**permanence**），指一个事务一旦提交，它对数据库中数据的改变就应该是永久性的。接下来的其他操作或故障不应该对其有任何影响。

2、结合具体项目或者应用，说一说事务，比如 flume

`Flume` 使用事务的办法来保证 `event` 的可靠传递。`Source` 和 `Sink` 分别被封装在事务中，这些事务由保存 `event` 的存储提供或者由 `Channel` 提供。这就保证了 `event` 在数据流的点对点传输中是可靠的。在多级数据流中，如下图，上一级的 `Sink` 和下一级的 `Source` 都被包含在事务中，保证数据可靠地从一个 `Channel` 到另一个 `Channel` 转移。



4.gc 算法

常见 GC 算法有哪些？

参考答案：

- 1) 复制收集算法
- 2) 标记清除算法
- 3) 标记整理算法

5.悲观锁与乐观锁

说说悲观锁和乐观锁？

参考答案：

- 1、悲观锁（Pessimistic Lock）：每次拿数据的时候都会担心会被别人修改（疑心重很悲观），所以每次在拿数据的时候都会上锁。确保自己使用的过程中不会被别人访问，自己使用完后在再解锁。期间需要访问该数据的都会等待。
- 2、乐观锁（Optimistic Lock）：每次拿数据的时候都完全不担心会被别人修改（心态好很乐观），所以每次在拿数据的时候都不会上锁。但是在更新数据的时候去判断该期间是否被别人修改过（使用版本号等机制），期间该数据可以随便被其他人读取。
- 3、两种锁各有优缺点，不能单纯的定义哪个好于哪个。乐观锁比较适合数据修改比较少，读取比较频繁的场景，即使出现了少量的冲突，这样也省去了大量的锁的开销，故而提高了系统的吞吐量。但是如果经常发生冲突（写数据比较多的情况下），上层应用不断的 `retry`，这样反而降低了性能，对于这种情况使用悲观锁就更合适。

6.连接池

为什么要用连接池？连接池的原理是什么？

参考答案：

1、首先在 Web 开发中，如果使用 JDBC 连接数据库，那么每次访问请求都必须建立连接——打开数据库——存取数据库——关闭连接等一系列步骤。但是我们知道数据库的连接打开不仅费时，而且消耗比较多的系统资源。如果进行数据库操作的次数比较少，那么还不至于有多大的影响，但是假如频繁的进行数据库操作，那么系统的性能将会受到很大影响。

2、其次，是造成数据库连接泄漏。数据库连接泄漏，指的是如果在某次使用或者某段程序中没有正确地关闭 **Connection**、**Statement** 和 **ResultSet** 资源，那么每次执行都会留下一些没有关闭的连接，这些连接失去了引用而不能得到重新使用，因此就造成了数据库连接的泄漏。数据库连接的资源是宝贵而且是有限的，如果在某段使用频率很高的代码中出现这种泄漏，那么数据库连接资源将被耗尽，影响系统的正常运转。

3、为了解决上述问题，因此就引入了数据库连接池技术。用一句话概括数据库连接池技术那就是负责分配、管理和释放数据库连接。

4、连接池用于创建和管理数据库连接的缓冲池技术，缓冲池中的连接可以被任何需要他们的线程使用。当一个线程需要用 JDBC 对一个数据库操作时，将从池中请求一个连接。当这个连接使用完毕后，将返回到连接池中，等待为其他的线程服务。

7.多线程

怎么理解多线程的？

参考答案：

1) 提到多线程,必须先提到进程和线程两个概念,进程指的是系统中正在运行的一个应用程序,而进程想要执行任务,就必须要有线程,一个进程至少有一条线程,程序在启动的时候会默认开启一条线程,我们称为主线程.

2) 多线程的优点 一个进程可以开启多条线程,每个线程执行不同的任务.它可以适当的提高程序的运行效率和资源利用率.一般我们在开发中将耗时任务放到后台去处理,例如网络请求数据,文件读写,用户输入等.此外可以使得界面更加吸引人,比如用户在点击了一个按钮去触发某些事件的处理,可以弹出进度条来显示处理的进程,减少用户等待的焦虑心理.

3) 多线程的缺点 但是线程并不是开的越多越好,因为多线程的同时执行,是 CPU 在多条线程之间来回切换造成的,由于切换速度太快,造成了多条线程同时执行的感觉.如果线程开启的过多,一方面会占用大量的内存空间,另一方面,CPU 在线程之间来回切换也会消耗大量 CPU 资源.此外,线程过多也会使得程序的设计更加复杂,比如线程之间的通信和多线程数据的共享.

8.Java 设计模式

java 有哪些设计模式?单例模式有哪些特点?

参考答案:

1) 总体来说设计模式分为三大类: 创建型模式, 共五种: 工厂方法模式、抽象工厂模式、单例模式、建造者模式、原型模式。 结构型模式, 共七种: 适配器模式、装饰器模式、代理模式、外观模式、桥接模式、组合模式、享元模式。 行为型模式, 共十一种: 策略模式、模板方法模式、观察者模式、迭代子模式、责任链模式、命令模式、备忘录模式、状态模式、访问者模式、中介者模式、解释器模式。 其实还有两类: 并发型模式和线程池模式。

2) java 单例模式是一种常见的设计模式, 单例模式有以下特点:

- a) 单例类只能有一个实例。
- b) 单例类必须自己创建自己的唯一实例。
- c) 单例类必须给所有其他对象提供这一实例。

9.Java 访问控制权限

说说 Java 的几个访问控制权限?

参考答案:

Java 中的访问控制权限有 4 种, 按照权限从大到小依次为:

`public` -> `protected` -> 包访问权限 (没有权限修饰词) -> `private`。 (1) 使用 `public` 关键字, 就意味着被声明的成员或方法对所有人都是可以访问的。 (2) `protected` 也提供包访问权限, 也就是说, 派生类以及相同包内的其他类都可以访问 `protected` 成员或方法。例如: 子类继承父类后可以访问父类的 `protected` 成员。 (3) 使用 `private` 关键字, 就意味着被声明的成员或方法, 除了本类, 其他任何类都无法访问。

10.hashmap 与 hashtable 的区别与优缺点

HashMap、Hashtable 的区别及其优缺点?

参考答案:

1) Hash Map 不是线程安全的。HashMap 是 map 接口的子类, 是将键映射到值的对象, 其中键和值都是对象, 并且不能包含重复键, 但可以包含重复值。HashMap 允许 null key 和 null value, 而 hashtable 不允许。

2) `HashTable` 是线程安全。`HashMap` 是 `Hashtable` 的轻量级实现（非线程安全的实现），他们都完成了 `Map` 接口，主要区别在于 `HashMap` 允许空（`null`）键值（`key`），由于非线程安全，效率上可能高于 `Hashtable`。

3) `HashMap` 允许将 `null` 作为一个 `entry` 的 `key` 或者 `value`，而 `Hashtable` 不允许。`HashMap` 把 `Hashtable` 的 `contains` 方法去掉了，改成 `containsvalue` 和 `containskey`。因为 `contains` 方法容易让人引起误解。`Hashtable` 继承自 `Dictionary` 类，而 `HashMap` 是 `Java1.2` 引进的 `Map` interface 的一个实现。最大的不同是，`Hashtable` 的方法是 `Synchronize` 的，而 `HashMap` 不是，在多个线程访问 `Hashtable` 时，不需要自己为它的方法实现同步，而 `HashMap` 就必须为之提供外同步。`Hashtable` 和 `HashMap` 采用的 `hash/rehash` 算法都大概一样，所以性能不会有很大差别。

11. `ArrayList` 和 `LinkedList` 的区别

`ArrayList` 和 `LinkedList` 的区别？

参考答案：

1) `ArrayList`、`Vector` 是实现了基于动态数组的数据结构，`LinkedList` 基于链表的数据结构。

2) 对于随机访问 `get` 和 `set`，`ArrayList` 觉得优于 `LinkedList`，因为 `LinkedList` 要移动指针。对于新增和删除操作 `add` 和 `remove`，`LinkedList` 比较占优势，因为 `ArrayList` 要移动数据。

3) `ArrayList`、`LinkedList` 是不同步的，而 `Vector` 是同步的。所以如果不要求线程安全的话，可以使用 `ArrayList` 或 `LinkedList`，可以节省为同步而耗费的开销。但在多线程的情况下，有时候就不得不使用 `Vector` 了。当然，也可以通过一些办法包装 `ArrayList`、`LinkedList`，使他们也达到同步，但效率可能会有所降低。

12. Java 中的重写和重载的区别

Java 中的重写和重载的区别？

参考答案：

重写方法的规则：

- 1、参数列表必须完全与被重写的方法相同，否则不能称其为重写而是重载。
- 2、返回的类型必须一直与被重写的方法的返回类型相同，否则不能称其为重写而是重载。
- 3、访问修饰符的限制一定要大于被重写方法的访问修饰符（`public`>`protected`>`default`>`private`）
- 4、重写方法一定不能抛出新的检查异常或者比被重写方法申明更加宽泛的检查型异常。例如：父类的一个方法申明了一个检查异常 `IOException`，在重写这个方法是不能抛出 `Exception`，只能抛出 `IOException` 的子类异常，可以抛出非检查异常。

重载的规则：

1、必须具有不同的参数列表；

2、可以有不同返回类型的返回类型，只要参数列表不同就可以了；

3、可以有不同访问修饰符； 4、可以抛出不同的异常； 重写与重载的区别在于：重写多态性起作用，对调用被重载过的方法可以大大减少代码的输入量，同一个方法名只要往里面传递不同的参数就可以拥有不同的功能或返回值。

13. 内存泄漏和内存溢出

内存泄漏和内存溢出有什么区别？

参考答案：

内存溢出: out of memory，是指程序在申请内存时，没有足够的内存空间供其使用，出现 out of memory；比如申请了一个 integer,但给它存了 long 才能存下的数，那就是内存溢出。 内存泄露: memory leak，是指程序在申请内存后，无法释放已申请的内存空间，一次内存泄露危害可以忽略，但内存泄露堆积后果很严重，无论多少内存,迟早会被占光。memory leak 最终会导致 out of memory！在 C/C++中可能会出现这类问题。

3.Linux 命令

1.shell 脚本应用

假设有 3 台主机，分别为 master、slave1、slave2，在开启 SSH 互信的情况下，编写一个或多个脚本实现在所有的远程主机上执行脚本的功能。

例如：runRemoteCmd.sh "jps" all

期望结果：

master:

XXXXXXXX

XXXXXXXX

XXXXXXXX

slave1:

XXXXXXXX

XXXXXXXX

XXXXXXXX

slave2:

XXXXXXXX

XXXXXXXX

XXXXXXXX

参考答案:

配置 deploy.conf

#规划集群角色

hadoop3-01, master, all,

hadoop3-02, slave, all,

hadoop3-03, slave, all,

脚本 runRemoteCmd.sh

#!/bin/bash

if [\$# -lt 2]

then

echo "Usage: ./runRemoteCmd.sh Command MachineTag"

echo "Usage: ./runRemoteCmd.sh Command MachineTag confFile"

exit

fi

cmd=\$1

tag=\$2

if ['a'\$3'a' == 'aa']

then

confFile=/home/hadoop/tools/deploy.conf

else

confFile=\$3

fi

if [-f \$confFile]

then

```

for server in `cat $confFile | grep -v '^#' |grep ','$tag'|' |awk -F',' '{print $1}'`
do
    echo "*****$server*****"
    ssh $server "source ~/.bashrc; $cmd"
done
else
    echo "Error: Please assign config file or run deploy.sh command with deploy.conf in same
directory"
fi

```

2.列举 Linux 常用命令

Linux 自带的常用命令至少举例 20 个？

参考答案：

Grep（查找） awk（解析） netstat（端口） free（内存） df（磁盘空间） ps（进程） jstat（查看 jvm GC）等等

3.Linux 内存使用

Linux 如何查看内存使用情况？

参考答案：

在 Linux 下查看内存我们一般用 free 命令：

```

[root@hadoop tmp]# free
total          used          free          shared
buffers      cached Mem:    3266180    3250004    16176          0
110652        2668236 -/+ buffers/cache:  471116    2795064 Swap:    2048276
80160         1968116

```

下面是对这些数值的解释：

total:总计物理内存的大小。

used:已使用多大。

free:可用有多少。

Shared:多个进程共享的内存总额。

Buffers/cached:磁盘缓存的大小。

4.Linux 系统常用配置文件

请列举出曾经修改过的/etc/下面的文件，并说明修改要解决什么问题？

参考答案：

- 1) `/etc/profile` 这个文件，主要是用来配置环境变量。让 `hadoop` 命令可以在任意目录下面执行。
- 2) `/etc/sudoers` `sudo` 是 `linux` 下常用的允许普通用户使用超级用户权限的工具。
- 3) `/etc/hosts` 配置 `ip` 与 `hostname` 之间的关系
- 4) `/etc/sysconfig/network` 修改 `hostname` 主机名称

5.shell 查询关键词

比如有一个文件 `deploy.conf`，如何使用 `shell` 脚本查询关键词'`slave`'？

参考答案：

`grep 'slave' deploy.conf`

6.Linux 查看 GC

Linux 如何查看 `gc`？

参考答案：

通常运行命令如下：`jstat -gc 30996 3000` 即：每 3 秒一次显示进程号为 30996 的 `java` 进程的 `GC` 情况

或使用命令：`jstat -gcutil 30996 3000`

4.SQL 面试

1.行列转换

案例 1

业务场景（MySQL）：考试记录编码转名称

a) 要求 1：答案选项表只能关联一次

b) 要求 2：答案顺序不能错位，没有被选择项，置空

考试记录			
考试人	试卷名称	题目编号	答案编码
张三	exam-a	0101	ACD

答案选项表				
试卷名称	题号编号	题目名称	答案	名称
exam-a	0101	你最近 5 年的职业（可多选）	A	教师

exam-a	0101	你最近 5 年的职业（可多选）	B	医生
exam-a	0101	你最近 5 年的职业（可多选）	C	销售人员
exam-a	0101	你最近 5 年的职业（可多选）	D	白领

实现将考试记录表的编码，转换成对应的题目名称及答案名称，关联后结果如下：

考试人	试卷名称	题目编号	题目名称	答案 A	答案 B	答案 C	答案 D
张三	exam-a	0101	你最近 5 年的职业（可多选）	教师	NULL	销售人员	白领

参考答案：

1.创建考试记录表（exam_record）并初始化数据

```
DROP TABLE IF EXISTS `exam_record`;
```

```
CREATE TABLE `exam_record` (
```

```
  `username` varchar(20) DEFAULT NULL COMMENT '考试人',
  `exam_name` varchar(20) DEFAULT NULL COMMENT '试卷名称',
  `question_num` varchar(20) DEFAULT NULL COMMENT '题目编号',
  `answer_coding` varchar(20) DEFAULT NULL COMMENT '答案编码'
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

```
insert into `exam_record`(`username`,`exam_name`,`question_num`,`answer_coding`) values ('张三','exam-a','0101','ACD');
```

2.创建答案选项表（answer）并初始化数据

```
DROP TABLE IF EXISTS `answer`;
```

```
CREATE TABLE `answer` (
```

```
  `exam_name` varchar(20) DEFAULT NULL COMMENT '试卷名称',
  `question_num` varchar(20) DEFAULT NULL COMMENT '题目编号',
  `question_name` varchar(20) DEFAULT NULL COMMENT '题目名称',
  `answer` varchar(20) DEFAULT NULL COMMENT '答案',
  `answer_name` varchar(20) DEFAULT NULL COMMENT '名称'
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

```
insert into `answer`(`exam_name`,`question_num`,`question_name`,`answer`,`answer_name`) values ('exam-a','0101','你最近 5 年的职业（可多选）','A','教师'),('exam-a','0101','你最近 5 年的职业（可多选）','B','医生'),('exam-a','0101','你最近 5 年的职业（可多选）','C','销售人员'),('exam-a','0101','你最近 5 年的职业（可多选）','D','白领');
```

3.编写 SQL

```
select d.username,d.exam_name,d.question_num,d.question_name,
```

```

max(case d.result when 'A' then d.answer_name else 'NULL' end) '答案 A',
max(case d.result when 'B' then d.answer_name else 'NULL' end) '答案 B',
max(case d.result when 'C' then d.answer_name else 'NULL' end) '答案 C',
max(case d.result when 'D' then d.answer_name else 'NULL' end) '答案 D'
from
(select e.*,a.question_name,a.answer,a.answer_name,
(case locate(a.answer,e.answer_coding) when 0 then 'NULL' else a.answer end) as 'result'
from exam_record e join answer a on (e.exam_name=a.exam_name and
e.question_num=a.question_num) order by e.username) d group by d.username

```

4.测试运行实现效果

username	exam_name	question_num	question_name	答案A	答案B	答案C	答案D
张三	exam-a	0101	你最近5年的职业(可多选)	教师	NULL	销售人员	白领

案例 2

业务场景（MySQL）：

部门绩效表中，记录了每个部分每年的平均绩效考核成绩。

部门绩效表 dept

Dt（年份）	Dname（部门）	Score（绩效）
2021	研发部	8
2022	市场部	10
2021	市场部	9
2022	研发部	7

Q1：将 dept 表内容转换为如下输出结果

Dt	col_yf	col_sc
2021	8	9
2022	7	10

A1:

1.创建 dept 表并导入源数据

```
DROP TABLE IF EXISTS `dept`;
```

```
CREATE TABLE `dept` (
```

```
  `dt` varchar(20) DEFAULT NULL,
```

```
  `dname` varchar(20) DEFAULT NULL,
```

```
  `score` int(11) DEFAULT NULL
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

```
insert into `dept`(`dt`,`dname`,`score`) values ('2021','研发部',8),('2022','市场部',10),('2021','市场部',9),('2022','研发部',7);
```

2.编写 SQL（多行转多列）

```
create table dept_2 as
```

```

select
    dt,
    max(case when dname="市场部" then score end) col_sc,
    max(case when dname="研发部" then score end) col_yf
from dept
group by dt;

```

3.测试运行结果

	dt	col_sc	col_yf
☐	2021	9	8
☐	2022	10	7
*	(NULL)	(NULL)	(NULL)

Q2: 将 dept_2 表的结果转换为 dept (源表)

部门绩效表 dept

Dt (年份)	Dname (部门)	Score (绩效)
2021	研发部	8
2022	市场部	10
2021	市场部	9
2022	研发部	7

A2:

1.编写 SQL (多列转多行)

```

select
    dt,
    dname,
    score
from (
    select dt,"研发部" as dname,col_yf as score from dept_2
    union all
    select dt,"市场部" as dname,col_sc as score from dept_2
) a;

```

2.测试运行效果

	dt	dname	score
☐	2021	研发部	8
☐	2022	研发部	7
☐	2021	市场部	9
☐	2022	市场部	10

Q3:2021 年因为组织架构调整, 导致部门出现多个绩效, 业务及人员不同, 无法合并计算绩效

dept 表内容

Dt（年份）	Dname（部门）	Score（绩效）
2021	研发部	8
2022	市场部	10
2021	市场部	9
2022	研发部	7
2021	研发部	9

要求输出结果

Dt	col_yf	col_sc
2021	8,9	9
2022	7	10

A3:

1.创建 dept 表并初始化数据

DROP TABLE IF EXISTS `dept`;

CREATE TABLE `dept` (

 `dt` varchar(20) DEFAULT NULL,

 `dname` varchar(20) DEFAULT NULL,

 `score` int(11) DEFAULT NULL

) ENGINE=InnoDB DEFAULT CHARSET=utf8;

insert into `dept`(`dt`,`dname`,`score`) values ('2021','研发部',8),('2022','市场部',10),('2021','市场部',9),('2022','研发部',7),('2021','研发部',9);

2.编写 SQL（多行转多列）

```
select
  dt,
  max(case when dname='研发部' then score end) col_yf,
  max(case when dname='市场部' then score end) col_sc
from (
  select
    dt,
    dname,
    concat_ws(',',collect_set(score)) as score
  from dept
  group by dt,dname
)tmp group by dt;
```

3.测试运行效果

dt	col_yf	col_sc
2021	8,9	9
2022	7	10

2.排名取值

案例 1

业务场景（MySQL）：

公司员工绩效表中，记录了每个员工每年的绩效考核成绩。

员工绩效表 staff

dt	employee	Score
2021	张三	6
2021	李四	2
2021	王五	4
2022	张三	8
2022	赵六	3

Q1：按年份分组取每年绩效考核最差的员工。

A1：

1.创建 staff 表并初始化数据

```
DROP TABLE IF EXISTS `staff`;
```

```
CREATE TABLE `staff` (
```

```
  `dt` varchar(20) COLLATE utf8mb4_general_ci DEFAULT NULL,
```

```
  `employee` varchar(20) COLLATE utf8mb4_general_ci DEFAULT NULL,
```

```
  `score` int DEFAULT NULL
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
insert into `staff`(`dt`,`employee`,`score`) values ('2021','张三',6),('2021','李四',2),('2021','王五',4),('2022','张三',8),('2022','赵六',3);
```

2.编写 SQL

```
select
```

```
  dt,
```

```
  employee,
```

```
  score as min_score
```

```
from
```

```
(
```

```
  select
```

```
    dt,
```

```
    employee,
```

```
    score,
```

```
    row_number() over(partition by dt order by score) as rn
```

```
  from staff
```

```
)dt  
where rn = 1;
```

3.测试运行结果

	dt	employee	min_score
☐	2021	李四	2
☐	2022	赵六	3

Q2：按年份分组取每年绩效最差和最好的分数是多少？

A2：

1.编写 SQL

```
select  
    dt,  
    min(if(asc_rn = 1, score, null)) as min_score,  
    max(if(desc_rn = 1, score, null)) as max_score  
from  
(  
    select  
        dt,  
        employee,  
        score,  
        row_number() over(partition by dt order by score) as asc_rn,  
        row_number() over(partition by dt order by score desc) as desc_rn  
    from staff  
)dt  
where asc_rn = 1 or desc_rn = 1  
group by dt;
```

2.测试运行结果

	dt	min_score	max_score
☐	2021	2	6
☐	2022	3	8

3.累计求值

案例 1

业务场景（MySQL）：

现在有一个表 t_accumulate 及其内容如下所示。

dt	name	score
----	------	-------

2021	chinese	6
2021	math	2
2021	english	4
2022	chinese	8
2022	computer	6

Q1: 按 dt 分组，按 name 字段排序，对 score 累计求和

A1:

1.创建 t_accumulate 表并初始化数据

DROP TABLE IF EXISTS `t\_accumulate`;

CREATE TABLE `t\_accumulate` (

 `dt` varchar(20) COLLATE utf8mb4_general_ci DEFAULT NULL,

 `name` varchar(20) COLLATE utf8mb4_general_ci DEFAULT NULL,

 `score` int DEFAULT NULL

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

insert into `t\_accumulate`(`dt`,`name`,`score`) values

('2021','chinese',6),('2021','math',2),('2021','english',4),('2022','chinese',8),('2022','computer',6);

2.编写 SQL

select

 dt,

 name,

 score,

 sum(score) over(partition by dt order by name) as sum_score

from t_accumulate;

3.测试运行结果

	dt	name	score	sum_score
☐	2021	chinese	6	6
☐	2021	english	4	10
☐	2021	math	2	12
☐	2022	chinese	8	8
☐	2022	computer	6	14

Q:2: 按 dt 分组，按 name 字段排序，对 score 累计求平均值

A2:

1.编写 SQL

select

 dt,

 name,

```

score,
avg(score) over(partition by dt order by name) as avg_score
from t_accumulate;

```

2.测试运行结果

	dt	name	score	avg_score
☐	2021	chinese	6	6.0000
☐	2021	english	4	5.0000
☐	2021	math	2	4.0000
☐	2022	chinese	8	8.0000
☐	2022	computer	6	7.0000

Q3: 按 dt 分组，按 name 字段排序，对 name 累计求排名比例

A3:

1.编写 SQL

```

select
dt,
name,
score,
round(row_number() over(partition by dt order by name) / (count(score) over(partition by
dt)),2) as ratio_score
from t_accumulate
order by dt,name;

```

2.测试运行结果

	dt	name	score	ratio_score
☐	2021	chinese	6	0.33
☐	2021	english	4	0.67
☐	2021	math	2	1.00
☐	2022	chinese	8	0.50
☐	2022	computer	6	1.00

Q1: 按 dt 分组，按 name 字段排序，对 name 取累计求和比例

A1:

1.编写 SQL

```

select
dt,
name,
score,
round(sum(score) over(partition by dt order by name) / (sum(score) over(partition by dt)),2) as
ratio_c

```

```
from t_accumulate
order by dt,name;
```

2.测试运行结果

	dt	name	score	ratio_c
☐	2021	chinese	6	0.50
☐	2021	english	4	0.83
☐	2021	math	2	1.00
☐	2022	chinese	8	0.57
☐	2022	computer	6	1.00

案例 2

业务场景（MySQL）：

用 2 种方法，分别用一句 sql 实现下列统计逻辑的实现。

表 T

YF	RQ	SUDO
2018-01-01	2018-01-01	3
2018-01-01	2018-01-02	4
2018-01-01	2018-01-03	5
2018-01-01	2018-01-04	4
.....		
2018-02-01	2018-02-01	2
2018-02-01	2018-02-02	4

结果统计

YF	RQ	SUDO
2018-01-01	2018-01-01	3
2018-01-01	2018-01-02	7
2018-01-01	2018-01-03	12
2018-01-01	2018-01-04	16
.....		
2018-02-01	2018-02-01	2
2018-02-01	2018-02-02	6

参考答案：

1.创建 T 表并导入数据源

```
DROP TABLE IF EXISTS `T`;
```

```
CREATE TABLE `T` (
```

```
  `YF` varchar(20) DEFAULT NULL,
```

```
  `RQ` varchar(20) DEFAULT NULL,
```

```
  `SUDO` int(11) DEFAULT NULL
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

```
insert into `T`(`YF`,`RQ`,`SUDO`) values
('2018-01-01','2018-01-01',3),('2018-01-01','2018-01-02',4),('2018-01-01','2018-01-03',5),('2018-01-01','2018-01-04',4),('2018-02-01','2018-02-01',2),('2018-02-01','2018-02-02',4);
```

2.编写 sql

方式一：

```
select YF,RQ,SUDO,
(select sum(SUDO) from T as t1 where t1.RQ<=t2.RQ and t1.YF=t2.YF)
as total from T as t2;
```

方式二：

```
select b.YF,b.RQ,b.SUDO,sum(a.SUDO) as total from T a,T b where a.RQ<=b.RQ and a.YF=b.YF
group by RQ
```

3.测试运行结果

	YF	RQ	SUDO	total
☐	2018-01-01	2018-01-01	3	3
☐	2018-01-01	2018-01-02	4	7
☐	2018-01-01	2018-01-03	5	12
☐	2018-01-01	2018-01-04	4	16
☐	2018-02-01	2018-02-01	2	2
☐	2018-02-01	2018-02-02	4	6

4.窗口计算

案例 1

业务场景（MySQL）：

现在有一个表 `t_window` 及其内容如下所示。

dt	name	score
2021	chinese	6
2021	math	2
2021	english	4
2022	chinese	8
2022	computer	6

Q1：按 `dt` 分组，按 `name` 字段排序，对 `score` 取前后各一行的和。

A1：

1.创建 `t_window` 表并初始化数据

```
DROP TABLE IF EXISTS `t_window`;
```

```
CREATE TABLE `t_window` (
  `dt` varchar(20) COLLATE utf8mb4_general_ci DEFAULT NULL,
  `name` varchar(20) COLLATE utf8mb4_general_ci DEFAULT NULL,
  `score` int DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
insert into `t_window`(`dt`,`name`,`score`) values
('2021','chinese',6),('2021','math',2),('2021','english',4),('2022','chinese',8),('2022','computer',6);
```

2. 编写 SQL

```
select
  dt,
  name,
  lag(score,1,0) over(partition by dt order by name)+lead(score,1,0) over(partition by dt order by
name) as sum_score
from t_window;
```

2. 测试运行结果

	dt	name	sum_score
☐	2021	chinese	4
☐	2021	english	8
☐	2021	math	4
☐	2022	chinese	6
☐	2022	computer	8

Q2: 按 dt 分组，按 name 字段排序，对 score 取前一行和当前行的平均值

A1:

1. 编写 SQL

```
select
  dt,
  name,
  avg(score) over(partition by dt order by name rows between 1 preceding and current row )
as avg_score
from t_window;
```

2. 测试运行结果

	dt	name	avg_score
☐	2021	chinese	6.0000
☐	2021	english	5.0000
☐	2021	math	3.0000
☐	2022	chinese	8.0000
☐	2022	computer	7.0000

案例 2

业务场景（MySQL）：

手写 SQL 统计用户表中，连续登入 7 日的用户。

数据格式

第一列(uid)：用户 id

第二列(dt)：登录时间

第三列(status)：用户状态

示例数据

1,2022-05-11,1

1,2022-05-12,1

1,2022-05-13,1

1,2022-05-14,1

1,2022-05-15,1

1,2022-05-16,1

1,2022-05-17,1

1,2022-05-18,1

2,2022-05-11,1

2,2022-05-12,1

2,2022-05-13,0

2,2022-05-14,1

2,2022-05-15,1

2,2022-05-16,0

2,2022-05-17,1

2,2022-05-18,0

3,2022-05-11,1

3,2022-05-12,1

3,2022-05-13,1

3,2022-05-14,0

3,2022-05-15,1

3,2022-05-16,1

3,2022-05-17,1

3,2022-05-18,1

参考答案：

1.创建 user 表并导入原始数据

```
DROP TABLE IF EXISTS `ulogin`;
```

```
CREATE TABLE `ulogin` (
```

```
  `uid` int(11) DEFAULT NULL,
```

```
  `dt` varchar(20) DEFAULT NULL,
```

```
  `status` int(11) DEFAULT NULL
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

```
insert
```

```
into
```

```
`ulogin`(`uid`,`dt`,`status`)
```

```
values
```

(1,'2022-05-11',1),(1,'2022-05-12',1),(1,'2022-05-12',1),(1,'2022-05-14',1),(1,'2022-05-15',1),(1,'2022-05-16',1),(1,'2022-05-17',1),(1,'2022-05-18',1),(2,'2022-05-11',1),(2,'2022-05-12',1),(2,'2022-05-13',0),(2,'2022-05-14',1),(2,'2022-05-15',1),(2,'2022-05-16',0),(2,'2022-05-17',1),(2,'2022-05-18',0),(3,'2022-05-11',1),(3,'2022-05-12',1),(3,'2022-05-13',1),(3,'2022-05-14',0),(3,'2022-05-15',1),(3,'2022-05-16',1),(3,'2022-05-17',1),(3,'2022-05-18',1);

2.编写 SQL

```
select uid,count(gid) from (select uid, dt, date_sub(dt, row_number() over (partition by uid order by dt)) gid from ulogin where status=1) as u group by uid,gid having count(gid)>=7;
```

备注：row_number()函数在 MySQL8.0 版本才有。

3.测试运行结果

用户 uid	连续登录次数
1	8

5.扩展与炸裂

案例

业务场景（Hive）：

现在有一张表 t_add_dec 及其内容如下所示。

digit
9
5
2
7

Q1：每行增加一列，输出内容为小于 digit 字段的数字集合

A1:

1.创建表 t_add_dec

```
create table if not exists t_add_desc
```

```
(digit int)
```

```
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
```

```
STORED AS TEXTFILE ;
```

2.加载数据集至 t_add_dec 表

```
#原始数据 t_add_dec.log
```

```
9
```

```
5
```

2

7

#将 t_add_dec.log 文件加载至 t_add_dec 表

load data local inpath '/home/hadoop/app/hive/t_add_dec.log' overwrite into table t_add_dec;

#查询 t_add_dec 表

hive> select * from t_add_dec ;

3.编写 SQL

set hive.strict.checks.cartesian.product=false;

select

 t.digit,

 concat_ws('、',collect_set(cast(t.rn as string))) as b

from

(

 select

 t_add_desc.digit,

 b.rn

 from t_add_desc

 left join

 (

 select

 row_number() over() as rn

 from

 (select split(space(5), ' ') as x) t

 lateral view

 explode(x) pe

) b

 on 1 = 1

 where t_add_desc.digit >= b.rn

 order by t_add_desc.digit, b.rn desc

) t

group by t.digit;

4.测试运行结果



```
Total MapReduce CPU Time Spent: 17 seconds 460 msec
OK
2      2、1
5      5、4、3、2、1
7      6、5、4、3、2、1
9      6、5、4、3、2、1
Time taken: 236.002 seconds, Fetched: 4 row(s)
```

Q2: 如何将字符串"1-6,7,8-14,15"炸开, 生成字符串为"1,2,3,4,5,6,7,8,9,10,11,12,13,14,15", 同时保持顺序不变。

A2:

1.编写 SQL

```
select
    concat_ws(',',collect_list(cast(rn as string)))
from
(
    select
        a.rn,
        b.num,
        b.pos
    from
    (
        select
            row_number() over() as rn
        from (select split(space(20), ' ') as x) t
        lateral view
        explode(x) pe
    ) a lateral view outer
    posexplode(split('1-6,7,8-14,15', ',')) b as pos, num
    where a.rn between cast(split(num, '-')[0] as int) and cast(split(num, '-')[1] as int) or a.rn =
    num
    order by pos, rn
) t;
```

2.测试运行结果

```
Total MapReduce CPU Time Spent: 18 seconds 380 msec
OK
1,2,3,4,5,6,7,8,9,10,11,12,13,14,15
Time taken: 111.826 seconds, Fetched: 1 row(s)
hive>
```

6.合并与分裂

案例

应用场景（Hive）：

现在有一张表 t_compact_split 及其内容如下所示。

dt	name
2021	张三
2021	李四

2022	李四
2022	王五

Q1: 按照 dt 字段进行分组，将相同年份的 name 值合并在一起。

A1:

1.创建表 t_compact_split

```
create table if not exists t_compact_split
```

```
(dt string,name string)
```

```
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
```

```
STORED AS TEXTFILE ;
```

2.加载数据集至 t_compact_split 表

#原始数据集 t_compact_split.log

```
[hadoop@hadoop1 hive]$ vi t_compact_split.log
```

```
2021,张三
```

```
2021,李四
```

```
2022,李四
```

```
2022,王五
```

#将数据集 t_compact_split.log 加载至 t_compact_split 表

```
hive>load data local inpath '/home/hadoop/app/hive/t_compact_split.log' overwrite into table
t_compact_split;
```

#查询 t_compact_split 表

```
hive> select * from t_compact_split;
```

3.编写 SQL

```
select
```

```
dt,
```

```
concat_ws('、', collect_set(name)) name
```

```
from t_compact_split
```

```
group by dt;
```

4.测试运行结果

```
Total MapReduce CPU Time Spent: 6 seconds 560 msec
OK
2021    张三、李四
2022    李四、王五
Time taken: 46.551 seconds, Fetched: 2 row(s)
hive>
```

Q2: 将 name 字段的值炸裂开，然后逐行按照 dt 字段进行输出。

A1.

1.编写 SQL

```
select t.dt,d from(select dt,concat_ws('、', collect_set(name)) name from t_compact_split group by dt)t lateral view explode(split(t.name, '、')) tmp as d;
```

2.测试运行结果

```
Total MapReduce CPU Time Spent: 6 seconds 868 msec
OK
2021    张三
2021    李四
2022    李四
2022    王五
Time taken: 48.037 seconds, Fetched: 4 row(s)
hive>
```

7.数据去重

案例

应用场景（Hive）：

现在有一张表 t_distinct 及其内容如下所示。

dt1	dt2	dt3	Name
2020	2022	2021	张三
2020	2021	2021	李四

Q1：每行数据先将时间字段炸开，循环输出时间和 name，同时对数据去重，但不能使用 distinct 和 group by。

A1:

1.创建表 t_distinct

```
create table if not exists t_distinct
(dt1 string,dt2 string,dt3 string,name string)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE ;
```

2.加载数据集至 t_distinct 表

#原始数据集 t_distinct.log

```
[hadoop@hadoop1 hive]$ vi t_distinct.log
```

```
2020,2022,2021,张三
```

```
2020,2021,2021,李四
```

#将数据集 t_distinct.log 加载至 t_distinct 表

```
hive>load data local inpath '/home/hadoop/app/hive/t_distinct.log' overwrite into table
t_distinct;
```

#查询 t_distinct 表

```
hive> select * from t_distinct;
```

3.编写 SQL

```
select
    t2.year
    ,t2.name
from
    (
        select
            *
            ,row_number() over (partition by t1.year,t1.name) as rank_num
        from
            (
                select
                    dt1 as year,
                    name
                from t_distinct
                union all
                select
                    dt2 as year,
                    name
                from t_distinct
                union all
                select
                    dt3 as year,
                    name
                from t_distinct
            )t1
        )t2
where rank_num=1
order by name;
```

4.测试运行结果

```
Total MapReduce CPU Time Spent: 25 seconds 690 msec
OK
2022    张三
2021    张三
2020    张三
2021    李四
2020    李四
Time taken: 161.829 seconds, Fetched: 5 row(s)
hive>
```

8.多表连接

案例

业务场景（MySQL）：

使用 sql 查询出所有有音乐、美术特长的学生，数据如下所示。

姓名	特长
张三	美术
张三	音乐
李四	篮球
小明	美术
李四	美术
小明	音乐

参考答案：

1.创建学生表（stu）并初始化数据

```
DROP TABLE IF EXISTS `stu`;
```

```
CREATE TABLE `stu` (
```

```
  `name` varchar(20) DEFAULT NULL COMMENT '姓名',
```

```
  `speciality` varchar(20) DEFAULT NULL COMMENT '特长'
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

```
insert into `stu`(`name`,`speciality`) values ('张三','美术'),('张三','音乐'),('李四','篮球'),('小明','美术'),('李四','美术'),('小明','音乐');
```

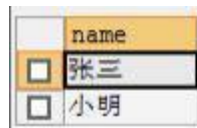
3. 编写 sql（自连接）

```
select name from
```

```
(select s1.name,s1.speciality as speciality1,s2.speciality as speciality2 from stu s1 join stu s2 on s1.name=s2.name) a
```

```
where a.speciality1='音乐' and a.speciality2='美术'
```

3.测试运行效果



name
张三
小明

9.SQL 优化

如何优化 SQL？数据库索引的作用？它的优点和缺点是什么？

参考答案：

1.SQL 优化方式很多，可以选择常见的 3~5 种即可

1)任何地方都不要使用 `select * from t`，用具体的字段列表代替“*”，不要返回用不到的任何字段。

2)尽量避免向客户端返回大数据量，若数据量过大，应该考虑相应需求是否合理。

3)不要在 `where` 子句中的“=”左边进行函数、算术运算或其他表达式运算，否则系统将可能无法正确使用索引。

4)应尽量避免在 `where` 子句中对字段进行表达式操作，这将导致引擎放弃使用索引而进行全表扫描。如：

```
select id from t where num/2=100
```

应改为：

```
select id from t where num=100*2
```

5)应尽量避免在 `where` 子句中使用 `or` 来连接条件，否则将导致引擎放弃使用索引而进行全表扫描

2.数据库索引可以提供 SQL 查询效率

3.

1) 优点

1.通过创建唯一性索引，可以保证数据库表中每一行数据的唯一性。

2.可以大大加快数据的检索速度，这也是创建索引的最主要的原因。

3.可以加速表和表之间的连接，特别是在实现数据的参考完整性方面特别有意义。

4.在使用分组和排序子句进行数据检索时，同样可以显著减少查询中分组和排序的时间。

5.通过使用索引，可以在查询的过程中，使用优化隐藏器，提高系统的性能。

2) 缺点

1.创建索引和维护索引要耗费时间，这种时间随着数据量的增加而增加。

2.索引需要占物理空间，除了数据表占数据空间之外，每一个索引还要占一定的物理空间，如果要建立聚集索引那么需要的空间就会更大。

3.当对表中的数据进行增加、删除和修改的时候，索引也要动态的维护，这样就降低了数据的维护速度。

10.数据库连接

数据库左连接和内连接的区别？

参考答案：

内连接（典型的连接运算，使用像 `=` 或 `<>` 之类的比较运算符）。包括相等连接和自然连接。

内连接使用比较运算符根据每个表共有的列的值匹配两个表中的行。例如，检索 `students` 和 `courses` 表中学生标识号相同的所有行。

左连接是一个外连接。外连接可以是左（向外）连接、右（向外）连接或完整外部连接。

11.数据库索引

谈谈你对数据库索引的理解？

参考答案：

因为数据中的记录很多，为了方便我们查找，提高查询的效率，可以使用索引；

索引的原理：

对要查询的字段建立索引其实就是把该字段按照一定的方式排序；建立的索引只对该字段有用，如果查询的字段改变，那么这个索引也就无效了，比如图书馆的书是按照书名的第一个字母排序的，那么你想要找作者叫张三的就不能用改索引了；还有就是如果索引太多会降低查询的速度

索引是优缺点：

首先明白为什么索引会增加速度，DB 在执行一条 Sql 语句的时候，默认的方式是根据搜索条件进行全表扫描，遇到匹配条件的就加入搜索结果集合。如果我们对某一字段增加索引，查询时就会先去索引列表中一次定位到特定值的行数，大大减少遍历匹配的行数，所以能明显增加查询的速度。

那么在什么时候都应该加索引么？这里有几个反例：

- 1、如果每次都需要取到所有表记录，无论如何都必须进行全表扫描了，那么是否加索引也没有意义了。
- 2、对非唯一的字段，例如“性别”这种大量重复值的字段，增加索引也没有什么意义。
- 3、对于记录比较少的表，增加索引不会带来速度的优化反而浪费了存储空间，因为索引是需要存储空间的，而且有个致命缺点是对于 `update/insert/delete` 的每次执行，字段的索引都必须重新计算更新。所以并不是任何情况下都改建立索引的。

12.数据库视图

谈谈你对数据库视图的理解？

参考答案：

数据库中视图的概念：

视图是从一个表或是多个表导出的表，视图与表不同，视图是一个虚表，即视图所对应的数据不进行实际存储，数据库中只存储视图的定义，在对视图的数据进行操作时，系统根据视图的定义去操作与视图相关联的基本表。

视图是虚表，是从一个或是几个基本表导出的表。可以将视图看做是移动的窗口，通过它可以看到自己感兴趣的数据，视图是从一个或是多个表中获得的，这些表的数据存放在数据库

中，那些用户产生视图的表叫做视图的基表，当然也可以从另一个视图中产生视图。

视图看上非常像是数据库的物理表，对它的操作同任何其他的表一样。当通过视图修改数据时候，实际上是改变基表中的数据，相反的，基表数据的改变也会自动反应在由基表产生的视图中。

13.数据库范式

阐述数据库的三大范式？

第一范式（1NF）：数据表中的每一列（每个字段）必须是不可拆分的最小单元，也就是确保每一列的原子性；

第二范式（2NF）：满足 1NF 后，要求表中的所有列，都必须依赖于主键，而不能有任何一列与主键没有关系，也就是说一个表只描述一件事情；

第三范式（3NF）：必须先满足第二范式（2NF），要求：表中的每一列只与主键直接相关而不是间接相关，（表中的每一列只能依赖于主键）；

14.等值连接

什么是等值连接？

参考答案：

```
SELECT M.columnname.....,N.*_columnname....FROM left_table M , right_table N WHERE M.
columnname_join=N. columnname_join AND M.columnname=XXX ... .. AND
N.columnname=XXX...
```

1、ON 子句连接条件，不再与左连接或右连接的功效一样，除了作为两个表记录匹配的条件外，还会起到过滤记录的作用，若 left_table 中记录无法在 right_table 中找到对应的记录，则会被过滤掉。

2、WHERE 字句，不管是涉及表 left_table、表 right_table 上的限制条件，还是涉及两个表连接的条件，都会对记录集起到过滤作用，把不符合要求的记录刷选掉。

15.表查询与排序

有一张表 phone_mail，如下：

Phone mail

137 1@163.com

137 2@163.com

135 3@163.com

135 1@163.com

136 1@163.com

136 1@163.com

(1) 查出前 2 条数据

(2) 根据手机号进行排序

参考答案:

(1) 查出前 2 条数据 `Select * from t_phone_mail limit 2;`

(2) 根据手机号进行排序 `Select phone,mail from t_phone_mail Distribute by mail Sort by phone;`

16.orderby 与 groupby 的区别

sql 语句中 order by 和 group by 的区别?

参考答案:

group by 是分组语句, 主要用于统计、合计等等;

order by 是排序语句。在一条 select 语句中 group by 语句的顺序要在 order by 之前。

17.sql 实现分页

sql 语句怎么实现分页? 请举例说明?

参考答案:

Mysql 使用 limit 进行分页, `select * from user limit 0,10;` 0 是开始的位置, 10 是页容量, 这是查询第一页的记录, 这一页最多有 10 条记录。

查询第二页则是 `select * from user limit 10,10;` 相当于 `select * from user limit (page-1)*pageSize,pageSize;`

5.Zookeeper 面试

1.Zookeeper 集群迁移

如何迁移大数据平台中的 Zookeeper 集群？

参考答案：

- 1、先将新增节点添加到原先的 Zookeeper 集群中同步数据
- 2、然后再将原先 Zookeeper 集群中的节点关闭掉即可

2.Zookeeper 应用场景

zookeeper 有哪些常用的应用场景？

参考答案：

- 1) 数据发布订阅
- 2) 统一命令服务
- 3) 分布式通知与协调
- 4) 分布式锁
- 5) 集群监控
- 6) 分布式队列
- 7) master 选举

3.Zookeeper 节点个数

Zookeeper 集群节点数，为什么使用奇数个，而不使用偶数个？

参考答案：

- 1) 所谓的偶数问题其实是另一个集群优化配置问题，即：集群的容灾数量=集群总节点数/2-1
- 2) 假如集群有 5 节点，那么最多允许 2 个节点挂掉，如果有 3 节点挂了，那么整个集群的选举结果不会满足条件：集群中超过半数的机器拥护。
- 3) 假如集群有 6 个节点，那么最多也只能挂掉 2 台，因为挂了 3 台时，选举结果也不会满足条件：集群中超过半数的机器拥护。
- 4) 结果可以看出，多那一台用处并不大。所以集群总数推荐为奇数。

4.Zookeeper 提供的服务

Zookeeper 提供了什么来实现各种服务？

参考答案：

- 1.文件系统
- 2.通知机制

5.zab 协议

谈谈你对 ZAB 协议的理解？

参考答案：

ZAB 协议是为分布式协调服务 Zookeeper 专门设计的一种支持崩溃恢复的原子广播协议。

ZAB 协议包括两种基本的模式：崩溃恢复和消息广播。

当整个 zookeeper 集群刚刚启动或者 Leader 服务器宕机、重启或者网络故障导致不存在过半的服务器与 Leader 服务器保持正常通信时，所有进程（服务器）进入崩溃恢复模式，首先选举产生新的 Leader 服务器，然后集群中 Follower 服务器开始与新的 Leader 服务器进行数据同步，当集群中超过半数机器与该 Leader 服务器完成数据同步之后，退出恢复模式进入消息广播模式，Leader 服务器开始接收客户端的事务请求生成事物提案来进行事务请求处理。

6.Zookeeper 节点类型

Zookeeper 包含哪四种类型的数据节点 Znode？

参考答案：

在 Zookeeper3.5 版本之前，Zookeeper 服务端包含 4 种节点类型。

1.临时与持久

所谓临时节点，就是当创建临时节点的客户端会话结束时，Zookeeper 会自动将该临时节点删除。

持久节点不依赖与客户端会话是否结束，当客户端会话结束，持久节点仍然存在。只有当客户端明确使用命令或者 API 执行删除操作，持久节点才会真正的消失。

2.持久顺序与临时顺序

这两种节点类型相对于临时与持久节点的特性，就是 Zookeeper 会自动在这两种节点之后增加一个数字的后缀，而路径 + 数字后缀能够保证 Znode 路径是唯一的，因为这个数字是由一个单调递增的计数器所添加的，由父节点维护。这种数字后缀的节点类型可以实现诸如分布式队列、分布式公平锁等应用场景。

在 Zookeeper3.6.2 版本之后，又增加了 3 种节点类型。

1.容器

容器节点的表现形式和持久节点是一样的，但区别是 Zookeeper 服务端启动后，会有一个单独的线程去扫描所有的容器节点，当发现容器节点的子节点数量为 0 时，会自动删除该节点，除此之外和持久节点没有区别。容器节点主要应用在 Leader、锁等应用场景中。

2.持久 TTL 与持久顺序 TTL

关于持久和顺序这两个关键字，这里就不再赘述了，这两种类型的节点重点是后面的 TTL。

TTL 是 time to live 的缩写，指带有存活时间。简单来说就是当该 Znode 节点下面没有子节点时，超过了 TTL 指定时间后，Znode 节点就会被自动删除。这两种节点类型的特性跟容器节点很像，只是容器节点没有 TTL 的概念。另外如果要启用 TTL 需要额外的进行配置，否则创建 TTL 时会报错。

因此当使用 Zookeeper3.6.2 以后的版本时，Zookeeper 服务端支持 7 种节点类型：持久、持久顺序、临时、临时顺序、容器、持久 TTL、持久顺序 TTL。

7.Zookeeper 监听机制

谈谈你对 Zookeeper Watcher 机制理解？

参考答案：

Zookeeper 允许客户端向服务端的某个 Znode 注册一个 Watcher 监听，当服务端的一些指定事件触发了这个 Watcher，服务端会向指定客户端发送一个事件通知来实现分布式的通知功能，然后客户端根据 Watcher 通知状态和事件类型做出业务上的改变。

8.Zookeeper 服务角色

Zookeeper 服务器有哪些角色？

参考答案：

1.Leader

事务请求的唯一调度和处理者，保证集群事务处理的顺序性
集群内部各服务的调度者

2.Follower

处理客户端的非事务请求，转发事务请求给 Leader 服务器
参与事务请求 Proposal 的投票
参与 Leader 选举投票

3.Observer

3.3.0 版本以后引入的一个服务器角色，在不影响集群事务处理能力的基础上提升集群的非事务处理能力

处理客户端的非事务请求，转发事务请求给 Leader 服务器
不参与任何形式的投票

9.Zookeeper Server 工作状态

Zookeeper 下 Server 工作状态有哪些？

参考答案：

服务器具有四种状态，分别是 LOOKING、FOLLOWING、LEADING、OBSERVING。

LOOKING：寻找 Leader 状态。当服务器处于该状态时，它会认为当前集群中没有 Leader，因此需要进入 Leader 选举状态。

FOLLOWING：跟随者状态。表明当前服务器角色是 Follower。

LEADING：领导者状态。表明当前服务器角色是 Leader。

OBSERVING：观察者状态。表明当前服务器角色是 Observer。

10.Zookeeper 事务

zookeeper 是如何保证事务的顺序一致性的？

参考答案：

zookeeper 采用了全局递增的事务 Id 来标识，所有的 proposal（提议）都在被提出的时候加上了 zxid，zxid 实际上是一个 64 位的数字，高 32 位是 epoch 用来标识 leader 周期，如果有新的 leader 产生出来，epoch 会自增，低 32 位用来递增计数。当新产生 proposal 的时候，会依据数据库的两阶段过程，首先会向其他的 server 发出事务执行请求，如果超过半数的机器都能执行并且能够成功，那么就会开始执行。

11.分布式集群的 Master

分布式集群中为什么会有 Master？

参考答案：

在分布式环境中，有些业务逻辑只需要集群中的某一台机器进行执行，其他的机器可以共享这个结果，这样可以大大减少重复计算，提高性能，于是就需要进行 leader 选举。

12.Zookeeper 节点宕机

zk 节点宕机如何处理？

参考答案：

Zookeeper 本身也是集群，推荐配置不少于 3 个服务器。Zookeeper 自身也要保证当一个节

点宕机时，其他节点会继续提供服务。

如果是一个 Follower 宕机，还有 2 台服务器提供访问，因为 Zookeeper 上的数据是有多个副本的，数据并不会丢失：

如果是一个 Leader 宕机，Zookeeper 会选举出新的 Leader。

ZK 集群的机制是只要超过半数的节点正常，集群就能正常提供服务。只有在 ZK 节点挂得太多，只剩一半或不到一半节点能工作，集群才失效。

所以

3 个节点的 cluster 可以挂掉 1 个节点(leader 可以得到 2 票 >1.5)

2 个节点的 cluster 就不能挂掉任何 1 个节点了(leader 可以得到 1 票 ≤ 1)

13.Zookeeper 部署模式

Zookeeper 有哪几种部署模式？

参考答案：

部署模式：单机模式、伪集群模式、集群模式。

14.Zookeeper 集群最少节点数

集群最少要几台机器，集群规则是怎样的？

参考答案：

集群规则为 $2N+1$ 台， $N>0$ ，即 3 台。

15.Zookeeper 支持动态添加节点么

Zookeeper 集群支持动态添加机器吗？

参考答案：

其实就是水平扩容了，有两种方式：

全部重启：关闭所有 Zookeeper 服务，修改配置之后启动。不影响之前客户端的会话。

逐个重启：在过半存活即可用的原则下，一台机器重启不影响整个集群对外提供服务。这是比较常用的方式。

3.5 版本开始支持动态扩容。

16.Zookeeper 监听节点是永久的吗

Zookeeper 对节点的 watch 监听通知是永久的吗？为什么不是永久的？

参考答案：

不是。

官方声明：一个 Watch 事件是一个一次性的触发器，当被设置了 Watch 的数据发生了改变的时候，则服务器将这个改变发送给设置了 Watch 的客户端，以便通知它们。

为什么不是永久的，举个例子，如果服务端变动频繁，而监听的客户端很多情况下，每次变动都要通知到所有的客户端，给网络和服务器造成很大压力。

一般是客户端执行 `getData(“/节点 A”,true)`，如果节点 A 发生了变更或删除，客户端会得到它的 watch 事件，但是在之后节点 A 又发生了变更，而客户端又没有设置 watch 事件，就不再给客户端发送。

在实际应用中，很多情况下，我们的客户端不需要知道服务端的每一次变动，我只要最新的数据即可。

17.Zookeeper Java 客户端有哪些

Zookeeper 的 java 客户端都有哪些？

参考答案：

java 客户端：zk 自带的 zkclient 及 Apache 开源的 Curator。

18.Zookeeper 与 Chubby

chubby 是什么，和 zookeeper 比你怎么看？

参考答案：

chubby 是 google 的，完全实现 paxos 算法，不开源。zookeeper 是 chubby 的开源实现，使用 zab 协议，paxos 算法的变种。

19.Zookeeper 常用命令

说几个 zookeeper 常用的命令。

参考答案：

常用命令：ls get set create delete 等。

20.zab 和 paxos 算法区别与联系

ZAB 和 Paxos 算法的联系与区别？

参考答案：

相同点：

两者都存在一个类似于 Leader 进程的角色，由其负责协调多个 Follower 进程的运行
Leader 进程都会等待超过半数的 Follower 做出正确的反馈后，才会将一个提案进行提交
ZAB 协议中，每个 Proposal 中都包含一个 epoch 值来代表当前的 Leader 周期，Paxos 中名字为 Ballot

不同点：

ZAB 用来构建高可用的分布式数据主备系统（Zookeeper），Paxos 是用来构建分布式一致性状态机系统。

21.Zookeeper 典型应用场景

Zookeeper 的典型应用场景有哪些？

参考答案：

Zookeeper 是一个典型的发布/订阅模式的分布式数据管理与协调框架，开发人员可以使用它来进行分布式数据的发布和订阅。

通过对 Zookeeper 中丰富的数据节点进行交叉使用，配合 Watcher 事件通知机制，可以非常方便的构建一系列分布式应用中都会涉及的核心功能，如：

- 1.数据发布/订阅
- 2.负载均衡
- 3.命名服务
- 4.分布式协调/通知
- 5.集群管理
- 6.Master 选举
- 7.分布式锁
- 8.分布式队列

22.Zookeeper 是什么

ZooKeeper 是什么？

参考答案：

ZooKeeper 是一个开放源码的分布式协调服务，它是集群的管理者，监视着集群中各个节点的状态根据节点提交的反馈进行下一步合理操作。最终，将简单易用的接口和性能高效、功能稳定的系统提供给用户。

分布式应用程序可以基于 Zookeeper 实现诸如数据发布/订阅、负载均衡、命名服务、分布式协调/通知、集群管理、Master 选举、分布式锁和分布式队列等功能。

23.Zookeeper 特性

Zookeeper 有哪些特性？

参考答案：

一致性、原子性、单一视图、可靠性、实时性。

24.Zookeeper 集群运行机制

Zookeeper 集群的机制是什么？

参考答案：

半数机制：集群中半数以上机器存活，集群可用。

25.Zookeeper 如何处理监听实现逻辑

Zookeeper 服务端处理 Watcher 实现

参考答案：

- 1.客户端向服务端注册 watcher，服务端接收 Watcher 并存储
- 2.Watcher 触发
- 3.调用 process 方法来触发 Watcher

6.HDFS 面试

1.HDFS 集群数据量计算

一个 hadoop 集群，hdfs 副本数为 3，此时存入 1G 数据。然后修改 hdfs 配置文件，将副本设置为 2，然后重启 hadoop 集群，此时再存入 1G 数据。请问此时 hdfs 集群上数据量一共有多大？

参考答案：

第一次存 1G 数据： $1G \times 3 = 3G$ 第二次存入 1G 数据： $1G \times 2 = 2G$ （配置修改重启后，只对后续接入的数据有效，已经存入的数据如果想改副本需要通过命令行来修改） 总数据大小： $3G + 2G = 5G$

2.NameNode 内存多大合适

HDFS 元数据都会存在于 NameNode 的内存中，因此 NameNode 的内存大小直接决定了集群支持的最大容量，那么如何估算 NameNode 需要的内存大小？比如一个含有 200 个节点的集群，每个节点有 24TB 的磁盘，每个 Block 的大小为 128MB，每个块有 3 个副本，那么请问，在这种场景下需要 NameNode 的内存大小为多少？

前提条件：一般 1GB 内存可以管理 100 万个 block 文件

参考答案：

先计算块文件的个数： $200 \times 25165824MB(24TB) / 128 \times 3 = 13107200$ 一般 1GB 内存可以管理 100 万万个 block 文件 参照该方法计算最终大概需要 13.1072GB 的内存。除此之外还要基于性万个 block 文件，所以在选择 NameNode 内存时要选择一个大于该值的一个合理的整数值

3.NameNode HA 机制

简述 namenode 的 HA 机制。他是如何实现故障切换的？

参考答案：

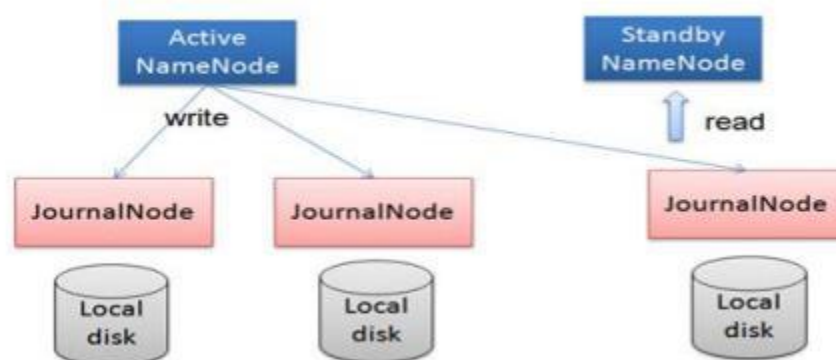
1.通过 QJM 解决 NameNode 元数据共享存储问题

NameNode 记录了 HDFS 的目录文件等元数据，客户端每次对文件的增删改等操作，Namenode 都会记录一条日志，叫做 editlog，而元数据存储 fsimage 中。为了保持 Standby 与 active 的状态一致，standby 需要尽量实时获取每条 editlog 日志，并应用到 fsimage 中。这时需要一个共享存储存放 editlog，standby 能实时获取日志。

有两个关键点需要保证：

- 1) 共享存储是高可用的。
- 2) 需要防止两个 NameNode 同时向共享存储写数据导致数据损坏。

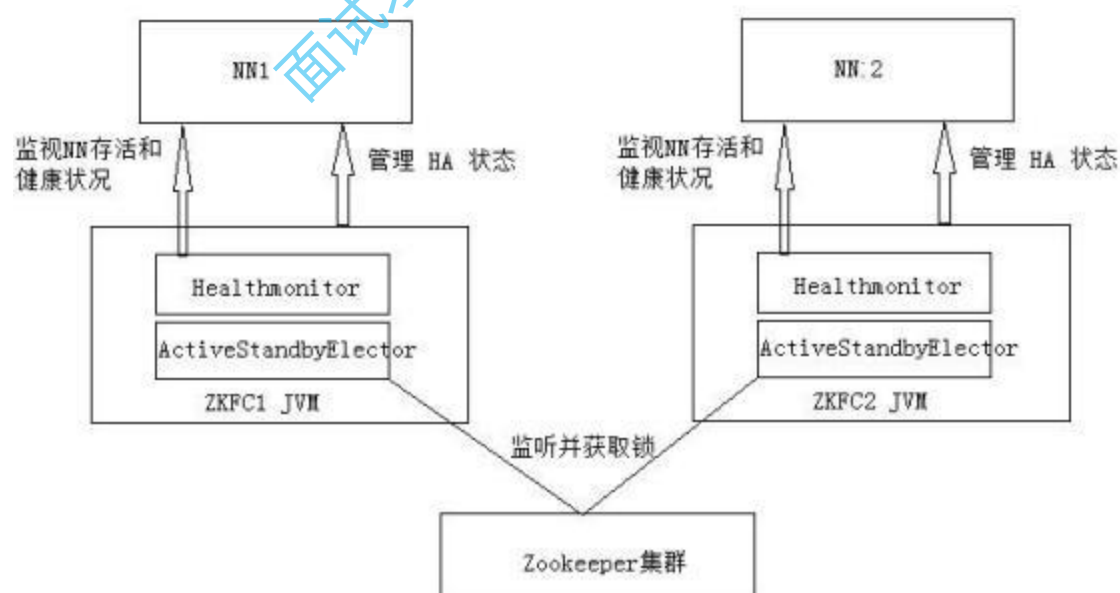
共享存储常用的方式是 Quorum Journal Manager，QJM 可以认为是包含一些 JournalNode 的集群，JournalNode 运行在不同的机器上，每个 JournalNode 是一个很轻量的守护进程，所以可以部署在 hadoop 集群的节点上，QJM 中至少要有 3 个 JournalNode，因为 edit log 必须要写到 JournalNodes 中大部分节点中，比如运行 3,5,7 个 JournalNode，如果你运行了 N 个 JournalNode，那么系统可以容忍最多 $(N-1)/2$ 个节点失败。



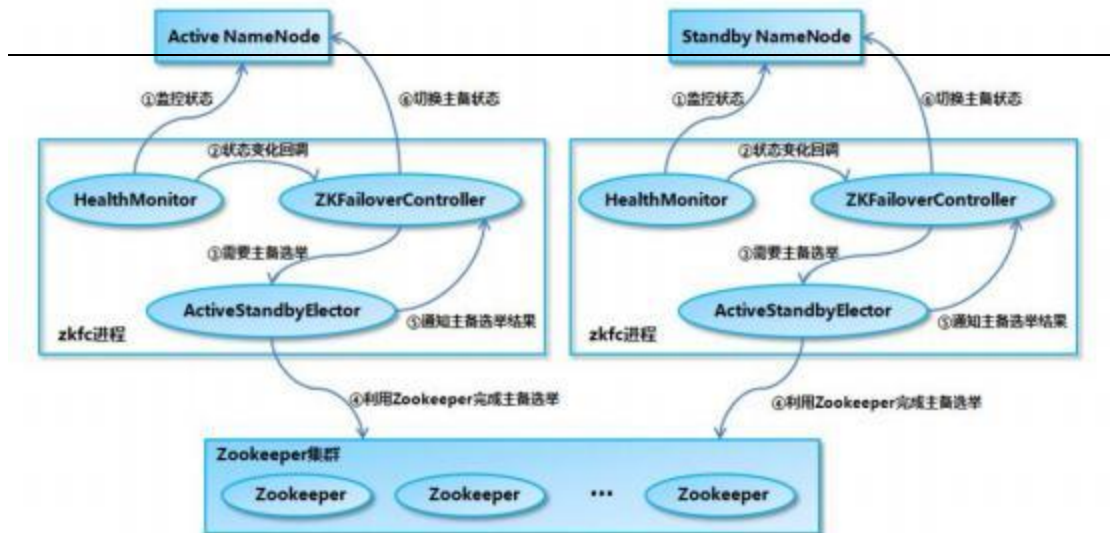
共享存储实现逻辑：

- 1) 初始化后，Active NN 把 editlog 写到大多数 JN 并返回成功（即大于等于 $N+1$ ）即认定写成功。
- 2) Standby NN 定期从 JN 读取一批 editlog，并应用到内存中的 FsImage 中。
- 3) NameNode 每次写 Editlog 都需要传递一个编号 Epoch 给 JN，JN 会对比 Epoch，如果比自己保存的 Epoch 大或相同，则可以写，JN 更新自己的 Epoch 到最新，否则拒绝操作。在切换时，Standby 转换为 Active 时，会把 Epoch+1，这样就防止即使之前的 NameNode 向 JN 写日志，即使写也会失败。

2.利用 Zookeeper 实现 NameNode 故障转移



3. HDFS2 NN 的主备切换流程



4.Hive 元数据存储位置

在 hive 数据库当中你们的元数据信息是存储在哪里的？

参考答案：

1. hive 默认内置元数据库为 derby 数据库
2. 生产环境中我们使用 mysql 数据库

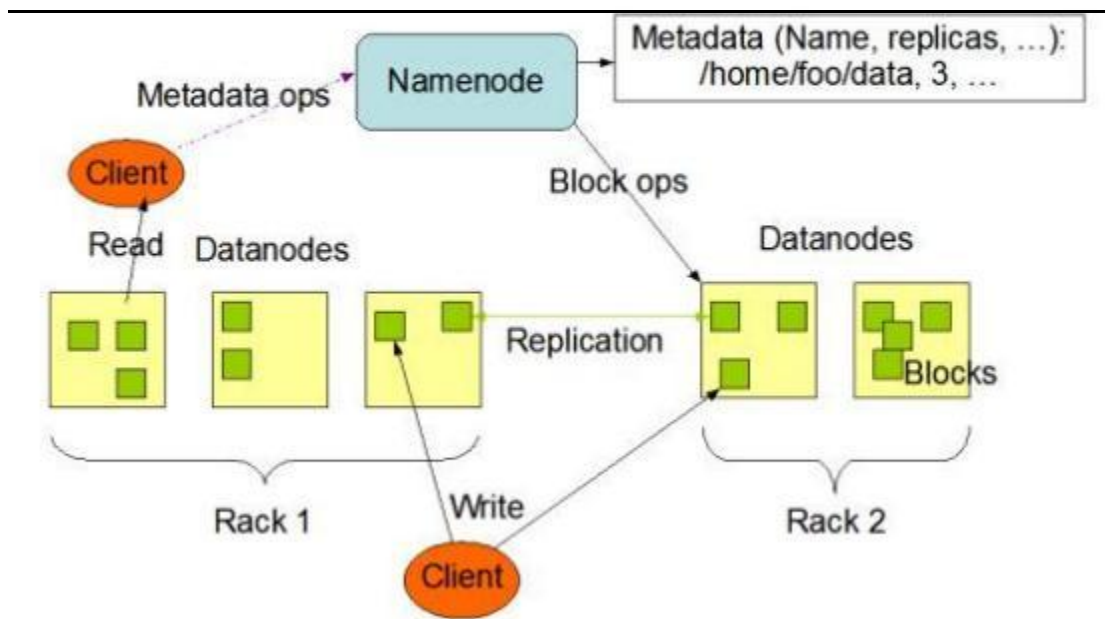
5.HDFS 体系结构

简述 HDFS 体系结构？

参考答案：

HDFS 具有主/从架构。一个 HDFS 集群包含一个 NameNode（一个主服务器），**用于管理文件系统名称空间并管理客户端对文件的访问**。此外，还有许多 DataNode，通常是群集中的每个节点一个 DataNode，**用于数据存储**。HDFS 对外暴露文件系统名称空间并允许用户数据存储在文件中。在内部，文件被分成一个或多个块，这些块存储在一组 DataNode 中。NameNode 执行文件系统命名空间操作，如打开，关闭和重命名文件和目录。它还确定块到 DataNode 的映射。DataNode 负责提供来自文件系统客户端的读取和写入请求。DataNode 还在 NameNode 的指令下执行块创建，删除，复制。

HDFS Architecture



6.HDFS 数据块大小

hdfs 数据块的默认大小是多少？过大过小有什么优缺点？

参考答案：

1、数据块默认大小 Hadoop2.0 之前，默认数据块大小为 64MB Hadoop2.0 之后，默认数据块大小为 128MB

2、优缺点

1) 数据块太小： (1) 会导致寻址时间相对比较长，如果读取数据块数据的时间小于寻址时间，这样是很不划算的，所以数据块要尽可能大一点。 (2) 数据块的大量元数据会占用 namenode 内存，影响 namenode 管理的集群规模

2) 数据块太大：为了保证数据的本地性，通常一个块的数据是交给一个 map 任务来处理，如果数据块太大，就导致 map 任务处理的数据量就会很多，时间就会很长，不能很好地实现并行处理。

7.HDFS 副本策略

简述 hadoop 的副本策略？副本过多过少有什么优缺点？

参考答案：

1、默认的副本策略：**BlockPlacementPolicyDefault**

- 1) **1st replica**. 如果写请求方所在机器是其中一个 **datanode**,则直接存放在本地,否则随机在集群中选择一个 **datanode**.
- 2) **2nd replica**. 第二个副本存放于不同第一个副本的所在的机架.
- 3) **3rd replica**.第三个副本存放于第二个副本所在的机架,但是属于不同的节点.
- 4) 剩下其它的副本随机选择存储的位置

2、副本过多导致存储空间的浪费, 增加节点之间数据传输对时间和资源的消耗 副本过少容易导致数据丢失, 起不到数据安全可靠的作用

8.DataNode 宕机对副本策略影响

DataNode 副本存储策略? 如果 Hadoop 在保存副本的时候, 比如, 第二个副本存完毕, 向第三个副本存入数据时 DataNode 挂了, 会出现什么情况?

参考答案:

1、默认的副本策略:

Block PlacementPolicyDefault

- 1) **1st replica**. 如果写请求方所在机器是其中一个 **datanode**,则直接存放在本地,否则随机在集群中选择一个 **datanode**.
- 2) **2nd replica**. 第二个副本存放于不同第一个副本的所在的机架.
- 3) **3rd replica**.第三个副本存放于第二个副本所在的机架,但是属于不同的节点.
- 4) 剩下其它的副本随机选择存储的位置

2、数据复制过程中出现问题怎么解决?

- 1) 这个问题其实也是写数据过程, 当数据写入 **DataNode** 的过程中, 当其中一个 **DataNode** 挂掉了, 会将这个 **DataNode** 移除流水线, 然后将数据写入到剩余的 **datanode**.
- 2) 在写入结束后, **namenode** 会收集 **datanode** 的信息, 发现此文件的数据块的 **replication** 没有达到配置的要求 (**default=3**), 然后再选择一个 **datanode** 保存副本。

9.HDFS 写数据流程

客户端向 HDFS 写入数据过程?

参考答案:

写数据流程:

- 1) 首先客户端对文件进行切分, 切分成数据块, 每个数据块默认存储到 3 个节点上
- 2) 在写入数据之前, 客户端首先会告诉 **NameNode** 要写一个文件, 此时 **NameNode** 会增加一个文件, 记录这个文件的详细信息, 把这个文件放入目录树里面
- 3) 然后开始写数据, 因为客户端把文件划分成了很多块, 在写第一个数据块时, **NameNode**

先要返回 3 个 `DataNode` 位置，这样第一个数据块会以一个一个数据包的方式（`packet`）写入第一个 `DataNode`，同时数据包也会同时向第二，第三个节点写，形成数据写的流水线，每个 `DataNode` 写完数据包之后，会返回确认。当第一个数据块写完之后，以同样的方式写下一个数据块。

4) 当所有数据块写完之后，它调用 `close` 关闭写入流，它会通知 `NameNode` 文件写完了，这时就会关闭 `socket`，结束数据的写过程。

10.HDFS 读数据流程

客户端从 HDFS 读数据过程？

读数据流程：

- 1) 客户端首先发送一个 `open` 请求到 `NameNode`，获取这个文件所包含数据块，以及每个数据块的存储位置。
- 2) 这些数据块信息返回给客户端之后，它依次按照这个数据块的顺序，读取每一个数据块。
- 3) 第一个数据块在这个节点上面读，第二个数据块在这个节点读，读完之后，按照文件实际数据块的先后位置，显示给用户，整个文件读完之后，就会调用 `close`，关闭输出流就 ok 了

11.DataNode 在什么情况下不会备份

datanode 什么情况下不会备份？

参考答案：

设置备份数为 1 时，就不会备份了。

延伸：Hadoop 中在哪里设置备份数，是哪个字段？

在 `hdfs-site.xml` 中的 `dfs.replication` 变量。

12.HDFS 小文件问题

HDFS 中大量小文件带来的问题以及解决方法

参考答案：

问题：

hadoop 中目录、文件和块都会以对象的形式保存在 `namenode` 的内存中，大概每个对象会占用 150bytes。小文件数量多会大量占用 `namenode` 的内存；使 `namenode` 读取元数据速度变

慢，启动时间延长；还因为占用内存过大，导致 gc 时间增加等。

解决办法：

两个角度，一是从根源解决小文件的产生，二是解决不了就选择合并。

从数据来源入手，如每小时抽取一次改为每天抽取一次等方法来积累数据量。

如果小文件无可避免，一般就采用合并的方式解决。可以写一个 MR 任务读取某个目录下的所有小文件，并重写为一个大文件。

13.fsimage 和 editlogs 的作用

fsimage 和 editlogs 是做什么用的？

参考答案：

命名空间镜像(FSImage)：HDFS 的目录树及文件/目录元信息是保存在内存中的，如果节点停电或进程崩溃，数据将不再存在,必须将上述信息保存到磁盘,FSImage 就是保存某一个时刻元数据的信息的磁盘文件。

镜像编辑日志(EditLog):对内存目录树的修改，也必须同步到磁盘元数据上，但每次修改都将内存元数据导出到磁盘，显然是不现实的，为此，namenode 引入了镜像编辑日志，将每次的改动都保存在日志中，如果 namenode 机器宕机或者 namenode 进程挂掉后可以使用 FSImage 和 EditLog 联合恢复内存元数据。

14.数据块大小多大合适

Linux 中的块大小为 4KB，为什么 HDFS 中块大小为 64MB 或 128MB？

参考答案：

1.块是存储在文件系统的数据的最小单元。如果采用 4kb 的块大小来存放存储在 Hadoop 中的数据，就会需要大量的块，大大增加了寻找块的时间，降低了读写效率。

2.一个 map 或者一个 reduce 都是以一个块为单位处理，如果块很小，mapreduce 任务数就会很多，任务之间的切换开销变大，效率降低。

15.数据能否并发写入 HDFS 文件

并发写入 HDFS 文件可行吗？

参考答案：

不行。因为客户端通过 `namenode` 接收到在数据块上写入的许可后，那个块会锁定直到写入操作完成，所以不能在同一个块上写入。

16.HDFS 集群角色

HDFS 分布式文件系统集群的角色主要有？

参考答案：

NameNode 、 DataNode 、 SecondaryNameNode

17.HDFS 格式化命令

首次启动 HDFS 时，必须对其进行格式化操作的命令？

参考答案：

`bin/hdfs namenode -format` 或者 `bin/hadoop namenode -format`

18.HDFS 启停命令

单独启停 HDFS 进程命令？

参考答案：

1) 启停 NameNode

`bin/hdfs --daemon start namenode`

`bin/hdfs --daemon stop namenode`

2) 启停 DataNode

`bin/hdfs --daemon start datanode`

`bin/hdfs --daemon stop datanode`

19.HDFS 垃圾桶配置

垃圾桶配置参数是什么？

参考答案：

`fs.trash.interval`

20.NameNode 的作用

NameNode 的主要作用有哪些?

参考答案:

- 一是管理文件系统文件的元数据信息(包括文件名称、大小、位置、属性、创建时间、修改时间等等)
- 二是维护文件到块的对应关系和块到节点的对应关系
- 三是维护用户对文件的操作信息(文件的增删改查)

21.DataNode 宕机如何恢复

一个 datanode 宕机,怎么恢复?

参考答案:

Datanode 宕机了后,如果是短暂的宕机,可以实现写好脚本监控,将它启动起来。如果是长时间宕机了,那么 datanode 上的数据应该已经被备份到其他机器了,那这台 datanode 就是一台新的 datanode 了,删除他的所有数据文件和状态文件,重新启动。

22.NameNode 宕机如何解决

Hadoop 的 namenode 宕机,怎么解决?

参考答案:

1.先分析宕机后的损失,宕机后直接导致 client 无法访问,内存中的元数据丢失,但是硬盘中的元数据应该还存在,如果只是节点挂了,重启即可,

2.如果是机器挂了,重启机器后看节点是否能重启,不能重启就要找到原因修复了。但是最终的解决方案应该是在设计集群的初期就考虑到这个问题,做 namenode 的 HA。提高容错:

第一种方式是将持久化存储在本地硬盘的文件系统元数据备份。

第二种方式是运行一个辅助的 NameNode (Secondary Namenode)。

第三种 zookeeper 提供的高可用(ha)机制。

23.HDFS 文件丢失如何处理

hadoop 出现文件块丢失怎么处理?

参考答案:

首先需要定位到哪的数据块丢失,可以通过查看日志进行检查和排除,找到文件块丢失的位

置后，如果文件不是很重要可以直接删除，然后重新复制到集群上一份即可，如果删除不了，每一个集群都会有备份，需要恢复备份。

24.如何查看 DataNode 健康状况

用命令显示所有 datanode 的健康状态？

参考答案：

```
bin/hadoop dfsadmin -report
```

25.HDFS 安全模式操作

hdfs 如何离开安全模式？

参考答案：

```
hadoop dfsadmin -safemode leave
```

26.HDFS 小文件如何合并

当 hdfs 小文件数量过多时，如何合并小文件？

参考答案：

1. 当每个小文件数据量比较小的时候，可以通过命令的方式进行小文件的合并如：

```
hdfs dfs -cat hdfs://mycluster/logs/*.log | hdfs dfs -appendToFile -  
hdfs://mycluster/logs/largeFile.log,
```

2. 当数据量比较大的时候建议使用 MR 进行小文件的合并。

27.HDFS 常见压缩算法

hdfs 常见的数据压缩算法？

参考答案：

1) Gzip 压缩

2) Bzip2 压缩

3) Lzo 压缩

4) Snappy 压缩

28.文件是先写入还是先复制

有 180M 的文件写入 HDFS 是先写 128M 复制完之后再写 52M 还是全部写完再复制？

参考答案：

HDFS 上在写入数据的时候，首先会对数据切块，然后从客户端到 `datanode` 形成一个管道，在至少将一个文件写入 `hdfs` 上后，表示文件写入成功，然后进行复制备份操作，所以是全部写完再复制。

7.YARN 面试

1.YARN 的优势

为什么会产生 YARN，它解决了什么问题？有什么优势？

参考答案：

1、产生原因

MapReduce 存在的问题：

- 1) JobTracker 单点故障。
- 2) JobTracker 承受的访问压力大，影响系统的扩展性。
- 3) 不支持 MapReduce 之外的计算框架，比如 Storm、Spark、Flink

2、解决的问题

- 1) yarn 解决了 JobTracker 负载过重的问题，将 MR1 中 JobTracker 的资源管理和作业调度两个功能分开，分别由 `ResourceManager` 和 `ApplicationMaster` 进程来实现。
- 2) 支持更多的计算框架，MapReduce storm Spark Flink 都可以运行在 yarn 上面。

3、优势

- 1) 更快地 MapReduce 计算
- 2) 对多框架支持
- 3) 框架升级更容易

2.Hadoop 调度器

简述 Hadoop 有哪些调度器？我们该如何选择？

参考答案：

1.hadoop 中常用的调度器有三种：

1)FIFO: (hadoop2.x 之前的默认调度器), 它先按照作业的优先级高低, 再按照到达时间的先后选择被执行的作业。

2)计算能力调度器(容量调度器) Capacity scheduler: (hadoop2.x 之后的默认调度器) 支持多个队列, 每个队列可配置一定的资源量, 每个队列采用 FIFO 调度策略, 为了防止同一个作业独占队列中的资源, 该调度器会对同一用户提交的作业所占资源进行限定。调度时, 首先按照以下策略选择一个合适队列: 计算每个正在运行任务数与其应该分得的计算资源之间的比值, 选择一个该比值最小的队列; 然后按以下策略选择队列中一个作业, 按照作业优先级和提交时间顺序选择, 同时考虑用户资源量限制和内存限制。

3)公平调度器 Fair scheduler 同计算能力调度器类似, 支持多队列多用户。每个队列的资源量可配置, 同一队列中的作业公平共享队列中所有资源

2.我们该如何选择调度器

如果你的场景需要先提交的 JOB 限制性, 那就 FIFO。

如果想着大家都有机会获取到资源, 就得用容量调度, 公平调度。容量调度不好的地方是多个队列资源不能相互抢占, 提前就把资源分走了, 哪怕那个队列没有任务, 所以一般都是公平调度。

FIFO 调度没人单独用。公平调度支持在某个队列内部选择公平调度还是 FIFO。

所以可以认为公平调度是一个混合的调度器。

3.YARN 集群角色

Yarn 资源调度系统集群的主要角色是哪些?

参考答案:

ResourceManager、NodeManager

8.Hadoop 面试

1.Hadoop 集群模式

hadoop 集群可以运行的 3 个模式是什么?

参考答案：

1.单机模式 2.伪分布模式 3.分布式

2.Hadoop Streaming 的理解

谈谈你对 Hadoop Streaming 的认识？

参考答案：

1、Hadoop Streaming 框架最大的好处是，让任何语言编写的 map, reduce 程序能够在 hadoop 集群上运行；map/reduce 程序只要遵循从标准输入 stdin 读，写出到标准输出 stdout 即可。

2、Hadoop Streaming 容易进行单机调试，通过管道前后相接的方式就可以模拟 streaming, 在本地完成 map/reduce 程序的调试 # cat inputfile | mapper | sort | reducer > output

3、streaming 框架还提供了作业提交时的丰富参数控制，直接通过 streaming 参数，而不需要使用 java 语言修改；很多 mapreduce 的高阶功能，都可以通过 steaming 参数的调整来完成

3.combiner 和 sort 发生在什么位置

简述在 combiner 和 sort 分别是发生在 map 端还是在 reduce 端？

参考答案：

Combiner 发生在 map 节点；sort 排序发生在 map 和 reduce 阶段。

4.MR 如何实现两个表的 Join 操作

用 mr 实现怎么实现一个大表和一个表关联，以及两个大表怎么关联？谈谈你的思路

参考答案：

- 1) 大小表关联用 Reduce join 的分布式缓存。
- 2) 两个大表关联：Reduce join + BloomFilter

5.YARN 提交 Job 的工作流程

在 yarn 中，一个 job 的运行流程是怎样的？

参考答案：

步骤 1：用户向 YARN 中提交应用程序，其中包括 ApplicationMaster 程序、启动 ApplicationMaster、用户程序等。

步骤 2：ResourceManager 为该应用程序分配第一个 Container，并与对应的 NodeManager

通信，要求它在这个 Container 中启动应用程序的 ApplicationMaster。

步骤 3：ApplicationMaster 首先向 ResourceManager 注册，这样用户可以直接通过 ResourceManager 查看应用程序的运行状态，然后它将为各个任务申请资源，并监控他的运行状态，直到运行结束，即要重复步骤 4-7。

步骤 4：ApplicationMaster 采用轮询的方式通过 RPC 协议找 ResourceManager 申请和领取资源。

步骤 5：一旦 Application 申请到资源后，便与对应的 NodeManager 通信，要求启动任务。

步骤 6：NodeManager 为任务设置好运行环境，包括环境变量、JAR 包、二进制程序等，然后将任务启动命令写到另一个脚本中，并通过运行该脚本启动任务。

步骤 7：各个任务通过 RPC 协议向 ApplicationMaster 汇报自己的状态和进度，ApplicationMaster 随时掌握各个任务的运行状态，从而可以再任务失败时重新启动任务。在应用程序运行过程中，用户可以随时通过 RPC 协议 ApplicationMaster 查询应用程序的当前运行状态。

步骤8：应用程序运行完成后，ApplicationMaster 向 ResourceManager 注销并关闭自己。

6.计算数据块和 map 数

假设 HDFS 数据块 blocksize 大小为 64MB，输入类为 FileInputFormat，现在有 3 个文件，大小分别为 64KB, 65MB, 200MB，请问 Hadoop 框架会把这些文件拆分为多少块？总共有多少个 map 任务？(假设 split 的大小等于块的大小)

参考答案：

文件	数据块数量	map 数量
64kb	1	1
65MB	2	1
200MB	4	4

7.哪些技术使用了缓存机制

描述一下 hadoop 生态圈中有哪些地方使用到了缓存机制，作用分别是什么？

参考答案：

1) MapReduce 中的 shuffle 过程中使用了缓存机制，map task 的输出数据需要先写入内存缓冲区中，缓冲区的作用是批量收集 map 结果，减少磁盘 IO 的影响。

2) Hbase 中的缓存分了两层：MemStore 和 BlockCache，MemStore 供写数据使用，BlockCache 供读数据使用

8.Hadoop 集群进程及作用

请列出正常的 hadoop 集群中都分别需要启动哪些进程，他们的作用分别都是什么？请尽量

列的详细一些。

参考答案：

- 1) NameNode : 管理 hdfs 名字空间、数据块映射关系, 配置副本策略, 接受读写请求
- 2) DataNode: 存储数据, 执行数据的读写操作
- 3) ResourceManager: 处理客户端请求, 启动并监控 ApplicationMaster, 监控 NodeManager, 资源分配与调度
- 4) NodeManager: 单节点资源管理, 处理来自 ResourceManager 和 ApplicationMaster 的指令
- 5) Zookeeper: 协调服务, 实现 NN RM 高可用
- 6) FailoverController (zkfc) : 不断监听本机 NN 的状态, 然后周期性的向 Zookeeper 发送心跳, 如果当前 NN 关掉了, 重新选举 NN Active
- 7) journalnode: 确保两个 NN 元数据保持同步

9.libjars 参数的作用及使用

Hadoop 中 libjars 参数的作用及使用?

参考答案:

作用: 指定 MR 第三方 jar 包

使用: `hadoop jar wordcount.jar com.dajiangtai.WordCount -libjars $LIBJARS input output` 注: libjars 这个参数一定要放到类名的后面, 参数的前面。这样就可以方便使用第三方 jar 包了。

10.Hadoop 性能调优

Hadoop 的性能调优从哪些方面着手?

参考答案:

一、硬件方面 (网络, 多磁盘, 多机架) ---系统集成人员来做
机架分开, 节点均匀放置

二、操作系统参数调优

- a) 多个网卡: 多网卡绑定, 做负载均衡或者主备
- b) 磁盘: 多个磁盘挂载到不同目录下, 存放数据做计算的磁盘不要做 raid
- c) 增大同时打开的文件数目的上线
- d) 关闭swap 分区, 避免内存与磁盘交换数据 (内存不足)
- e) 设置合理的预读取缓冲区大小 (磁盘 I/O 是瓶颈, CPU 和 内存配置一般较高)
- f) 文件系统的选择与配置 : Linux 有很多文件系统, 比如 ext3 和 ext4, 不同文件系统性能有差异。ext4 性能可靠性更好
- g) I/O 调度器选择: CFQ (Completely Fair Scheduler (完全公平调度器)) (cfq)、Noop 调度器 (noop)、截止时间调度器 (deadline)

三、jvm 层面

a) <http://www.cnblogs.com/edwardlauxh/archive/2010/04/25/1918603.html>

b) 使用最新稳定版 jdk

c) gc 算法 选择

d) 如何监控 jvm

jstat -gcutil 12122 1000

jvisualvm+jstatd

四、hadoop 层面性能调优

1、守护进程内存调优

a) NameNode 和 DataNode 内存调整在 `hadoop-env.sh` 文件中 NameNode:

```
export HADOOP_NAMENODE_OPTS="-Xmx512m -Xms512m"
-Dhadoop.security.logger=${HADOOP_SECURITY_LOGGER:-INFO,RFAS}
-Dhdfs.audit.logger=${HDFS_AUDIT_LOGGER:-INFO,NullAppender}
$HADOOP_NAMENODE_OPTS" DataNode : export HADOOP_DATANODE_OPTS="-Xmx256m
-Xms256m -Dhadoop.security.logger=ERROR,RFAS $HADOOP_DATANODE_OPTS" -Xmx -Xms 这两个参数一般保持一致，以避免每次垃圾回收完成后 JVM 重新分配内存。
```

b) ResourceManager 和 NodeManager 内存调整在 `yarn-env.sh` 文件中 ResourceManager:

```
export YARN_RESOURCEMANAGER_HEAPSIZE=1000 默认 export
YARN_RESOURCEMANAGER_OPTS="" 可以覆盖上面的值 NodeManager : export
YARN_NODEMANAGER_HEAPSIZE=1000 默认 export YARN_NODEMANAGER_OPTS=""; 可以覆盖上面的值
常驻内存经验配置: namenode:16G datanode:2-4G ResourceManager:4G
NodeManager:2G Zookeeper: 4G Hive Server: 2G
```

2、mr 中间目录要配置多个，分散 IO 压力 <http://hadoop.apache.org/docs/r2.6.0/> 查看 `yarn-default.xml` 分散 IO 压力 `yarn.nodemanager.local-dirs` `yarn.nodemanager.log-dirs`

配置文件 `mapred-default.xml`: `mapreduce.cluster.local.dir`

配置文件 `hdfs-default.xml`: 提高可靠性 `dfs.namenode.name.dir` `dfs.namenode.edits.dir` `dfs.datanode.data.dir`

3、mr 中间结果要压缩

a) 配置 `mapred-site.xml` 文件中配置 `mapreduce.map.output.compress` `true`
`mapreduce.map.output.compress.codec` `org.apache.hadoop.io.compress.SnappyCodec`
程序运行时指定参数 `hadoop jar /home/hadoop/tv/tv.jar MediaIndex`
`-Dmapreduce.compress.map.output=true`
`-Dmapreduce.map.output.compression.codec=org.apache.hadoop.io.compress.SnappyCodec`
`/tvdata /media`

b) 使用合理的压缩算法 (cpu 和磁盘) cpu: 如果是 cpu 的瓶颈, 可以更换速度快的压缩算法 磁盘: 如果是磁盘的瓶颈, 可以更换压缩力度大的压缩算法 一般情况我们使用 `snappy`

压缩，比较均衡

4、hdfs 文件系统中避免，大量小文件存在

5、根据具体情况，在 Map 节点使用 Combiner，减少输出结果

6、重用 Writable 类型，比如 声明一个对象 `Text word = new Text();` `map()`,`reduce()`方法里面重用

7、根据集群节点具体情况，调整 task 的并行度 设置 map 和 reduce 最大任务个数：
`mapreduce.tasktracker.map.tasks.maximum` `mapreduce.tasktracker.reduce.tasks.maximum` 设置
map 和 reduce 单个任务内存大小：`mapreduce.map.memory.mb` 1G 默认
`mapreduce.reduce.memory.mb` 1G 默认

11.Hadoop 作业推测执行

Hadoop 中通过拆分任务到多个节点运行来实现并行计算，但某些节点运行较慢会拖慢整个任务的运行，Hadoop 采用什么机制应对这个情况？

参考答案：

Speculative Execution 推测执行

（1）推测执行(Speculative Execution)是指在分布式集群环境下，因为程序 BUG，负载不均衡或者资源分布不均等原因，造成同一个 job 的多个 task 运行速度不一致，有的 task 运行速度明显慢于其他 task（比如：一个 job 的某个 task 进度只有 10%，而其他所有 task 已经运行完毕），则这些 task 拖慢了作业的整体执行进度，为了避免这种情况发生，Hadoop 会为该 task 启动备份任务，让该 speculative task 与原始 task 同时处理一份数据，哪个先运行完，则将谁的结果作为最终结果。

（2）推测执行优化机制采用了典型的以空间换时间的优化策略，它同时启动多个相同 task（备份任务）处理相同的数据块，哪个完成的早，则采用哪个 task 的结果，这样可防止拖后腿 Task 任务出现，进而提高作业计算速度，但是，这样却会占用更多的资源，在集群资源紧缺的情况下，设计合理的推测执行机制可在多用少量资源情况下，减少大作业的计算时间。

12.MapReduce 二次排序原理

MapReduce 二次排序原理？

参考答案：

1、简述

默认情况下，Map 输出的结果会对 Key 进行默认的排序，但是有时候需要对 Key 排序的同时还需要对 Value 进行排序，这时候就要用到二次排序了。

2、详细过程

1) Map 起始阶段

在 Map 阶段，使用 `job.setInputFormatClass()` 定义的 `InputFormat`，将输入的数据集分割成小数据块 `split`，同时 `InputFormat` 提供一个 `RecordReader` 的实现。在这里我们使用的是 `TextInputFormat`，它提供的 `RecordReader` 会将文本的行号作为 Key，这一行的文本作为 Value。这就是自定义 Mapper 的输入是 `Text` 的原因。然后调用自定义 Mapper 的 `map` 方法，将一个个键值对输入给 Mapper 的 `map` 方法

2) Map 最后阶段

在 Map 阶段的最后，会先调用 `job.setPartitionerClass()` 对这个 Mapper 的输出结果进行分区，每个分区映射到一个 Reducer。每个分区内又调用 `job.setSortComparatorClass()` 设置的 Key 比较函数类排序。可以看到，这本身就是一个二次排序。如果没有通过 `job.setSortComparatorClass()` 设置 Key 比较函数类，则使用 Key 实现的 `compareTo()` 方法

3) Reduce 阶段

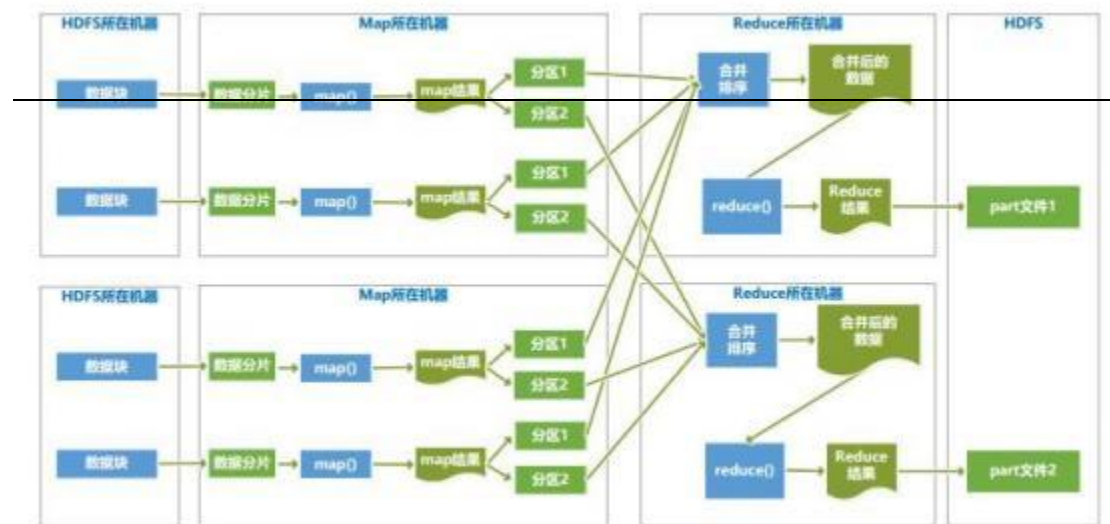
在 Reduce 阶段，`reduce()` 方法接受所有映射到这个 Reduce 的 map 输出后，也会调用 `job.setSortComparatorClass()` 方法设置的 Key 比较函数类，对所有数据进行排序。然后开始构造一个 Key 对应的 Value 迭代器。这时就要用到分组，使用 `job.setGroupingComparatorClass()` 方法设置分组函数类。只要这个比较器比较的两个 Key 相同，它们就属于同一组，它们的 Value 放在一个 Value 迭代器，而这个迭代器的 Key 使用属于同一个组的所有 Key 的第一个 Key。最后就是进入 Reducer 的 `reduce()` 方法，`reduce()` 方法的输入是所有的 Key 和它的 Value 迭代器，同样注意输入与输出的类型必须与自定义的 Reducer 中声明的一致

13.MapReduce Shuffle 过程

分析 MapReduce 数据处理及 shuffle 的流程，以及各个阶段的先后顺序。

参考答案：

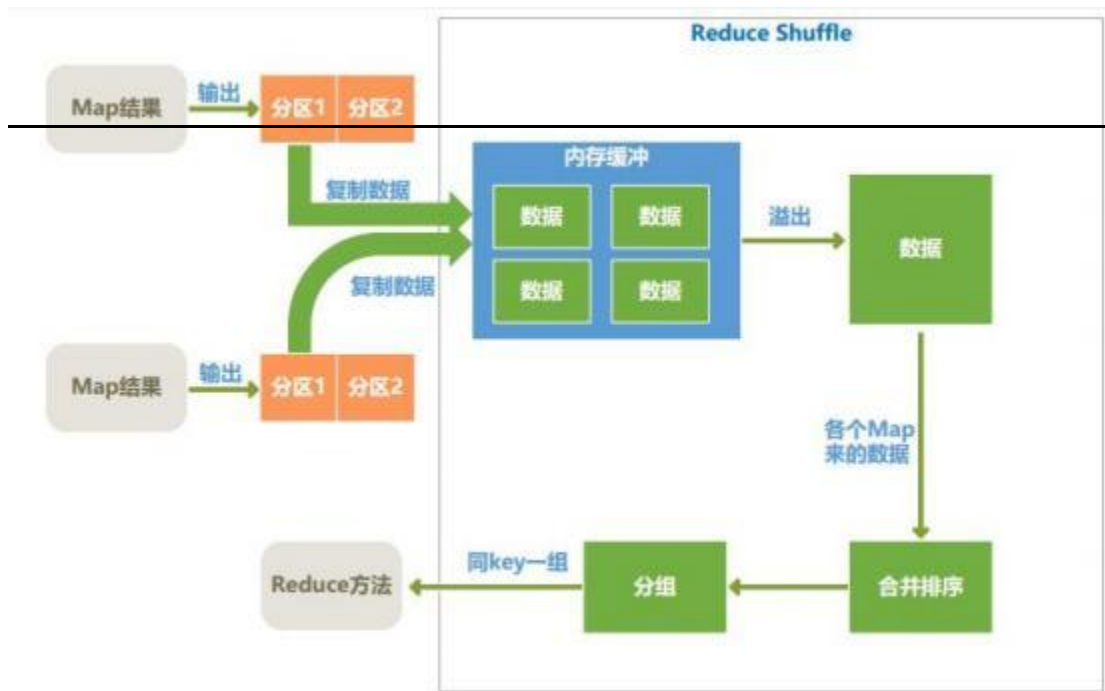
1) MapReduce 数据处理流程



2) Map 端 shuffle 过程



3) Reduce 端的 shuffle 过程



14.如何解决 MapReduce 数据倾斜

怎么解决 MapReduce 的数据倾斜？

参考答案：

现象：Mapreduce 程序在运行的时候，运行了大部分，但是还有部分 reduce 还在运行，甚至长时间运行，最终导致整个程序运行时间很长才结束。产生原因：reduce 程序处理的 key 的条数比其他 key 的条数大很多，这也就造成了分配到数据巨大的 key 的节点长时间运行。本质讲数据倾斜就是数据分布不均。 解决办法：

- 1) 增加 reduce 的 jvm 内存
- 2) 增加 reduce 个数
- 3) 自定义 partition
- 4) 设定 combiner
- 5) 如果数据倾斜很大的话，将那部分倾斜的数据可以单独过滤出来做计算 或者自定义分区分散倾斜的数据
- 6) 针对个别 key 或者少量 key 数据量特别大的，可以对该 key 进行改造，生成多个新的 key 分别处理，然后再进行聚合处理（分而治之）

15.Hadoop 分布式缓存

什么是 Hadoop 分布式缓存？

参考答案：

1) Hadoop 分布式缓存是 Map-Reduce 框架提供的用于缓存应用程序所需文件（文本文件、存档文件、Jar 文件等）的工具。

2) 当 MapReduce 处理大型数据集间的 join 操作时，此时如果一个数据集很大而另外一个集合很小，以至于可以分发到集群中的每个节点之中。这种情况下，我们就用到了 Hadoop 的分布式缓存机制，它能够在任务运行过程中及时地将文件和存档复制到任务节点以供使用。为了节约网络宽带，在每一个作业中，各个文件通常只需要复制到一个节点一次。

16.大数据集群监控

怎样对大数据集群进行有效监控？如何实现预警、报警？

参考答案：

1. 可以有很多方式对大数据集群进行监控

1) 可以使用 Ambari 或者 CM 进行监控

2) 也可以购买第三方服务来进行监控，如腾讯的蓝鲸

2.如何实现预警、告警

如果使用 Ambari 或者 CM，可以使用 alert 来实现。

17.Hadoop 框架的优点

Hadoop 特性优点？

参考答案：

1)扩容能力

2)成本低

3)高效率

4)可靠性

18.Hadoop 包含哪些核心组件

Hadoop 的核心组件是哪些？

参考答案：

1)HDFS

2)MapReduce

3)Yarn

19.Hadoop 目前包含几个大版本

Hadoop 目前有几个版本?

参考答案:

目前 hadoop 有 1.x、2.x、3.x。三个版本

20.Linux 主机名配置

设置主机名在哪个文件中?

参考答案:

/etc/sysconfig/network

21.IP 和主机名映射

配置 IP、主机名映射的文件是哪一个?

参考答案:

/etc/hosts

22.Hadoop3.x 的配置文件包含哪些

安装 hadoop3.x 时需要配置的文件有哪些?

参考答案:

- 1)hadoop-env.sh
- 2)core-site.xml
- 3)hdfs-site.xml
- 4)mapred-site.xml
- 5)yarn-site.xml
- 6)workers

23.启动 jobHistoryserver 服务进程的命令

hadoop3.x 启动 jobHistoryserver 服务进程的命令?

参考答案:

```
bin/mapred --daemon start historyserver 启动
bin/mapred --daemon stop historyserver 关闭
```

24.停止 Hadoop Job 运行的方式

Hadoop 如何快速杀死一个 job?

参考答案:

- 1、执行 `hadoop job -list` 拿到 `job-id`
- 2、`hadoop job -kill job-id`

9.Hive 面试

1.Hive UDF 函数有哪几种

Hive 中的 UDF 函数有哪几种? 你写过哪些?

参考答案:

1. Hive 中有三种 UDF:(普通)UDF、用户定义聚集函数 (user-defined aggregate function, UDAF)、用户定义表生成函数 (user-defined table-generating function, UDTF)。UDF 操作作用于单个数据行, 并且产生一个数据行作为输出。大多数函数都属于这一类 (比如数学函数和字符串函数)。

UDAF 接受多个输入数据行, 并产生一个输出数据行。想 COUNT 和 MAX 这样的函数就是聚集函数。

UDTF 操作作用于单个数据行, 并且产生多个数据行-----一个表作为输出

2. 只写过普通的 UDF 函数

2.Hive 的几种建表方式

Hive 中创建表有哪几种方式, 其区别是什么?

参考答案:

1.hive 创建表有以下几种方式

- 1) create table
- 2) create table as select ...
- 3) create table like tablename1;

2.不同方式的区别

1) create table 创建表，需要明确指定表结构和索引

2) create table as 创建表，只有表结构，没有索引。

3) create table like 创建出来的新表包含源表的完整表结构及索引信息

3.数据库中的 NULL 在 Hive 中如何存储

简要描述数据库中的 null，说出 null 在 hive 底层如何存储？并解释 `select a.* from t1 a left outer join on t2 b on a.id = b.id where b.id is null` 语句的含义？

参考答案：

1.HIVE 表中默认将 NULL 存为\N

2.表 t1 与 t2，根据 id 字段做做外链接，查询出 b.id 为 null 的值。

4.collect_list()和 collect_set()的区别

Hive 中，collect_list()、collect_set()两个函数的区别是什么？参数类型有什么限制？

参考答案：

1.Hive 中 collect 相关的函数有 collect_list 和 collect_set。它们都是将分组中的某列转为一个数组返回，不同的是 collect_list 不去重而 collect_set 去重。 2.collect_list(col)函数只接受基本数据类型，它的主要作用是将某字段的值进行汇总，产生array 类型字段。collect_set(col)函数只接受基本数据类型，它的主要作用是将某字段的值进行去重汇总，产生 array 类型字段。

5.Hive 查询内存溢出如何解决

Hive 在 select 查询数据后，执行 insert 操作插入 ORC 表和 parquet 表操作的过程中，遇到 over gc limit、java.lang.OutOfMemoryError:Java heap space 等字样的错误，大概从哪些地方查找原因解决问题？

参考答案：

1、可能数据分布不均匀造成的，可以在 select 阶段加 cluster by rand()让数据均匀分布。

2、map 和 reduce 阶段的内存不够，通过 hive 参数增加内存 set mapreduce.reduce.memory.mb=16384;（实际调试的时候，8G 内存不够，增加到 16G 解决问题） set mapreduce.map.memory.mb=4096;

3、另外在数据格式不统一的情况下，不同格式的 insert 操作效率奇低。

6.Hive 4 种 by 操作的区别与联系

hive 中 order by、distribute by、sort by 和 cluster by 的区别和联系?

参考答案:

order by

order by 会对数据进行全局排序,和 oracle 和 mysql 等数据库中的 order by 效果一样, 它只在一个 reduce 中进行所以数据量特别大的时候效率非常低。 而且当设置 : set hive.mapred.mode=strict 的时候不指定 limit, 执行 select 会报错, 如下: LIMIT must also be specified.

sort by

sort by 是单独在各自的 reduce 中进行排序, 所以并不能保证全局有序, 一般和 distribute by 一起执行, 而且 distribute by 要写在 sort by 前面 如果 mapred.reduce.tasks=1 和 order by 效果一样, 如果大于 1 会分成几个文件输出每个文件会按照指定的字段排序, 而不保证全局有序。 sort by 不受 hive.mapred.mode 是否为 strict ,nostrict 的影响

distribute by 用 distribute by 会对指定的字段按照 hashCode 值对 reduce 的个数取模, 然后将任务分配到对应的 reduce 中去执行。 也就是在 mapreduce 程序中的 patition 分区过程, 默认根据指定 key.hashCode() & Integer.MAX_VALUE % numReduce 确定处理该任务的 reduce 示例如下:

```
public class myPartitioner extends Partitioner{
    @Override
    public int getPartition(TextPair key, Text value, int num) {
        // TODO Auto-generated method stub
        if(num == 0 ){
            return 0;
        }
        int a = (key.getFirst().hashCode() & Integer.MAX_VALUE) % num;
        return a;
    }
}
```

Cluster By distribute by 和 sort by 合用就相当于 cluster by, 但是 cluster by 不能指定排序为 asc 或 desc 的规则, 只能是 desc 倒序排列。

7.Hive 性能调优

hive 性能调优的常见方法?

参考答案:

1、HQL 层面优化

1) 利用分区表优化

2) 利用桶表优化

3) join 优化

4) Group By 数据倾斜优化 解决这个问题 的方法是配置一个参数： set hive.groupby.skewindata=true。

5) Order By 优化

6) 一次读取多次插入

7) Join 字段显示类型转换

2、Hive 架构层面优化

1) 不执行 MapReduce

2) 本地模式执行 MapReduce

3) JVM 重用

4) 并行化

3、底层 MapReduce 优化

1) 合理设置 map 数

2) 合理设置 Reduce 数

8.Hive 包含哪些 Join 操作

Hive 的 join 有几种方式，怎么实现 join 的？

参考答案：

1、Hive 中常见的 join 方式

Hive 中除了支持和传统数据库中一样的内关联、左关联、右关联、全关联，还支持 LEFT SEMI JOIN 和 CROSS JOIN，但这两种 JOIN 类型也可以用前面的代替。

2、如何实现 join？

1) 内关联 (JOIN) 只返回能关联上的结果。SELECT a.id, a.name, b.age FROM djt_a a join djt_bb ON (a.id = b.id);

2) 左外关联 (LEFT [OUTER] JOIN) 以 LEFT [OUTER] JOIN 关键字前面的表作为主表，和其他表进行关联，返回记录数和主表的记录数一致，关联不上的字段置为 NULL。是否指定 OUTER 关键字，貌似对查询结果无影响。

SELECT a.id, a.name, b.age FROM djt_a a left join djt_bb ON (a.id = b.id);

3) 右外关联 (RIGHT [OUTER] JOIN) 和左外关联相反，以 RIGHT [OUTER] JOIN 关键词后面的表作为主表，和前面的表做关联，返回记录数和主表一致，关联不上的字段为 NULL。是否指定 OUTER 关键字，貌似对查询结果无影响。

SELECT a.id, a.name, b.age FROM djt_a a RIGHT OUTER JOIN djt_bb ON (a.id = b.id);

4) 全外关联 (FULL [OUTER] JOIN) 以两个表的记录为基准, 返回两个表的记录去重之和, 关联不上的字段为 NULL。是否指定 OUTER 关键字, 貌似对查询结果无影响。注意: FULL JOIN 时候, Hive 不会使用 MapJoin 来优化。

```
SELECT a.id, a.name, b.age FROM djt_a a FULL OUTER JOIN djt_bb ON (a.id = b.id);
```

5) LEFT SEMI JOIN 以 LEFT SEMI JOIN 关键字前面的表为主表, 返回主表的 KEY 也在副表中的记录。

```
SELECT a.id, a.name FROM djt_a a LEFT SEMI JOIN djt_bb ON (a.id = b.id);
```

6) 笛卡尔积关联 (CROSS JOIN) 返回两个表的笛卡尔积结果, 不需要指定关联键。

```
SELECT a.id, a.name, b.age FROM djt_a a CROSS JOIN djt_bb;
```

9.Hive 内部表和外部表

Hive 内部表和外部表的区别? 应该如何选择使用哪种表呢?

参考答案:

1) Hive 向内部表导入数据时, 会将数据移动到数据仓库指向的路径; 若是外部表, 数据的具体存放目录由用户建表时指定, 在导入数据到外部表, 数据并没有移动到自己的数据仓库目录下, 也就是说外部表中的数据并不是由它自己来管理的! 而内部表则不一样;

2) 在删除内部表的时候, Hive 将会把属于表的元数据和数据全部删掉; 而删除外部表的时候, Hive 仅仅删除外部表的元数据, 数据是不会删除的! 那么, 应该如何选择使用哪种表呢? 在大多数情况没有太多的区别, 因此选择只是个人喜好的问题。但是作为一个经验, 如果所有处理都需要由 Hive 完成, 那么你应该创建内部表, 否则使用外部表! 外部表相对来说更加安全些, 数据组织也更加灵活, 方便共享源数据。

10.Hive 支持索引吗?

Hive 支持索引 (3.0 版本之前), 但是 Hive 的索引与关系型数据库中的索引并不相同, 比如, Hive 不支持主键或者外键。并且 Hive 索引提供的功能很有限, 效率也并不高, 因此 Hive 索引很少使用。

(1) 索引适用的场景:

适用于不更新的静态字段。以免总是重建索引数据。每次建立、更新数据后, 都要重建索引以构建索引表。

(2) Hive 索引的机制如下:

hive 在指定列上建立索引, 会产生一张索引表 (Hive 的一张物理表), 里面的字段包括: 索引列的值、该值对应的 HDFS 文件路径、该值在文件中的偏移量。

Hive 0.8 版本后引入 bitmap 索引处理器, 这个处理器适用于去重后, 值较少的列 (例如, 某字段的取值只可能是几个枚举值) 因为索引是用空间换时间, 索引列的取值过多会导致建立 bitmap 索引表过大。

注意: Hive 中每次有数据时需要及时更新索引, 相当于重建一个新表, 否则会影响数据查询的效率和准确性, Hive 官方文档已经明确表示 Hive 的索引不推荐被使用, 在新版本的 Hive 中已经被废弃了。

扩展: Hive 是在 0.7 版本之后支持索引的, 在 0.8 版本后引入 bitmap 索引处理器, 在 3.0 版本开始移除索引的功能, 取而代之的是 2.3 版本开始的物化视图, 自动重写的物化视图替代了索引的功能。

11. 如何对 Hive 进行调度?

对 Hive 进行调度有三种方式:

- (1) 将 hive 的 sql 定义在脚本当中;
- (2) 使用 azkaban 或者 oozie 进行任务的调度;
- (3) 监控任务调度页面。

12. ORC、Parquet 等列式存储的优点

ORC 和 Parquet 都是高性能的存储方式, 这两种存储格式总会带来存储和性能上的提升。

Parquet:

(1) Parquet 支持嵌套的数据模型, 类似于 Protocol Buffers, 每一个数据模型的 schema 包含多个字段, 每一个字段有三个属性: 重复次数、数据类型和字段名。

重复次数可以是以下三种: required(只出现 1 次), repeated(出现 0 次或多次), optional(出现 0 次或 1 次)。每一个字段的数据类型可以分成两种: group(复杂类型)和 primitive(基本类型)。

(2) Parquet 中没有 Map、Array 这样的复杂数据结构, 但是可以通过 repeated 和 group 组合来实现的。

(3) 由于 Parquet 支持的数据模型比较松散, 可能一条记录中存在比较深的嵌套关系, 如果为每一条记录都维护一个类似的树状结构可能会占用较大的存储空间, 因此 Dremel 论文中提出了一种高效的对于嵌套数据格式的压缩算法: Striping/Assembly 算法。通过 Striping/Assembly 算法, parquet 可以使用较少的存储空间表示复杂的嵌套格式, 并且通常 Repetition level 和 Definition level 都是较小的整数值, 可以通过 RLE 算法对其进行压缩, 进一步降低存储空间。

(4) Parquet 文件是以二进制方式存储的, 是不可以直接读取和修改的, Parquet 文件是自

解析的，文件中包括该文件的数据和元数据。

ORC:

(1) ORC 文件是自描述的，它的元数据使用 Protocol Buffers 序列化，并且文件中的数据尽可能的压缩以降低存储空间的消耗。

(2) 和 Parquet 类似，ORC 文件也是以二进制方式存储的，所以是不可以直接读取，ORC 文件也是自解析的，它包含许多的元数据，这些元数据都是同构 ProtoBuffer 进行序列化的。

(3) ORC 会尽可能合并多个离散的区间尽可能的减少 I/O 次数。

(4) ORC 中使用了更加精确的索引信息，使得在读取数据时可以指定从任意一行开始读取，更细粒度的统计信息使得读取 ORC 文件跳过整个 row group，ORC 默认会对任何一块数据和索引信息使用 ZLIB 压缩，因此 ORC 文件占用的存储空间也更小。

(5) 在新版本的 ORC 中也加入了对 Bloom Filter 的支持，它可以进一步提升谓词下推的效率，在 Hive 1.2.0 版本以后也加入了对此的支持。

13.Hive 如何解析 Json 字符串数据？

Hive 处理 json 数据总体来说有两个方向的路走：

(1) 将 json 以字符串的方式整个入 Hive 表，然后通过使用 UDF 函数解析已经导入到 hive 中的数据，比如使用 LATERAL VIEW json_tuple 的方法，获取所需要的列名。

(2) 在导入之前将 json 拆成各个字段，导入 Hive 表的数据是已经解析过的。

14.Hive 数据倾斜如何解决？

(1) 空值引发的数据倾斜

实际业务中有些大量的 null 值或者一些无意义的数据参与到计算作业中，表中有大量的 null 值，如果表之间进行 join 操作，就会有 shuffle 产生，这样所有的 null 值都会被分配到一个 reduce 中，必然产生数据倾斜。

之前有小伙伴问，如果 A、B 两表 join 操作，假如 A 表中需要 join 的字段为 null，但是 B 表中需要 join 的字段不为 null，这两个字段根本就 join 不上啊，为什么还会放到一个 reduce 中呢？

这里我们需要明确一个概念，数据放到同一个 reduce 中的原因不是因为字段能不能 join 上，而是因为 shuffle 阶段的 hash 操作，只要 key 的 hash 结果是一样的，它们就会被拉到同一个 reduce 中。

解决方案：

第一种：可以直接不让 null 值参与 join 操作，即不让 null 值有 shuffle 阶段

```
SELECT *
```

```
FROM log a
```

```
JOIN users b
```

```
ON a.user_id IS NOT NULL
```

```

    AND a.user_id = b.user_id
UNION ALL
SELECT *
FROM log a
WHERE a.user_id IS NULL;

```

第二种：因为 null 值参与 shuffle 时的 hash 结果是一样的，那么我们可以给 null 值随机赋值，这样它们的 hash 结果就不一样，就会进到不同的 reduce 中：

```

SELECT *
FROM log a
LEFT JOIN users b ON CASE
    WHEN a.user_id IS NULL THEN concat('hive_', rand())
    ELSE a.user_id
END = b.user_id;

```

（2）不同数据类型引发的数据倾斜

对于两个表 join，表 a 中需要 join 的字段 key 为 int，表 b 中 key 字段既有 string 类型也有 int 类型。当按照 key 进行两个表的 join 操作时，默认的 Hash 操作会按 int 型的 id 来进行分配，这样所有的 string 类型都被分配成同一个 id，结果就是所有的 string 类型的字段进入到一个 reduce 中，引发数据倾斜。

解决方案：

如果 key 字段既有 string 类型也有 int 类型，默认的 hash 就都会按 int 类型来分配，那我们直接把 int 类型都转为 string 就好了，这样 key 字段都为 string，hash 时就按照 string 类型分配了：

```

SELECT *
FROM users a
LEFT JOIN logs b ON a.user_id = CAST(b.user_id AS string);

```

（3）不可拆分大文件引发的数据倾斜

当集群的数据量增长到一定规模，有些数据需要归档或者转储，这时候往往会对数据进行压缩；当对文件使用 GZIP 压缩等不支持文件分割操作的压缩方式，在日后有作业涉及读取压缩后的文件时，该压缩文件只会被一个任务所读取。如果该压缩文件很大，则处理该文件的 Map 需要花费的时间会远多于读取普通文件的 Map 时间，该 Map 任务会成为作业运行的瓶颈。这种情况也就是 Map 读取文件的数据倾斜。

解决方案：

这种数据倾斜问题没有什么好的解决方案，只能将使用 GZIP 压缩等不支持文件分割的文件转为 bzip 和 zip 等支持文件分割的压缩方式。

所以，我们在对文件进行压缩时，为避免因不可拆分大文件而引发数据读取的倾斜，在数据压缩的时候可以采用 bzip2 和 Zip 等支持文件分割的压缩算法。

（4）数据膨胀引发的数据倾斜

在多维聚合计算时，如果进行分组聚合的字段过多，如下：

```
select a, b, c, count(1) from log group by a, b, c with rollup;
```

注：对于最后的 **with rollup** 关键字不知道大家用过没，**with rollup** 是用来在分组统计数据的基础上再进行统计汇总，即用来得到 **group by** 的汇总信息。

如果上面的 **log** 表的数据量很大，并且 **Map** 端的聚合不能很好地起到数据压缩的情况下，会导致 **Map** 端输出的数据急速膨胀，这种情况容易导致作业内存溢出的异常。如果 **log** 表含有数据倾斜 **key**，会加剧 **Shuffle** 过程的数据倾斜。

解决方案：

可以拆分上面的 **sql**，将 **with rollup** 拆分成如下几个 **sql**：

```
SELECT a, b, c, COUNT(1)
FROM log
GROUP BY a, b, c;
```

```
SELECT a, b, NULL, COUNT(1)
FROM log
GROUP BY a, b;
```

```
SELECT a, NULL, NULL, COUNT(1)
FROM log
GROUP BY a;
```

```
SELECT NULL, NULL, NULL, COUNT(1)
FROM log;
```

但是，上面这种方式不太好，因为现在是对 3 个字段进行分组聚合，那如果是 5 个或者 10 个字段呢，那么需要拆解的 **SQL** 语句会更多。

在 **Hive** 中可以通过参数 **hive.new.job.grouping.set.cardinality** 配置的方式自动控制作业的拆解，该参数默认值是 30。表示针对 **grouping sets/rollups/cubes** 这类多维聚合的操作，如果最后拆解的键组合大于该值，会启用新的任务去处理大于该值之外的组合。如果在处理数据时，某个分组聚合的列有较大的倾斜，可以适当调小该值。

（5）表连接时引发的数据倾斜

两表进行普通的 **repartition join** 时，如果表连接的键存在倾斜，那么在 **Shuffle** 阶段必然会引起数据倾斜。

解决方案：

通常做法是将倾斜的数据存到分布式缓存中，分发到各个 Map 任务所在节点。在 Map 阶段完成 join 操作，即 MapJoin，这避免了 Shuffle，从而避免了数据倾斜。

MapJoin 是 Hive 的一种优化操作，其适用于小表 JOIN 大表的场景，由于表的 JOIN 操作是在 Map 端且在内存进行的，所以其并不需要启动 Reduce 任务也就不需要经过 shuffle 阶段，从而能在一定程度上节省资源提高 JOIN 效率。

在 Hive 0.11 版本之前，如果想在 Map 阶段完成 join 操作，必须使用 MAPJOIN 来标记显示地启动该优化操作，由于其需要将小表加载进内存所以要注意小表的大小。

如将 a 表放到 Map 端内存中执行，在 Hive 0.11 版本之前需要这样写：

```
select /* +mapjoin(a) */ a.id , a.name, b.age
from a join b
on a.id = b.id;
```

如果想将多个表放到 Map 端内存中，只需在 mapjoin() 中写多个表名称即可，用逗号分隔，如将 a 表和 c 表放到 Map 端内存中，则 /* +mapjoin(a,c) */ 。

在 Hive 0.11 版本及之后，Hive 默认启动该优化，也就是不在需要显示的使用 MAPJOIN 标记，其会在必要的时候触发该优化操作将普通 JOIN 转换成 MapJoin，可以通过以下两个属性来设置该优化的触发时机：

hive.auto.convert.join=true 默认值为 true，自动开启 MAPJOIN 优化。

hive.mapjoin.smalltable.filesize=2500000 默认值为 2500000(25M)，通过配置该属性来确定使用该优化的表的大小，如果表的大小小于此值就会被加载进内存中。

注意：使用默认启动该优化的方式如果出现莫名其妙的 BUG(比如 MAPJOIN 并不起作用)，就将以下两个属性置为 false 手动使用 MAPJOIN 标记来启动该优化：

hive.auto.convert.join=false (关闭自动 MAPJOIN 转换操作)

hive.ignore.mapjoin.hint=false (不忽略 MAPJOIN 标记)

再提一句：将表放到 Map 端内存时，如果节点的内存很大，但还是出现内存溢出的情况，我们可以通过这个参数 mapreduce.map.memory.mb 调节 Map 端内存的大小。

(6) 确实无法减少数据量引发的数据倾斜

在一些操作中，我们没有办法减少数据量，如在使用 collect_list 函数时：

```
select s_age,collect_list(s_score) list_score
from student
group by s_age
```

collect_list: 将分组中的某列转为一个数组返回。

在上述 sql 中, **sample** 如果存在数据倾斜, 当数据量大到一定的数量, 会导致处理倾斜的 **reduce** 任务产生内存溢出的异常。

注: **collect_list** 输出一个数组, 中间结果会放到内存中, 所以如果 **collect_list** 聚合太多数据, 会导致内存溢出。

有小伙伴说这是 **group by** 分组引起的数据倾斜, 可以开启 **hive.groupby.skewindata** 参数来优化。我们接下来分析下:

开启该配置会将作业拆解成两个作业, 第一个作业会尽可能将 **Map** 的数据平均分配到 **Reduce** 阶段, 并在这个阶段实现数据的预聚合, 以减少第二个作业处理的数据量; 第二个作业在第一个作业处理的数据基础上进行结果的聚合。

hive.groupby.skewindata 的核心作用在于生成的第一个作业能够有效减少数量。但是对于 **collect_list** 这类要求全量操作所有数据的中间结果的函数来说, 明显起不到作用, 反而因为引入新的作业增加了磁盘和网络 I/O 的负担, 而导致性能变得更为低下。

解决方案:

这类问题最直接的方式就是调整 **reduce** 所执行的内存大小。

调整 **reduce** 的内存大小使用 **mapreduce.reduce.memory.mb** 这个配置。

15.Hive 小文件过多如何解决?

(1) 使用 **hive** 自带的 **concatenate** 命令, 自动合并小文件

使用方法:

#对于非分区表

```
alter table A concatenate;
```

#对于分区表

```
alter table B partition(day=20201224) concatenate;
```

注意:

1、**concatenate** 命令只支持 **RCFILE** 和 **ORC** 文件类型。

2、使用 **concatenate** 命令合并小文件时不能指定合并后的文件数量, 但可以多次执行该命令。

3、当多次使用 **concatenate** 后文件数量不在变化, 这个跟参数 **mapreduce.input.fileinputformat.split.minsize=256mb** 的设置有关, 可设定每个文件的最小 **size**。

(2) 调整参数减少 **Map** 数量

设置 **map** 输入合并小文件的相关参数 (执行 **Map** 前进行小文件合并):

在 **mapper** 中将多个文件合成一个 **split** 作为输入 (**CombineHiveInputFormat** 底层是 **Hadoop** 的 **CombineFileInputFormat** 方法):

set hive.input.format=org.apache.hadoop.hive.ql.io.CombineHiveInputFormat; -- 默认

每个 Map 最大输入大小（这个值决定了合并后文件的数量）：

set mapred.max.split.size=256000000; -- 256M

一个节点上 split 的至少大小（这个值决定了多个 DataNode 上的文件是否需要合并）：

set mapred.min.split.size.per.node=100000000; -- 100M

一个交换机下 split 的至少大小(这个值决定了多个交换机上的文件是否需要合并)：

set mapred.min.split.size.per.rack=100000000; -- 100M

（3）减少 Reduce 的数量

reduce 的个数决定了输出的文件的个数，所以可以调整 reduce 的个数控制 hive 表的文件数量。

hive 中的分区函数 distribute by 正好是控制 MR 中 partition 分区的，可以通过设置 reduce 的数量，结合分区函数让数据均衡的进入每个 reduce 即可：

#设置 reduce 的数量有两种方式，第一种是直接设置 reduce 个数

set mapreduce.job.reduces=10;

#第二种是设置每个 reduce 的大小，Hive 会根据数据总大小猜测确定一个 reduce 个数

set hive.exec.reducers.bytes.per.reducer=512000000; -- 默认是 1G，设置为 5G

#执行以下语句，将数据均衡的分配到 reduce 中

set mapreduce.job.reduces=10;

insert overwrite table A partition(dt)

select * from B

distribute by rand();

对于上述语句解释：如设置 reduce 数量为 10，使用 rand()，随机生成一个数 $x \% 10$ ，这样数据就会随机进入 reduce 中，防止出现有的文件过大或过小。

（4）使用 hadoop 的 archive 将小文件归档

Hadoop Archive 简称 HAR，是一个高效地将小文件放入 HDFS 块中的文件存档工具，它能够将多个小文件打包成一个 HAR 文件，这样在减少 namenode 内存使用的同时，仍然允许对文件进行透明的访问。

#用来控制归档是否可用

set hive.archive.enabled=true;

#通知 Hive 在创建归档时是否可以设置父目录

set hive.archive.har.parentdir.settable=true;

#控制需要归档文件的大小

set har.partfile.size=1099511627776;

使用以下命令进行归档：

```
ALTER TABLE A ARCHIVE PARTITION(dt='2022-09-28', city='bj');
```

对已归档的分区恢复为原文件：

```
ALTER TABLE A UNARCHIVE PARTITION(dt='2022-09-28', city='bj');
```

注意：

归档的分区可以查看不能 insert overwrite，必须先 unarchive

10.HBase 面试

1.HBase 数据存储位置

HBase 中的数据存储在哪里？以什么格式存储？和 Zookeeper 什么关系？

参考答案：

1.HBase 中的所有数据文件都存储在 Hadoop HDFS 文件系统上。

2.包含两种文件类型：

HFile：HBase 中 KeyValue 数据的存储格式。HFile 是 Hadoop 的二进制格式文件。

HLog File：HBase 中 WAL（Write Ahead Log）的存储格式，物理上是 Hadoop 的 Sequence File

3.

1)hbase meta 表的位置存储在 Zookeeper

2)hbase master 利用 Zookeeper 选举

3) master 与 RegionServer 通信通过 Zookeeper

4) Zookeeper 存储 hbase 元数据信息

2.HBase 与 Hive 对比

底层数据同样存储在 HDFS,HBase 为什么比 Hive 查询速度快？

参考答案：

Hive

1、hive 是基于 MapReduce 来处理，离线计算速度慢。

2、MapReduce 处理数据是基于行的模式，查询数据默认是扫描整个表。

3、hive 中的表是纯逻辑表，只是表的定义，本身不存储数据，完全依赖 hdfs 和 MapReduce。

Hbase

1、hbase 是物理表，有独立的物理结构，查询的时候将相应的数据加载到内存，提升后续的查询效率。

2、Hbase 是基于列存储的，查询的时候可以只扫描某一个列或者某几列，避免扫描所有数

据。

3、Hbase 有一级索引 rowkey，根据 rowkey 查询速度非常快。

4、Hbase 提供了一个超大的内存 hash 表，搜索引擎通过这个 hash 表存储索引，提升查询效率。

3.HBase 节点宕机怎么办

如果 HBase节点宕机了，后续是如何处理的？是影响线上业务？

参考答案：

1、如果是 Master 宕机，通过 Zookeeper 重新选举另外一个 Master 提供服务，此短暂期间负载均衡，region 迁移，元数据操作无法进行，但不影响数据读写。2、如果是 RegionServer 宕机，当前 RegionServer 上面的 region 会迁移到其他 RegionServer。此短暂期间，无法响应 region I/O 请求以及 region 切分。

4.HBase 写数据原理

HBase 写数据的原理是什么？

参考答案：

1.客户端：（1）客户端发起 Put 写请求，将 put 写入 writeBuffer，如果是批量提交，写满缓存后自动提交（2）根据 rowkey 将 put 分发给不同 regionserver

2.服务端：

（1）Regionserver 将 put 按 rowkey 分给不同的 region

（2）Region 首先把数据写入 wal（3）wal 写入成功后，把数据写入 memstore，此时写成功，并返回通知客户端（4）Memstore 写完后，检查 memstore 大小是否达到 flush 阈值（5）如果达到 flush 阈值，将 memstore 写入到 HDFS，生成 HFile 文件

5.HBase Rowkey 如何设计

Hbase 如何设计 rowkey？列簇怎么设计比较好？

参考答案：

1、Rowkey 设计：

- 1) 业务逻辑简单，同时满足业务查询
 - 2) 数据是连续查询还是随机查询
 - 3) 考虑热点问题，选择不同的高位散列方法
 - 4) 设计原则：长度、散列、唯一
- 2、列簇设计：

2、列簇设计

- 1) 列簇一般 2~3 个比较合适;
 - 2) 具有相同 IO 特性的列放在一个列簇, 降低 IO;
 - 3) 列簇的名称尽可能短。
-

6.HBase 预分区

怎么实现 Hbase 的预分区?

参考答案:

1. 首先确定分区个数

$\text{regionNum} = \text{数据总量} / \text{region 大小}$

2. 确定 rowkey 范围

1) 可以抽样

2) 可以指定一个范围 (如使用 java short 类型 16 进制)

3. 计算参数中的 startkey 和 endkey

4. 调用 Java api 分区方法

`admin.createTable(tableDescriptor, Bytes.toBytes(startkey), Bytes.toBytes(endkey), regionNum);`

7.HBase 特点

HBase 的特点是什么?

参考答案:

大: 单表可以数十亿行, 数百万列

无模式: 同一个表的不同行可以有截然不同的列

面向列: 存储、权限控制、检索均面向列

稀疏: 空列不占用存储, 表是稀疏的

多版本: 每个单元中的数据可以有多个版本, 默认情况下版本号自动分配, 是单元格插入时的时间戳

数据类型单一: 数据都是字符串, 没有类型

8.HBase 如何导入数据

hbase 如何导入数据？

参考答案：

- 1.通过 HBase API 进行批量写入数据；
- 2.使用 Sqoop 工具批量导入到 HBase 集群；
- 3.使用 MapReduce 批量导入；
- 4.HBase BulkLoad 的方式。

9.HBase 的存储结构

hbase 的存储结构？

参考答案：

Hbase 中的每张表都通过行键(rowkey)按照一定的范围被分割成多个子表（HRegion），默认一个 HRegion 超过 256M 就要被分割成两个，由 HRegionServer 管理，管理哪些 HRegion 由 Hmaster 分配。HRegion 存取一个子表时，会创建一个 HRegion 对象，然后对表的每个列族（Column Family）创建一个 store 实例，每个 store 都会有 0 个或多个 StoreFile 与之对应，每个 StoreFile 都会对应一个 HFile，HFile 就是实际的存储文件，因此，一个 HRegion 还拥有一个 MemStore 实例。

10.HBase 数据查询原理

解释下 hbase 实时查询的原理

参考答案：

实时查询，可以认为是从内存中查询，一般响应时间在实时查询，可以认为是从内存中查询，一般响应时间在 1 秒内。HBase 的机制是数据先写入到内存中，当数据量达到一定的量（如 128M），再写入磁盘中，在内存中，是不进行数据的更新或合并操作的，只增加数据，这使得用户的写操作只要进入内存中就可以立即返回，保证了 HBase I/O 的高性能。

11.HBase scan 和 get 操作异同

描述 Hbase 中 scan 和 get 的功能以及实现的异同？

参考答案：

1.按指定 RowKey 获取唯一一条记录， get 方法（org.apache.hadoop.hbase.client.Get）Get 的方法处理分两种：设置了 ClosestRowBefore 和没有设置的 rowlock .主要是用来保证行的事务性，即每个 get 是以一个 row 来标记的.一个 row 中可以有很多 family 和 column.

2.按指定的条件获取一批记录， scan 方法(org.apache.Hadoop.hbase.client.Scan)实现条件查询功能使用的就是 scan 方式.

1)scan 可以通过 setCaching 与 setBatch 方法提高速度(以空间换时间);

2)scan 可以通过 setStartRow 与 setEndRow 来限定范围([start, end]start 是闭区间， end 是开区间)。范围越小，性能越高。

3)scan 可以通过 setFilter 方法添加过滤器，这也是分页、多条件查询的基础。

3.全表扫描，即直接扫描整张表中所有行记录

12.HBase 文件合并的作用

简述 HBASE 中 compact 用途是什么，什么时候触发，分为哪两种,有什么区别，有哪些相关配置参数？

参考答案：

在 hbase 中每当有 memstore 数据 flush 到磁盘之后，就形成一个 storefile，当 storeFile 的数量达到一定程度后，就需要将 storefile 文件来进行 compaction 操作。

Compact 的作用：

1>.合并文件

2>.清除过期，多余版本的数据

3>.提高读写数据的效率

HBase 中实现了两种 compaction 的方式：minor and major.

这两种 compaction 方式的区分是：

1、 Minor 操作只用来做部分文件的合并操作以及包括 minVersion=0 并且设置 ttl 的过期版本清理，不做任何删除数据、多版本数据的清理工作。

2、 Major 操作是对 Region 下的 HStore 下的所有 StoreFile 执行合并操作，最终的结果是整理合并出一个文件。

13.每天百亿数据如何存入 HBase

每天百亿数据存入 HBase，如何保证数据的存储正确和在规定的时间内全部录入完毕，不残留数据？

参考答案：

分析：

- 1) 百亿数据：证明数据量非常大
- 2) 存入 HBase：证明是跟 HBase 的写入数据有关
- 3) 保证数据的正确：要设计正确的数据结构保证正确性
- 4) 在规定时间内完成：对存入速度是有要求的

解决方案：

- 1) 批量导入推荐使用 BulkLoad 方式
- 2) 存入 HBase：普通写入是用 JavaAPI put 来实现，批量导入推荐使用 BulkLoad
- 3) 保证数据的正确：这里需要考虑 RowKey 的设计、预建分区和列族设计等问题
- 4) 在规定时间内完成也就是存入速度不能过慢，并且当然是越快越好，使用 BulkLoad

14.Web 界面如何访问 HBase

HBase 如何给 WEB 前端提供接口来访问？

参考答案：

- 1) 使用 JavaAPI 来编写 WEB 应用；
- 2) 使用 HBase 提供的 RESTful 接口

15.HBase 常见优化手段

说一说 HBase 优化常见手段？

参考答案：

- 1) 减少调整

减少调整这个如何理解呢？HBase 中有几个内容会动态调整，如 region（分区）、HFile，所以通过一些方法来减少这些会带来 I/O 开销的调整

Region

如果没有预建分区的话，那么随着 region 中条数的增加，region 会进行分裂，这将增加 I/O 开销，所以解决方法就是根据你的 RowKey 设计来进行预建分区，减少 region 的动态分裂

HFile

HFile 是数据底层存储文件，在每个 memstore 进行刷新时会生成一个 HFile，当 HFile 增加到一定程度时，会将属于一个 region 的 HFile 进行合并，这个步骤会带来开销但不可避免，但是合并后 HFile 大小如果大于设定的值，那么 HFile 会重新分裂。为了减少这样的无谓的 I/O 开销，建议估计项目数据量大小，给 HFile 设定一个合适的值

2) 减少启停

数据库事务机制就是为了更好地实现批量写入，较少数据库的开启关闭带来的开销，那么 HBase 中也存在频繁开启关闭带来的问题。

关闭 Compaction，在闲时进行手动 Compaction

因为 HBase 中存在 Minor Compaction 和 Major Compaction，也就是对 HFile 进行合并，所谓合并就是 I/O 读写，大量的 HFile 进行肯定会带来 I/O 开销，甚至是 I/O 风暴，所以为了避免这种不受控制的意外发生，建议关闭自动 Compaction，在闲时进行 compaction

批量数据写入时采用 BulkLoad

如果通过 HBase-Shell 或者 JavaAPI 的 put 来实现大量数据的写入，那么性能差是肯定并且还可能带来一些意想不到的问题，所以当需要写入大量离线数据时建议使用 BulkLoad

3) 减少数据量

虽然我们是在进行大数据开发，但是如果可以通过某些方式在保证数据准确性同时减少数据量，何乐而不为呢？

开启过滤，提高查询速度

开启 BloomFilter，BloomFilter 是列族级别的过滤，在生成一个 StoreFile 同时会生成一个 MetaBlock，用于查询时过滤数据

使用压缩：一般推荐使用 Snappy 和 LZO 压缩

4) 合理设计

在一张 HBase 表格中 RowKey 和 ColumnFamily 的设计是非常重要的，好的设计能够提高性能和保证数据的准确性

RowKey 设计：应该具备以下几个属性

散列性：散列性能够保证相同相似的 rowkey 聚合，相异的 rowkey 分散，有利于查询

简短性：rowkey 作为 key 的一部分存储在 HFile 中，如果为了可读性将 rowKey 设计得过长，那么将会增加存储压力

唯一性：rowKey 必须具备明显的区别性

业务性：举些例子

假如我的查询条件比较多，而且不是针对列的条件，那么 rowKey 的设计就应该支持多条件查询

如果我的查询要求是最近插入的数据优先，那么 rowKey 则可以采用叫上 Long.Max-时间戳的方式，这样 rowKey 就是递减排列

列族的设计

列族的设计需要看应用场景

多列族设计的优劣

优势：HBase 中数据是按列进行存储的，那么查询某一列族的某一列时就不需要全盘扫描，只需要扫描某一列族，减少了读 I/O；其实多列族设计对减少的作用不是很明显，适用于读多写少的场景

劣势：降低了写的 I/O 性能。原因如下：数据写到 store 以后是先缓存在 memstore 中，同一个 region 中存在多个列族则存在多个 store，每个 store 都一个 memstore，当其实 memstore 进行 flush 时，属于同一个 region

的 store 中的 memstore 都会进行 flush，增加 I/O 开销

16.HBase 列簇为什么不能太多

为什么不建议在 HBase 中使用过多的列族？

参考答案：

在 Hbase 的表中，每个列族对应 Region 中的一个 Store，Region 的大小达到阈值时会分裂，因此如果表中有多个列族，则可能出现以下现象：

- 1.一个 Region 中有多个 Store，如果每个 CF 的数据量分布不均匀时，比如 CF1 为 100 万，CF2 为 1 万，则 Region 分裂时导致 CF2 在每个 Region 中的数据量太少，查询 CF2 时会横跨多个 Region 导致效率降低。
- 2.如果每个 CF 的数据分布均匀，比如 CF1 有 50 万，CF2 有 50 万，CF3 有 50 万，则 Region 分裂时导致每个 CF 在 Region 的数据量偏少，查询某个 CF 时会导致横跨多个 Region 的概率增大。
- 3.多个 CF 代表有多个 Store，也就是说有多个 MemStore(2MB)，也就导致内存的消耗量增大，使用效率下降。
- 4.Region 中的 缓存刷新 和 压缩 是基本操作，即一个 CF 出现缓存刷新或压缩操作，其它 CF 也会同时做一样的操作，当列族太多时就会导致 IO 频繁的问题。

17.HBase Rowkey 设计原则

描述 Hbase 的 rowKey 的设计原则？

参考答案:

1.rowkey 长度原则

rowkey 是一个二进制码流，可以是任意字符串，最大长度 64kb，实际应用中一般为 10-100bytes，以 byte[] 形式保存，一般设计成定长。建议越短越好，不要超过 16 个字节，原因如下:

数据的持久化文件 HFile 中是按照 KeyValue 存储的，如果 rowkey 过长会极大影响 HFile 的存储效率 MemStore 将缓存部分数据到内存，如果 rowkey 字段过长，内存的有效利用率就会降低，系统不能缓存更多的数据，这样会降低检索效率

2.rowkey 散列原则

如果 rowkey 按照时间戳的方式递增，不要将时间放在二进制码的前面，建议将 rowkey 的高位作为散列字段，由程序随机生成，低位放时间字段，这样将提高数据均衡分布在每个 RegionServer，以实现负载均衡的几率。如果没有散列字段，首字段直接是时间信息，所有的数据都会集中在一个 RegionServer 上，这样在数据检索的时候负载会集中在个别的 RegionServer 上，造成热点问题，会降低查询效率。

3.rowkey 唯一原则

必须在设计上保证其唯一性，rowkey 是按照字典顺序排序存储的，因此，设计 rowkey 的时候，要充分利用这个排序的特点，将经常读取的数据存储到一块，将最近可能会被访问的数据放到一块。

18.HBase 写数据流程

HBase 写数据流程?

参考答案:

1. 当客户端发起一个 Put 请求时，首先根据 RowKey 寻址，从 hbase:meta 表中查出该 Put 数据最终需要去的 HRegionServer
2. 客户端将 Put 请求发送给相应的 HRegionServer，在 HRegionServer 中它首先会将该 Put 操作写入 WAL 日志文件中(Flush 到磁盘中)
3. 写完 WAL 日志文件后，HRegionServer 根据 Put 中的 TableName 和 RowKey 找到对应的 HRegion，并根据 Column Family 找到对应的 HStore
4. 将 Put 数据写入到该 HStore 的 MemStore 中。此时写成功，并返回通知客户端

19.HBase 读数据流程

HBase 读数据流程？

参考答案：

BlockCache 主要提供给读使用。读请求先到 Memtore 中查数据，查不到就到 BlockCache 中查，再查不到就会到磁盘上读，并把读的结果放入 BlockCache。BlockCache 采用的算法为 LRU（最近最少使用算法），因此当 BlockCache 达到上限后，会启动淘汰机制，淘汰掉最老的一批数据。

20.HBase Region 如何定位

HBase Region 如何定位？

参考答案：



21.HBase Region 分裂的意义

HBase Region 分裂的意义是什么？

参考答案：

HBase 数据写入首先会写入缓存，缓存写满会执行一次 flush 操作，每次 flush 都会生成一个 HFile 文件。随着 HFile 的增多，文件的读取效率势必会降低，HBase 采用 compact 机制不断的对这些文件进行合并，将小文件合并成大文件。然而，对 HBase 而言，大文件也不是什么好事，原因如下：

1) 数据分布不均匀。同一 region server 上数据文件越来越大，读请求也会越来越多。一旦所有的请求都落在同一个 region server 上，尤其是很多热点数据，必然会导致很严重的性能问题。

2) **compaction** 性能损耗严重。**compaction** 本质上是一个排序合并的操作，合并操作需要占用大量内存，因此文件越大，占用内存越多。另一方面，**compaction** 有可能需要迁移远程数据到本地进行处理（**balance** 之后的 **compaction** 就会存在这样的场景），如果需要迁移的数据是大文件的话，带宽资源就会损耗严重。

3) 太大文件读取效率也会受到影响。

另外，HBase 的数据写入量也是很惊人的，每天都可能有上亿条的数据写入，因此不做切分的话一个热点 **region** 的新增数据量就有可能几十 G，用不了多长时间大量读请求就会把单台 **region server** 的资源耗光。只有通过切分，一个 **region** 变为两个近似相同大小的子 **region**，再通过 **balance** 机制均衡到不同 **region server** 上，才能使系统资源使用更加均衡。

22.HBase Region 具体分裂过程

HBase Region 分裂过程是什么样的？

参考答案：

最初，一个 Table 只有一个 HRegion，随着数据写入增加，如果一个 HRegion 到达一定的大小，就需要 Split 成两个 HRegion，这个大小由 `hbase.hregion.max.filesize` 指定 split 时，两个新的 HRegion 会在同一个 HRegionServer 中创建，它们各自包含父 HRegion 一半的数据，当 Split 完成后，父 HRegion 会下线，而新的两个子 HRegion 会向 HMaster 注册上线；处于负载均衡的考虑，这两个新的 HRegion 可能会被 HMaster 分配到其他的 HRegionServer。

23.生产环境 HBase 如何做预分区

产线环境 HBase 为什么要做预分区？

参考答案：

表默认初始状态只有一个 region，region 的大小超过阈值时会自动 split 成两个，但 split 操作会带来资源消耗，在数据量大、访问量大，或被批处理程序读写情况下性能可能会很差，并且伴随大批量读写而来的 GC 可能会使 regionserver 宕机，接着 region 被 transit 到其他节点上，将逐步拖垮 HBase 集群上的所有节点。

24.HBase 分区过多有什么影响

HBase 分区过多有什么影响？

参考答案：

1. 频繁刷写

Region 的一个列族对应一个 MemStore，假设 HBase 表都有统一的 1 个列族配置，则每个 Region 只包含一个 MemStore。通常 HBase 的一个 MemStore 默认大小为 128 MB，见参数 `hbase.hregion.memstore.flush.size`。当可用内存足够时，每个 MemStore 可以分配 128 MB 空间。当可用内存紧张时，假设每个 Region 写入压力相同，则理论上每个 MemStore 会平均分配可用内存空间。

因此，当节点 Region 过多时，每个 MemStore 分到的内存空间就会很小。这个时候，写入很小的数据量就会被强制 Flush 到磁盘，将会导致频繁刷写。频繁刷写磁盘，会对集群 HBase 与 HDFS 造成很大的压力，可能会导致不可预期的严重后果。

2. 压缩风暴

因 Region 过多导致的频繁刷写，将在磁盘上产生非常多的 HFile 小文件，当小文件过多的时候 HBase 为了优化查询性能就会做 Compaction 操作，合并 HFile 减少文件数量。当小文件一直很多的时候，就会出现 "压缩风暴"。Compaction 非常消耗系统 io 资源，还会降低数据写入的速度，严重的会影响正常业务的进行。

3. MSLAB 内存消耗较大

MSLAB (MemStore-local allocation buffer) 存在于每个 MemStore 中，主要是为了解决 HBase 内存碎片问题，默认会分配 2 MB 的空间用于缓存最新数据。如果 Region 数量过多，MSLAB 总的空间占用就会比较大。比如当前节点有 1000 个包含 1 个列族的 Region，MSLAB 就会使用 1.95GB 的堆内存，即使没有数据写入也会消耗这么多内存。

25. HBase 文件合并过程

谈谈你对 HBase 合并的理解？

参考答案：

随着 MemStore 不断有数据写入，当 MemStore 的大小超过配置或默认大小的时候，HBase 会把 MemStore 中的数据刷新到 HDFS 中，生成新的 StoreFile 文件，当一个 region 中包含的 StoreFile 文件数目超过配置地的阈值，后台合并任务，会把这些文件合并成数量更少，体积更大的文件。

Minor 合并：是把最后生成的几个小的 StoreFile 文件 合并一个更大的 StoreFile，默认同时合并的文件数为 10，主要控制参数如下；

`hbase.hstore.compaction.min` 最小合并文件 数量大小，默认 3

`hbase.hstore.compaction.max` 最大合并文件 数量大小，默认 10

`hbase.hstore.compaction.max.size` 最大合并文件 的大小，默认值 为 Long.MAX_VALUE

`hbase.hstore.compaction.min.size` 最小合并文件 的大小，默认值是 memStore 刷写的大小，

它是一个阈值， 它包含所有小于限制的文件 ， 直到达到每次压缩允许的总文件数量

Major 合并：是把整个 Store 中所有 StoreFile 文件 合并成一个单独的 StoreFile 文件，后台定时任务定期检查， 这个检查周期是由下面这两个参数决定

`hbase.server.thread.wakefrequency * hbase.server.thread.wakefrequency.multiplier`(默认值为 1000)

由于 Major 合并执行期间会对整个集群的磁盘和带宽带来较大影响，一般建议设置 `hbase.hregion.majorcompaction` 设为 0 来禁用该功能，并在夜间集群负载较低时通过定时任务脚本来执行。

26.Rowkey 的作用是什么

Rowkey 的作用是什么？

参考答案：

- 读写数据时通过Row Key找到对应的Region
- MemStore 中的数据按RowKey字典顺序排序
- HFile中的数据按RowKey字典顺序排序

27.HBase 内部机制

HBase 内部机制是什么？

参考答案：

Hbase 是一个能适应联机业务的数据库系统物理存储：hbase 的持久化数据是将数据存储在 HDFS 上。

存储管理：一个表是划分为很多 region 的，这些 region 分布式地存放在很多 regionserver 上 Region 内部还可以划分为 store，store 内部有 memstore 和 storefile。

版本管理：hbase 中的数据更新本质上是不断追加新的版本，通过 compact 操作来做版本间的文件合并 Region 的 split。

集群管理：ZooKeeper + HMaster + HRegionServer。

28.如何提高 HBase 客户端读写性能

如何提高 HBase 客户端的读写性能？请举例说明

参考答案：

1. 开启 bloomfilter 过滤器，开启 bloomfilter 比没开启要快 3、4 倍
2. Hbase 对于内存有特别的需求，在硬件允许的情况下配足够多的内存给它
3. 通过修改 hbase-env.sh 中的
`export HBASE_HEAPSIZE=3000` #这里默认为 1000m
4. 增大 RPC 数量
通过修改 hbase-site.xml 中的 `hbase.regionserver.handler.count` 属性，可以适当的放大 RPC 数量，默认值为 10 有点小。

29.为什么将时间戳作为行键 HBase会发生热点问题

直接将时间戳作为行键，在写入单个 region 时候会发生热点问题，为什么呢？

参考答案：

region 中的 rowkey 是有序存储，若时间比较集中。就会存储到一个 region 中，这样一个 region 的数据变多，其它的 region 数据很少，加载数据就会很慢，直到 region 分裂，此问题才会得到缓解。

30.HBase 查询的三种方式

访问 hbase 中行的三种方式？

参考答案：

与 nosql 数据库们一样,row key 是用来检索记录的主键。只有 3 种方式：

- 1 通过单个 row key 访问（get 访问单行数据）
- 2 通过 row key 的 range（访问某个区间内的 rowkey，访问多行数据）
- 3 全表扫描（scan 访问全本的数据）

31.什么是 HBase 数据热点

什么是 Hbase 数据热点？

参考答案：

热点：检索 habse 的记录首先要通过 row key 来定位数据行。当大量的 client 访问 hbase 集群的一个或少数几个节点，造成少数 region server 的读/写请求过多、负载过大，而其他 region server 负载却很小，就造成了“热点”现象。

HBase 中，表会被划分为 $1 \cdots n$ 个 Region，被托管在 RegionServer 中。Region 有二个重要的属性：StartKey 与 EndKey 表示这个 Region 维护的 rowKey 范围，当我们要读/写数据时，如果 rowKey 落在某个 start-end key 范围内，那么就会定位到目标 region 并且读/写到相关的数据。

32.HBase 读写性能哪个高

HBase 读数据快还是写数据快？

参考答案：

HBase 写数据性能优于读数据性能，一般在一个数量级，主要由于 HBase 写数据直接写入 MemStore 缓存，然后立即返回 ack，效率非常高。HBase 读最差的情况可能需要读写磁盘，自然效率就比较低。

33.HBase 与 RDBMS 的区别

HBase 与传统关系型数据库(如 MySQL)的区别？

参考答案：

数据类型：没有数据类型，都是字节数组。

数据操作：HBase 只有很简单的插入、查询、删除、清空等操作，表和表之间是分离的，没有复杂的表和表之间的关系，而传统数据库通常有各式各样的函数和连接操作。

存储模式：Hbase 适合于非结构化数据存储，基于列存储而不是行。

数据维护：HBase 的更新操作不应该叫更新，它实际上是插入了新的数据，而传统数据库是替换修改

时间版本：Hbase 数据写入 cell 时，还会附带时间戳，默认为数据写入时 RegionServer 的时间，但是也可以指定一个不同的时间。数据可以有多个版本。

可伸缩性，Hbase 这类分布式数据库就是为了这个目的而开发出来的，所以它能够轻松增加或减少硬件的数量，并且对错误的兼容性比较高。而传统数据库通常需要增加中间层才能实现类似的功能。

34.HBase 二级索引

做过 hbase 的二级索引吗？

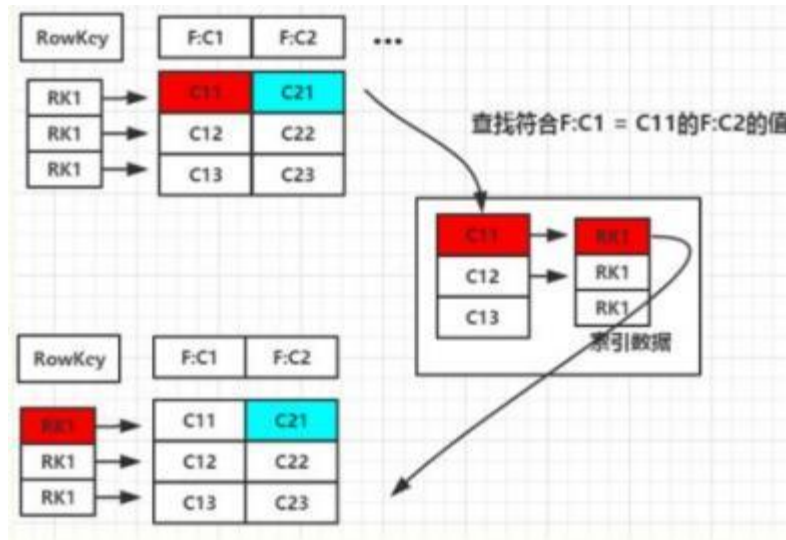
参考答案：

HBase 的一级索引就是 rowkey，我们只能通过 rowkey 进行检索。如果我们想对 hbase 里面

列族的列进行一些组合查询，就需要采用 HBase 的二级索引方案来进行多条件的查询。

二级索引的本质：就是建立各列值与行键之间的映射关系。

设计思路：



其查询步骤如下：

1. 根据 C1=C11 到索引数据中查找其对应的 RK，查询得到其对应的 RK=RK1
2. 得到 RK1 后就自然能根据 RK1 来查询 C2 的值了 这是构建二级索引大概思路，其他组合查询的联合索引的建立也类似。

其他方案：ElasticSearch+HBase 或者 solr+HBase。

35.HBase 有哪些缺点

HBase 有哪些缺点？

参考答案：

1. 单一 RowKey 固有的局限性决定了它不可能有效地支持多条件查询
2. 不适合于大范围扫描查询
3. 不直接支持 SQL 的语句查询

36.HBase 的应用场景

什么情况下适合使用 HBase 数据库（应用场景）？

参考答案：

半结构化或非结构化数据:

对于数据结构字段不够确定或杂乱无章非常难按一个概念去进行抽取的数据适合用 HBase,
因为 HBase 支持动态添加列。

记录很稀疏:

RDBMS 的行有多少列是固定的。为 null 的列浪费了存储空间。而如上文提到的, HBase 为 null 的 Column 不会被存储, 这样既节省了空间又提高了读性能。

多版本号数据:

依据 Row key 和 Column key 定位到的 Value 能够有随意数量的版本号值, 因此对于须要存储变动历史记录的数据, 用 HBase 是很方便的。比方某个用户的 Address 变更, 用户的 Address 变更记录也许也是具有研究意义的。

仅要求最终一致性:

对于数据存储事务的要求不像金融行业和财务系统这么高, 只要保证最终一致性就行。(比如 HBase+elasticsearch 时, 可能出现数据不一致)

高可用和海量数据以及很大的瞬间写入量:

WAL 解决高可用, 支持 PB 级数据, put 性能高

索引插入比查询操作更频繁的情况。比如, 对于历史记录表和日志文件。(HBase 的写操作更加高效)

业务场景简单:

不需要太多的关系型数据库特性, 列入交叉列, 交叉表, 事务, 连接等。

37..META.表数据更新对客户端有何影响

Client (客户端) 会缓存.META.表中的数据, 如果.META.表中的数据更新了怎么办?

参考答案:

其实, Client 的元数据缓存不更新, 当.META.的数据发生更新。比如因为 region 重新均衡, 某个 Region 的位置发生了变化, Client 再次根据缓存去访问的时候, 会出现错误, 当出现异常达到最大重试次数后, client 就会重新去.META.所在的 RegionServer 获取最新的 Region 信息, 如果.META.所在的 RegionServer 也变了, Client 就会重新去 ZK 上获取.META.所在的 RegionServer 的最新地址。

38.HBase Master 宕机的原因有哪些

哪些情况下会导致 HBase master 挂掉?

参考答案:

1.zk 异常会导致 master 停止服务

- 1) Zk 连接超时
- 2) 在 split region 过程中, 从 zk 删除 split 节点时
- 3) active Master 节点改变时
- 2.在 assign 时, 如果无法从.META.表中读取 region 信息
- 3.把新的 hbase 集群加入到正在运行的 hbase 集群时, 如果 zk 的/hbase/unassigned 节点没有数据;
- 4.在启动 master 的服务线程时, 出现了异常;

39.HBase RegionServer 宕机的原因有哪些

哪些情况下会导致 HBase RegionServer 挂掉?

参考答案:

- 1.在读写 hdfs 时, 如果出现 IOException 异常, 此时会发起 hdfs 的文件系统检查 (checkFileSystem)
- 2.Regionserver 的服务线程出现了未捕获异常;
- 3.在启动 HRegionServer 时出现异常;
- 4.在进行 HLog 回滚时, 出现异常;
- 5.在 flush memstore 时, 如果持久化失败, 会重启 RS, 在重启中把 hlog 的内容重新加载到 memstore;
- 6.出现 zk 异常
- 7.在 close region 时, 如果出现异常, 比如: 不能成功的 flush memstore;

40.HBase 查询速度快的原因

HBase 为什么查询快? (寻址机制)

参考答案:

根据 rowkey 查询时, 首先根据 region server 判断出该 rowkey 在不在这个 regionserver 管理的 region 的 rowkey 范围中, 这样就 能锁定 rowkey 所在的 region, 然后根据布隆过滤器可以反向找出这个 rowkey 在这个 region 的哪一个 StoreFile 中, 这样就实现了快速寻址。

41.HBase 过滤器

介绍一下 hbase 过滤器。

参考答案：

一般来说调整表设计就可以优化访问模式。但是有时已经把表设计调整得尽可能好了，为不同访问模式优化得尽可能好了。当仍然需要减少返回客户端的数据时，这就是考虑使用过滤器的时候了。

过滤器也被称为下推判断器（push-down predicates），支持把数据过滤标准从客户端下推到服务器，带有 Filter 条件的 RPC 查询请求会把 Filter 分发到各个 RegionServer，所有的过滤器都在服务端生效，使被过滤掉的数据不会被传送到客户端，这些过滤逻辑在读操作时使用，对返回客户端的数据有影响。这样可以降低网络传输的压力。

Filter 可以根据 簇、列、版本等更多的条件来对数据进行过滤，基于 HBase 本身提供的三维有序（主键有序、列有序、版本有序），这些 Filter 可以高效的完成查询过滤的任务。

42.HBase 过滤器实现原则

HBase 过滤器实现原则？

参考答案：

采用 bloomfilter 进行过滤，Bloom Filter 是一种空间效率很高的随机数据结构

1) BLOOMFILTER 在 HBase 的作用

HBase 利用 BLOOMFILTER 来提供随机读（GET）的性能，对于顺序读（Scan），设置 BLOOMFILTER 是没有作用的。

2) BLOOMFILTER 在 HBase 的开销

BLOOMFILTER 是一个列族级别的配置，如果你表中设置了 BLOOMFILTER，那么 HBase 在生成 StoreFile 时候包含一份 BLOOMFILTER 的结构数据，称为 MetaBlock；开启 BLOOMFILTER 会有一些的存储以及内存的开销。

3) BLOOMFILTER 如何提供随机读（GET）的性能

对于某个 region 的随机读，HBase 会遍历读 memstore 及 storefile（按照一定的顺序），将结果合并返回给客户端。如果你设置了 bloomfilter，那么在遍历读 storefile 时，就可以利用 bloomfilter，忽略某些 storefile。

4) Region 的 StoreFile 数目越多，BLOOMFILTER 效果越好

5) Region 下的 storefile 数目越少, HBase 读性能越好

43.HBase 过滤器有哪些特点

HBase 过滤器有哪些特点?

参考答案:

- 1.Filter 是在 HBase 服务器端上执行判断操作;
- 2.Filter 可以应用到行键 (RowFilter), 列限定符 (QualifierFilter) 或者数据值 (ValueFilter);
- 3.Filter 允许对数据分页处理 (PageFilter), 限制扫描器返回行数;
- 4.FilterList 可以组合使用多个 Filter。

11.Flume 面试

1.为什么消费者不直接消费 Flume

Flume 采集过来的数据为什么交给 Kafka, 而不是直接交给消费者消费 (比如 storm, spark)?

参考答案:

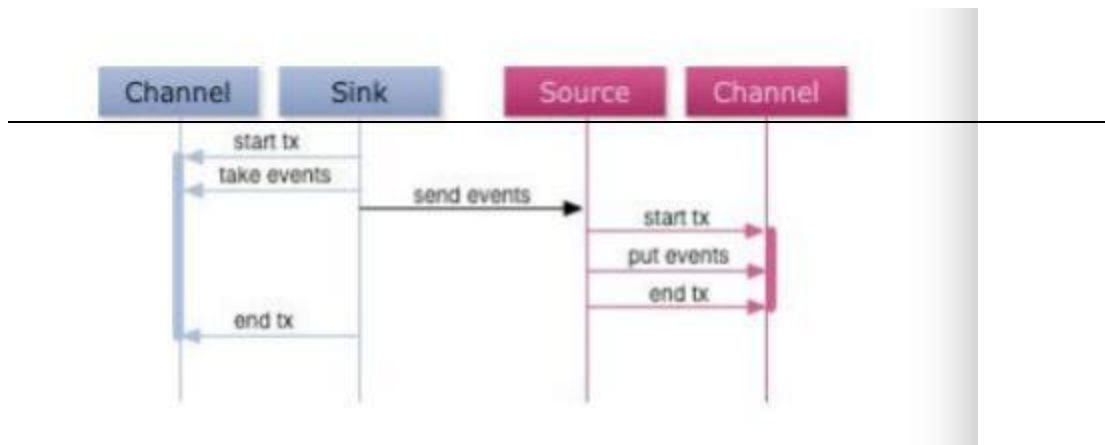
Kafka 可以调节输入接入速度与数据消费速度, 防止数据消费者出现阻塞。

2.Flume 如何保证数据不丢失

数据在 Flume 集群之间传递, 如何保证不丢失的?

参考答案:

Flume 使用事务的办法来保证 event 的可靠传递。Source 和 Sink 分别被封装在事务中, 这些事务由保存 event 的存储提供或者由 Channel 提供。这就保证了 event 在数据流的点对点传输中是可靠的。在多级数据流中, 如下图, 上一级的 Sink 和下一级的 Source 都被包含在事务中, 保证数据可靠地从一个 Channel 到另一个 Channel 转移。



其次，数据流中 Channel 的持久性。Flume 中 MemoryChannel 是可能丢失数据的（当 Agent 死掉时），而 FileChannel 是持久性的，提供类似 mysql 的日志机制，保证数据不丢失。

12.Kafka 面试

1.Kafka 如何实现高吞吐率

Kafka 如何实现高吞吐率？

参考答案：

1) 顺序读写 2) 零拷贝技术 3) 文件分段 4) 批量发送 5) 数据压缩

2.Kafka 数据可靠性如何保证

Kafka 数据可靠性如何保证？

参考答案：

对于 kafka 来说，以下几个方面来保障消息分发的可靠性：

- 1.消息发送的可靠性保障(producer)
- 2.消息消费的可靠性保障(consumer)
- 3.Kafka 集群的可靠性保障（Broker）

生产者

目前生产者发送消息(acks)有三种方式。

acks=0

producer 不等待 broker 的 acks。发送的消息可能丢失，但永远不会重发。

acks=1

leader 不等待其他 follower 同步完毕，leader 直接写 log，然后发送 acks 给 producer。这种情况下会有数据重发现象，可靠性比 only once 好点，但是仍然会丢消息。例如 leader 挂了，但是其他 replication 还没完成同步。

acks=all

leader 等待所有 follower 同步完成才返回 acks。消息可靠不丢失（丢了会重发），没收到 ack 会重发。

消费者

如果将 consumer 设置为 autocommit，consumer 一旦读到数据立即自动 commit。如果只讨论这一读取消息的过程，那 Kafka 确保了 Exactly once。

但实际使用中应用程序并非在 consumer 读取完数据就结束了，而是要进行进一步处理，而数据处理与 commit 的顺序在很大程度上决定了 consumer delivery guarantee:

At most once: 读完消息先 commit，再处理消息。

如果 consumer 在 commit 后还没来得及处理消息就 crash 了，下次重新开始工作后就无法读到刚刚已提交而未处理的消息，因为此时 offset 已经改变了。

At least once（默认）：读完消息先处理再 commit。

如果在处理完消息之后 commit 之前，consumer crash 了，下次重新开始工作时还会处理刚刚未 commit 的消息，实际上该消息已经被处理过。但即使数据可能重复，这个在应用上需要可以容忍的。

Exactly once（避免重复消费）: 保存 offset 和处理消息这两个环节采用 two-phase commit(2PC)。传统方式：把消息处理和提交 offset 做成原子操作。offset 提交失败，消息处理也要回滚。kafka 推荐方式：消息处理结果和 offset 的存储放在一起存储。比如将 offset 和处理结果一起写到 HDFS，那就可以保证数据的输出和 offset 的更新要么都完成，要么都不完成，间接实现 Exactly once。

Kafka 集群

对于 broker，落盘的数据，有 replica 机制保证数据不丢失。

对于内存(没有 flush 磁盘)数据，broker 重启会丢失。可以通过 log.flush.interval.messages 和 log.flush.interval.ms 来配置 flush 间隔，如果 interval 设置过大丢的数据多些，设置过小会影响性能。

3.Kafka 如何保证数据不丢失

生产环境中，Kafka 如何保证数据不丢失？

参考答案：

producer.type=sync 或者 async 来设置同步或者异步

1.生产者端

同步模式：确认机制设置为-1（`request.required.acks=-1`），也就是让消息写入 leader 和所有的副本。

异步模式：如果消息发出去了，但还没有收到确认的时候，缓冲池满了，在配置文件中设置成不限制阻塞超时的时间（`queue.enqueue.timeout.ms=-1`），也就说让生产端一直阻塞，这样也能保证数据不会丢失。

2.消费者端

- 1) 如果使用 spark 消费 kafka，启用 checkpoint 容错机制保证数据不丢失（保存 offset）。
- 2) 如果使用 spark 消费 kafka，业务处理完之后，再手动提交 offset，也可保证数据不丢失。

4.Kafka 如何保证消息幂等性

生产环境中，Kafka 如何保证消息幂等性问题？

参考答案：

1.Kafka 幂等性的必要性？

Producer 在生产发送消息时，难免会重复发送消息。Producer 进行 retry 时会产生重试机制，发生消息重复发送。而引入幂等性后，重复发送只会生成一条有效的消息。Kafka 作为分布式消息系统，它的使用场景常见与分布式系统中，比如消息推送系统、业务平台系统（如物流平台、银行结算平台等）。以银行结算平台来说，业务方作为上游把数据上报到银行结算平台，如果一份数据被计算、处理多次，那么产生的影响会很严重。

2.影响 Kafka 幂等性的因素有哪些？

在使用 Kafka 时，需要确保 Exactly-Once 语义。分布式系统中，一些不可控因素有很多，比如网络、OOM、FullGC 等。在 Kafka Broker 确认 Ack 时，出现网络异常、FullGC、OOM 等问题时导致 Ack 超时，Producer 会进行重复发送。可能出现的情况如下：

- 1) 先 commit，再执行业务逻辑：提交成功，处理失败。造成丢失
- 2) 先执行业务逻辑，再 commit：提交失败，执行成功。造成重复执行
- 3) 先执行业务逻辑，再 commit：提交成功，异步执行 fail。造成丢失



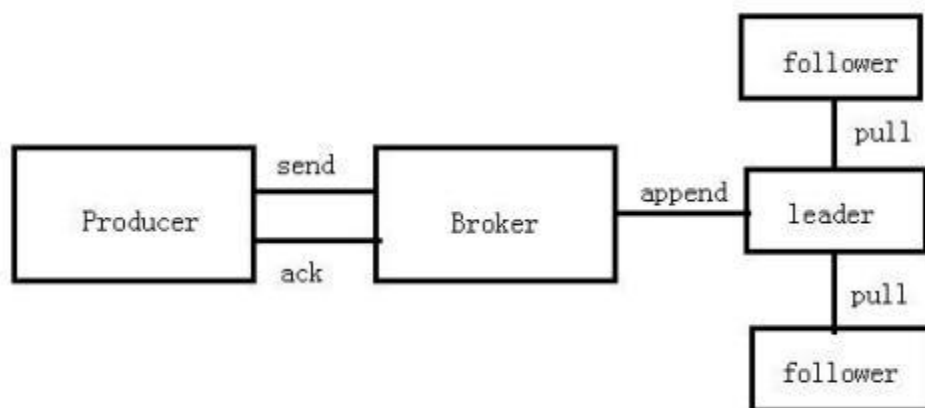
3.Kafka 的幂等性是如何实现的？

Kafka 为了实现幂等性，它在底层设计架构中引入了 ProducerID 和 SequenceNumber。

ProducerID：在每个新的 Producer 初始化时，会被分配一个唯一的 ProducerID，这个 ProducerID 对客户端使用者并不可见

SequenceNumber: 对于每个 **ProducerID**，**Producer** 发送数据的每个 **Topic** 和 **Partition** 都对应一个从 0 开始单调递增的 **SequenceNumber** 值。

4. 幂等性引入之前的问题？



目前生产者发送消息(acks)有三种方式。

1) **acks=0** (数据可能会丢失)

producer 不等待 **broker** 的 **acks**。发送的消息可能丢失，但永远不会重发。

2) **acks=1** (数据可能会丢失，也可能会重复)

leader 不等待其他 **follower** 同步完毕，**leader** 直接写 **log**，然后发送 **acks** 给 **producer**。这种情况下会有数据重发现象，可靠性比 **only once** 好点，但是仍然会丢消息。例如 **leader** 挂了，但是其他 **replication** 还没完成同步。

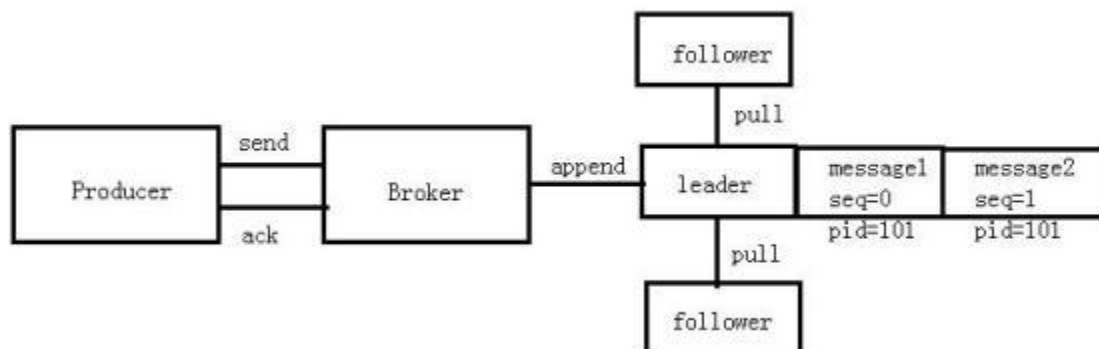
3) **acks=all** (数据可能会重复)

leader 等待所有 **follower** 同步完成才返回 **acks**。消息可靠不丢失（丢了会重发），没收到 **ack** 会重发。

5. 引入幂等性解决数据重发问题

当 **producer** 发送消息 **message2** 给 **Broker** 时，**Broker** 接受到消息并追加到 **leader** 中。数据写成功后，**Broker** 返回 **ACK** 给 **Producer**，结果发生异常导致 **Producer** 接受 **ACK** 信号失败。

对于 **Producer** 来说，会触发重试机制，将消息 **message2** 再次发送，但是，由于引入了幂等性，在每条消息中附带了 **PID** 和 **Seq**。相同的 **PID** 和 **Seq** 发送给 **Broker**，而之前 **Broker** 缓存过之前发送的相同的消息，那么在消息流中的消息就只有一条 **message2**，不会出现重复发送的情况。



6. Producer 幂等性使用

生产者要使用幂等性很简单，只需要增加以下配置即可：

```
enable.idempotence=true
```

7.Kafka 事务

Kafka 中的事务与数据库的事务类似，Kafka 中的事务属性是指一系列的 Producer 生产消息和消费消息提交 Offsets 的操作在一个事务中，即原子性操作。对应的结果是同时成功或者同时失败。

8.Kafka 事务应用场景

在 Kafka 事务中，一个原子性操作，根据操作类型可以分为 3 种情况。情况如下：

- 1)只有 Producer 生产消息，这种场景需要事务的介入。Producer 多次发送消息可以封装成一个原子性操作，即同时成功，或者同时失败；
- 2)消费消息和生产消息并存，需要事务介入。消费者&生产者模式下，因为 Consumer 在 Commit Offsets 出现问题时，导致重复消费消息，Producer 也可能重复生产消息。需要将这个模式下 Consumer 的 Commit Offsets 操作和 Producer 一系列生产消息的操作封装成一个原子性操作。
- 3)只有 Consumer 消费消息，这种操作在实际项目中意义不大，和手动 Commit Offsets 的结果一样，而且这种场景不是事务的引入目的。

9.结合业务解决 kafka 幂等性（扩展）

1. 数据写入数据库，根据主键查询，如果数据不存在就插入，如果存在就更新。
2. 数据写入 redis，每次都是 set 操作，天然幂等性。
3. 如果非上述场景，生产者给每条消息全局唯一 id，消费每条数据的时候去查一下 redis，如果没有查到就可以消费，同时 id 写入 redis；如果查到了就丢弃掉，不做任何操作。

5.Kafka 消费者组数量对性能影响

kafka 消费者组数量较大会不会对 kafka 性能造成影响？如果性能下降严重，该如何优化？

参考答案：

1.消费者和消费者组对 Kafka 性能影响

1.1 消费者数量

引入更多的 Consumer 可以提升消费性能，提升消费的吞吐量；但引入的 Consumer 越多，所属的 group 经过 rebalance 后达到稳定状态的时候也就越长，

rebalance 过程的开销就越有可能增大。不过 group 内的 Consumer 数量对于 broker 端的影响，不是那么显著。

1.2 消费者组数量

Consumer group 的个数对性能的影响肯定会有，但也很难去量化。每个 Consumer Group 都需要 Broker 端选举一个 coordinator（用来存储 Group 的相关 Meta 信息，并将对应 Partition 的 Offset 信息记录到 Kafka 内置 Topic）为其服务，另外还需要考虑每个 group id 的散列程度，避免所有的 group 都分配在相同的分区下，否则会增加该 leader broker 的压力。

2.消费者和消费者组造成性能下降如何优化

2.1 最简单的方法就是扩容

2.2 设置分区数是磁盘的整数倍，这样吞吐最高

6.如何提高 Kafka 吞吐量

有没有遇到过 Kafka 吞吐瓶颈问题？一般该如何提高 Kafka 吞吐量？

参考答案：

在 Producer 端，向每一个分区写入数据是可以完全并行化的，此时，可以通过加大硬件资源的利用率来提升系统的吞吐量，例如对数据进行压缩。单个分区的吞吐量大小会受到**批量大小**（batch.size）、**数据压缩方法**、**确认类型**（同步/异步）、**复制因子**等配置参数的影响。

在 consumer 端，kafka 只允许单个 partition 的数据同时被一个 consumer 线程消费。因此，在 consumer 端，每一个 Consumer Group 内部的 consumer 并行度完全依赖于被消费的分区数量。因此，通常情况下，在一个 Kafka 集群中，**partition 的数量**越多，意味着可以到达的吞吐量越大。单个 partition 的吞吐量依赖于 consumer 端的**应用逻辑处理速度**及**每次拉取数据量**（fetch.min.bytes）。因此，我们需要对 consumer 端的吞吐量进行测量。

7.Kafka 数据传输的事务定义有哪几种

数据传输的事务定义有哪三种？

参考答案：

数据传输的事务定义通常有以下三种级别：

（1）最多一次：消息不会被重复发送，最多被传输一次，但也有可能一次不传输。

（2）最少一次：消息不会被漏发送，最少被传输一次，但也有可能被重复传输。

（3）精确的一次（Exactly once）：不会漏传输也不会重复传输,每个消息都传输被一次而且仅仅被传输一次，这是大家所期望的。

8.Kafka 判断节点存活的条件

Kafka 判断一个节点是否还活着有哪两个条件？

参考答案:

- 1) 节点必须可以维护和 ZooKeeper 的连接, Zookeeper 通过心跳机制检查每个节点的连接。
- 2) 如果节点是个 follower, 他必须能及时的同步 leader 的写操作, 延时不能太久。

9.生产者将数据写入 Kafka 哪个节点

producer 是否直接将数据发送到 broker 的 leader(主节点)?

参考答案:

producer 直接将数据发送到 broker 的 leader(主节点), 然后 follower 节点从 leader 同步数据。

10.Kafka consumer 是否可以消费指定分区消息

Kafka consumer 是否可以消费指定分区消息?

参考答案:

Kafka consumer 消费消息时, 向 broker 发出"fetch"请求, 可以去消费特定分区的消息, consumer 指定消息在日志中的偏移量 (offset), 就可以消费从这个位置开始的消息, customer 拥有了 offset 的控制权, 可以向后回滚去重新消费之前的消息, 这是很有意义的。

11.Kafka 消息的生产与消费模式

Kafka 消息是采用 Pull 模式, 还是 Push 模式?

参考答案:

producer 将消息推送 (push) 到 broker, consumer 从 broker 拉取 (pull) 消息。

12.Kafka 高效文件存储设计特点

Kafka 高效文件存储设计特点?

参考答案:

- 1) 一个 Kafka topic 可以包含多个 partition, 一个 partition 大文件可以分成多个 segment 文件段, 通过多个小文件段就容易定

期清除或删除已经消费完文件, 减少磁盘占用。

2)通过跳表可以快速定位到 `segment` 文件段

3) 通过稀疏索引 `Index` 可以快速定位 `message` 消息

4) 稀疏索引 `Index` 可以全部加载到内存，可以避免 `segment file` 整个文件的 IO 磁盘操作。

5)通过索引文件稀疏存储，可以大幅降低 `index` 文件元数据占用空间大小。

13.Kafka 与传统消息系统区别

Kafka 与传统消息系统之间有三个关键区别

1)Kafka 持久化日志，这些日志可以被重复读取和无限期保留。

2)Kafka 是一个分布式系统：它以集群的方式运行，可以灵活伸缩，在内部通过复制数据提升容错能力和高可用性。

3)Kafka 支持实时的流式处理。

14.Kafka 中的分区放置策略

Kafka 创建 Topic 时如何将分区放置到不同的 `Broker` 中？

参考答案：

1) 副本因子不能大于 `Broker` 的个数；

2) 第一个分区（编号为 0）的第一个副本放置位置是随机从 `brokerList` 选择的；

3) 其他分区的第一个副本放置位置相对于第 0 个分区依次往后移。也就是如果我们有 5 个

`Broker`，5 个分区，假设第一个分区放在第四个 `Broker` 上，那么第二个分区将会放在第五

个 `Broker` 上；第三个分区将会放在第一个 `Broker` 上；第四个分区将会放在第二个

`Broker` 上，依次类推。

4) 剩余的副本相对于第一个副本放置位置其实是由 `nextReplicaShift` 决定的，而这个数也是随机产生的。

15.Kafka 新建分区放置位置

Kafka 新建的分区会在哪个目录下创建？

参考答案：

Kafka 会在含有分区目录最少的文件夹中创建新的分区目录，分区目录名为 Topic

名+分区 ID。注意，是分区文件夹总数最少的目录，而不是磁盘使用量最少的目录！也就

是说，如果你给 log.dirs 参数新增了一个新的磁盘，新的分区目录肯定是先在这个新的磁

盘上创建直到这个新的磁盘目录拥有的分区目录不是最少为止。

16.Kafka partition 的数据如何保存到硬盘

Kafka partition 的数据如何保存到硬盘？

参考答案：

topic 中的多个 partition 以文件夹的形式保存到 broker，每个分区序号从 0 递增，且消息有序

Partition 文件下有多个 segment (xxx.index, xxx.log)。segment 文件里的大小和配置文件大小一致，可以根据要求修改，默认为 1g。

如果 segment 大小大于 1g 时，会滚动一个新的 segment 并且以上一个 segment 最后一条消息的偏移量命名。

17.说一说 kafka 的 ack 机制

说一说 kafka 的 ack 机制？

参考答案：

request.required.acks 有三个值：0、1、-1。

1) 0: 生产者不会等待 broker 的 ack，这个延迟最低但是存储的保证最弱，当 server 挂掉的时候就会丢数据。

2) 1: 生产者会等待 broker 的 ack，leader 确认接收到消息后发送 ack，但是如果 leader

挂掉后，他不确保 follower 是否复制完成，可能会导致数据丢失。

3) -1: 生产者会等待 broker 的 ack, leader 和 follower 都收到消息后才会向生产者发送 ack, 这样数据不会丢失。

18.Kafka 的消费者如何消费数据

Kafka 的消费者如何消费数据？

参考答案：

消费者每次消费数据的时候，消费者都会记录消费的物理偏移量（offset）的位置等到下次消费时，他会接着上次位置继续消费。

19.说一说消费者负载均衡策略

说一说消费者负载均衡策略？

参考答案：

一个 kafka 分区只能被一个消费者消费，一个消费者可以消费多个 kafka 分区。

当 kafka 分区数跟消费者组中的消费者数量一致，一个消费者对应一个分区。

当 kafka 分区数大于消费者组中的消费者数量，一个消费者对应多个分区。

当 kafka 分区数小于消费者组中的消费者数量，一个消费者对应一个分区，此时消费者会有空闲。

20.说一说 Kafka 数据有序性

说一说 Kafka 数据有序性？

参考答案：

一个消费者组里它的内部数据是有序的；消费者组与消费者组之间数据是无序的。

21.kafaka 生产数据时数据的分组策略

kafaka 生产数据时数据的分组策略？

参考答案：

生产者决定数据产生到集群的哪个 partition 中，每一条消息都是以（key，value）格式，

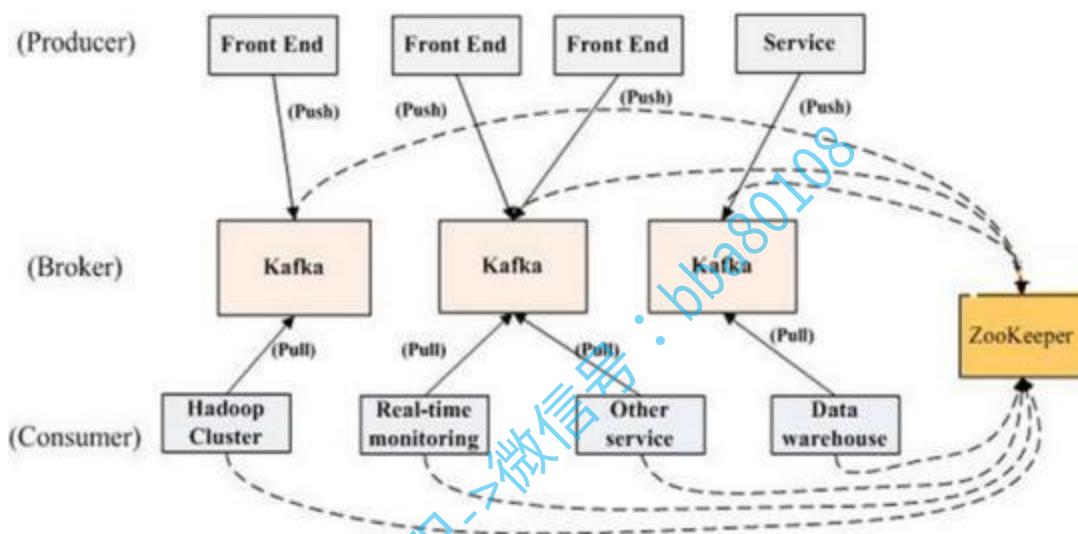
Key 是由生产者发送数据传入，所以生产者（key）决定了数据产生到集群的哪个 partition。

22.Kafka 架构

Kafka 架构设计是什么样子的？

参考答案：

一个典型的 Kafka 集群中包含若干个 Producer（可以是某个模块下，发的 Command，或者是 Web 前端产生 Page View，或者是服务器日志等），若干个 Broker（Kafka 集群支持水平扩展，一般 Broker 数量越多，整个 Kafka 集群的吞吐率也就越高），若干个 Consumer Group，以及一个 Zookeeper 集群。Kafka 通过 Zookeeper 管理集群配置。Producer 使用 push 模式将消息发布到 Broker 上，Consumer 使用 pull 模式从 Broker 上订阅并消费消息。



23.为什么要使用 kafka

为什么要使用消息队列，为什么要使用 kafka？

参考答案：

- 1) 缓冲和削峰：上游数据时有突发流量，下游可能扛不住，或者下游没有足够多的机器来保证冗余，kafka 在中间可以起到一个缓冲的作用，把消息暂存在 kafka 中，下游服务就可以按照自己的节奏进行慢慢处理。
- 2) 解耦和扩展性：项目开始的时候，并不能确定具体需求。消息队列可以作为一个接口层，解耦重要的业务流程。只需要遵守约定，针对数据编程即可获取扩展能力。
- 3) 冗余：可以采用一对多的方式，一个生产者发布消息，可以被多个订阅 topic 的服务消费

到，供多个毫无关联的业务使用。

4) 健壮性：消息队列可以堆积请求，所以消费端业务即使短时间死掉，也不会影响主要业务的正常进行。

5) 异步通信：很多时候，用户不想也不需要立即处理消息。消息队列提供了异步处理机制，允许用户把一个消息放入队列，但并不立即处理它。想向队列中放入多少消息就放多少，然后在需要的时候再去处理它们。

24.Kafka 中的 ISR、AR 代表什么

Kafka 中的 ISR、AR 又代表什么？ISR 的伸缩又指什么？

参考答案：

ISR: In-Sync Replicas 副本同步队列

AR: Assigned Replicas 所有副本

ISR 是由 leader 维护，follower 从 leader 同步数据有一些延迟（包括延迟时间 `replica.lag.time.max.ms` 和延迟条数 `replica.lag.max.messages` 两个维度，当前最新的版本 0.10.x 中只支持 `replica.lag.time.max.ms` 这个维度），任意一个超过阈值都会把 follower 剔除出 ISR，存入 OSR（Outof-Sync Replicas）列表，新加入的 follower 也会先存放在 OSR 中。
AR=ISR+OSR。

25.Kafka Broker 作用

kafka 中的 broker 是干什么的？

参考答案：

broker 是消息的代理，Producers 往 Brokers 里面的指定 Topic 中写消息，Consumers 从 Brokers 里面拉取指定 Topic 的消息，然后进行业务处理，broker 在中间起到一个代理保存消息的中转站。

26.Zookeeper 在 Kafka 集群中的作用

kafka 中的 zookeeper 起到什么作用，可以不用 zookeeper 么？

参考答案：

zookeeper 是一个分布式的协调组件，早期版本的 kafka 用 zk 做 meta 信息存储，consumer 的消费状态，group 的管理以及 offset 的值。考虑到 zk 本身的一些因素以及整个架构较大

概率存在单点问题，新版本中逐渐弱化了 zookeeper 的作用。新的 consumer 使用了 kafka 内部的 group coordination 协议，也减少了对 zookeeper 的依赖，

但是 broker 依然依赖于 ZK，zookeeper 在 kafka 中还用来选举 controller 和 检测 broker 是否存活等等。

27.kafka follower 如何与 leader 同步数据

kafka follower 如何与 leader 同步数据？

参考答案：

Kafka 的复制机制既不是完全的同步复制，也不是单纯的异步复制。完全同步复制要求 All Alive Follower 都复制完，这条消息才会被认为 commit，这种复制方式极大的影响了吞吐率。而异步复制方式下，Follower 异步的从 Leader 复制数据，数据只要被 Leader 写入 log 就被认为已经 commit，这种情况下，如果 leader 挂掉，会丢失数据，kafka 使用 ISR 的方式很好的均衡了确保数据不丢失以及吞吐率。Follower 可以批量的从 Leader 复制数据，而且 Leader 充分利用磁盘顺序读以及 send file(zero copy)机制，这样极大的提高复制性能，内部批量写磁盘，大幅减少了 Follower 与 Leader 的消息量差。

28.什么情况下一个副本会从 ISR 中踢出去

什么情况下一个副本会从 ISR 中踢出去？

参考答案：

leader 会维护一个与其基本保持同步的 Replica 列表，该列表称为 ISR(in-sync Replica)，每个 Partition 都会有一个 ISR，而且是由 leader 动态维护，如果一个 follower 比一个 leader 落后太多，或者超过一定时间未发起数据复制请求，则 leader 将其从 ISR 中移除。

29.kafka 为什么那么快

kafka 为什么那么快？

参考答案：

Kafka 写比较快：当 broker 接收到 producer 发送过来的消息时，需要根据消息的主题和分区信息，将该消息写入到该分区当前最后的 segment 文件中，文件的写入方式是追加写。由于是对 segment 文件追加写，故实现了对磁盘文件的顺序写，避免磁盘随机写时的磁盘寻道的开销，同时由于是追加写，故写入速度与磁盘文件大小无关。

kafka 读比较快:

- 1) 使用跳表, 二分查找法快速定位 segment
- 2) 使用稀疏索引+顺序扫描快速定位 message
- 3) 使用零拷贝技术将 message 快速发送给消费者消费

30.kafka producer 如何优化写入速度

kafka producer 如何优化写入速度?

参考答案:

- 1) 增加线程
- 2) 提高 batch.size
- 3) 增加更多 producer 实例
- 4) 增加 partition 数
- 5) producer 参数设置为 acks=0, 提高吞吐量, 但是消息存在丢失风险。

31.Kafka 生产者数据可靠性

kafka producer 写数据, ack 的值 0、1、-1 分别代表什么意思, ack 设置 -1 的时候, 什么情况下, leader 会认为一条消息 commit 了?

参考答案:

- 1) 1 (默认) 数据发送到 Kafka 后, 经过 leader 成功接收消息的确认, 就算是发送成功了。在这种情况下, 如果 leader 宕机了, 则会丢失数据。
- 2) 0 生产者将数据发送出去就不管了, 不去等待任何返回。这种情况下数据传输效率最高, 但是数据可靠性确是最低的。
- 3) -1 producer 需要等待 ISR 中的所有 follower 都确认接收到数据后才算一次发送完成, 可靠性最高。当 ISR 中所有 Replica 都向 Leader 发送 ACK 时, leader 才 commit, 这时候 producer 才能认为一个请求中的消息都 commit 了。

32.如果 Kafka leader crash 时, ISR 为空怎么办

如果 Kafka leader crash 时, ISR 为空怎么办?

参考答案:

kafka 在 Broker 端提供了一个配置参数, `unclean.leader.election` 这个参数有两个值:

`true` (默认): 允许不同步副本成为 leader, 由于不同步副本的消息较为滞后, 此时成为 leader, 可能会出现消息不一致的情况。

`false`: 不允许不同步副本成为 leader, 此时如果发生 ISR 列表为空, 会一直等待旧 leader 恢复, 降低了可用性。

33.kafka 的 message 是什么格式

kafka 的 message 是什么格式?

参考答案:

一个 Kafka 的 Message 由一个固定长度的 header 和一个变长的消息体 body 组成。

1) header 部分由一个字节的 magic(文件格式)和四个字节的 CRC32(用于判断 body 消息体是否正常)构成。

2) body 是由 N 个字节构成的一个消息体, 包含了具体的 key/value 消息。

34.kafka 中 consumer group 是什么

kafka 中 consumer group 是什么?

参考答案:

消费者组, 每个 Consumer 属于一个 Consumer Group; 反过来, 每个 Consumer Group 中可以包含多个 Consumer。如果所有的 Consumer 都具有相同的 Consumer Group, 那么消息将会在 Consumer 之间进行负载均衡。也就是说一个 Partition 中的消息只会被相同 Consumer Group 中的某个 Consumer 消费, 每个 Consumer Group 消息消费是相互独立的。

35.Kafka 中的消息是否会丢失和重复消费

Kafka 中的消息是否会丢失和重复消费?

参考答案:

要确定 Kafka 的消息是否丢失或重复, 从两个方面分析入手: 消息发送和消息消费。

1、消息发送

Kafka 消息发送有两种方式：同步（sync）和异步（async），默认是同步方式，可通过 `producer.type` 属性进行配置。Kafka 通过配置 `request.required.acks` 属性来确认消息的生产：

0：表示不进行消息接收是否成功的确认；

1：表示当 Leader 接收成功时确认；

-1：表示 Leader 和 Follower 都接收成功时确认；

综上所述，有 6 种消息生产的情况，下面分情况来分析消息丢失的场景：

（1）acks=0，不和 Kafka 集群进行消息接收确认，则当网络异常、缓冲区满了等情况时，消息可能丢失；

（2）acks=1、同步模式下，只有 Leader 确认接收成功后但挂掉了，副本没有同步，数据可能丢失；

2、消息消费

Kafka 消息消费有两个 consumer 接口，Low-level API 和 High-level API：

Low-level API：消费者自己维护 offset 等值，可以实现对 Kafka 的完全控制；

High-level API：封装了对 partition 和 offset 的管理，使用简单；

如果使用高级接口 High-level API，可能存在一个问题就是当消息消费者从集群中把消息取出来、并提交了新的消息 offset 值后，还没来得及消费就挂掉了，那么下次再消费时之前没消费成功的消息就“诡异”的消失了；

解决办法：

1）针对消息丢失：同步模式下，确认机制设置为-1，即让消息写入 Leader 和 Follower 之后再确认消息发送成功；异步模式下，为防止缓冲区满，可以在配置文件设置不限制阻塞超时时间，当缓冲区满时让生产者一直处于阻塞状态；

2）针对消息重复：将消息的唯一标识保存到外部介质中，每次消费时判断是否处理过即可。

36.为什么 Kafka 不支持读写分离

为什么 Kafka 不支持读写分离？

参考答案：

在 Kafka 中，生产者写入消息、消费者读取消息的操作都是与 leader 副本进行交互的，从而实现的是一种主写主读的生产消费模型。

Kafka 并不支持主写从读，因为主写从读有 2 个很明显的缺点：

1)数据一致性问题。数据从主节点转到从节点必然会有一个延时的时间窗口，这个时间窗口会导致主从节点之间的数据不一致。某一时刻，在主节点和从节点中 A 数据的值都为 X，之后将主节点中 A 的值修改为 Y，那么在这个变更通知到从节点之前，应用读取从节点中的 A 数据的值并不为最新的 Y，由此便产生了数据不一致的问题。

2)延时问题。类似 Redis 这种组件，数据从写入主节点到同步至从节点中的过程需要经历网络→主节点内存→网络→从节点内存这几个阶段，整个过程会耗费一定的时间。而在 Kafka 中，主从同步会比 Redis 更加耗时，它需要经历网络→主节点内存→主节点磁盘→网络→从节点内存→从节点磁盘这几个阶段。对延时敏感的应用而言，主写从读的功能并不太适用。

37.Kafka 中是怎么体现消息顺序性的

Kafka 中是怎么体现消息顺序性的？

参考答案：

kafka 每个 partition 中的消息在写入时都是有序的，消费时，每个 partition 只能被每一个 group 中的一个消费者消费，保证了消费时也是有序的。

整个 topic 不保证有序，如果为了保证 topic 整个有序，那么将 partition 调整为 1。

38.Kafka 中有那些地方需要选举

Kafka 中有那些地方需要选举？

参考答案：

1) kafka 在所有 broker 中选出一个 controller。

2) 所有 Partition 的 Leader 选举都由 controller 决定。

controller 会将 Leader 的改变直接通过 RPC 的方式通知需为此作出响应的 Broker。同时 controller 也负责增删 Topic 以及 Replica 的重新分配。

39.Kafka 中为什么使用偏移量

Kafka 中为什么使用偏移量？

参考答案：

给分区中的消息提供了一个顺序 ID 号，我们称之为偏移量。因此，为了唯一地识别分区中的每条消息，我们使用这些偏移量。

40.为什么 Kafka 的分区副本至关重要

为什么 Kafka 的分区副本至关重要？

参考答案：

使用副本，我们可以确保发布的消息不会丢失，并且可以在发生任何机器错误、程序错误或频繁的软件升级时使用。

41.如果副本长时间不在 ISR 中，这意味着什么

如果副本长时间不在 ISR 中，这意味着什么？

参考答案：

简单地说，这意味着 follower 不能像 leader 收集数据那样快速地获取数据。

42.说一说 RabbitMQ 与 Kafka 的区别与联系

说一说 RabbitMQ 与 Kafka 的区别与联系？

参考答案：

联系：Apache Kafka 和 RabbitMQ 都是消息队列。

区别：

1.功能

- 1) Kafka 是分布式的、持久的和高度可用的，这里共享和复制数据
- 2) RabbitMQ 中没有此类功能

2.性能速度

- 1) Kafka - 达到每秒 100000 条消息。
- 2) RabbitMQ - 每秒 20000 条消息。

43.Kafka 的流处理是什么意思

Kafka 的流处理是什么意思？

参考答案：

连续、实时、并发和以逐记录方式处理数据的类型，我们称之为 **Kafka** 流处理。

44. 如何获取 topic 主题的列表

如何获取 topic 主题的列表？

参考答案：

```
bin/kafka-topics.sh --list --zookeeper localhost:2181
```

45. 消费者故障，出现活锁问题如何解决

消费者故障，出现活锁问题如何解决？

参考答案：

出现“活锁”的情况，是它持续的发送心跳，但是没有处理。为了预防消费者在这种情况下一直持有分区，我们使用 `max.poll.interval.ms` 活跃检测机制。在此基础上，如果你调用的 `poll` 的频率大于最大间隔，则客户端将主动地离开组，以便其他消费者接管该分区。发生这种情况时，你会看到 `offset` 提交失败（调用 `commitSync()` 引发的 `CommitFailedException`）。这是一种安全机制，保障只有活动成员能够提交 `offset`。所以要留在组中，你必须持续调用 `poll`。

46. Kafka 如何保证消息的顺序消费

kafka 分布式（不是单机）的情况下，如何保证消息的顺序消费？

参考答案：

Kafka 分布式的单位是 `partition`，同一个 `partition` 用一个 `write ahead log` 组织，所以可以保证 `FIFO` 的顺序。不同 `partition` 之间不能保证顺序。但是绝大多数用户都可以通过 `message key` 来定义，因为同一个 `key` 的 `message` 可以保证只发送到同一个 `partition`。

Kafka 中发送 1 条消息的时候，可以指定 `(topic, partition, key)` 3 个参数。`partition` 和 `key` 是可选的。如果你指定了 `partition`，那就是所有消息发往同 1 个 `partition`，就是有序的。并且在消费端，Kafka 保证，1 个 `partition` 只能被 1 个 `consumer` 消费。或者你指定 `key`（比如 `order id`），具有同 1 个 `key` 的所有消息，会发往同 1 个 `partition`。

47.Kafka 都有哪些特点

Kafka 都有哪些特点？

参考答案：

高吞吐量、低延迟：kafka 每秒可以处理几十万条消息，它的延迟最低只有几毫秒，每个 topic 可以分多个 partition, consumer group 对 partition 进行 consume 操作。

可扩展性：kafka 集群支持热扩展

持久性、可靠性：消息被持久化到本地磁盘，并且支持数据备份防止数据丢失

容错性：允许集群中节点失败（若副本数量为 n,则允许 n-1 个节点失败）

高并发：支持数千个客户端同时读写

48.请简述下你在哪些场景下会选择 Kafka

请简述下你在哪些场景下会选择 Kafka？

参考答案：

日志收集：一个公司可以用 Kafka 可以收集各种服务的 log，通过 kafka 以统一接口服务的方式开放给各种 consumer，例如 hadoop、HBase、Solr 等。

消息系统：解耦和生产者和消费者、缓存消息等。

用户活动跟踪：Kafka 经常被用来记录 web 用户或者 app 用户的各种活动，如浏览网页、搜索、点击等活动，这些活动信息被各个服务器发布到 kafka 的 topic 中，然后订阅者通过订阅这些 topic 来做实时的监控分析，或者装载到 hadoop、数据仓库中做离线分析和挖掘。

运营指标：Kafka 也经常用来记录运营监控数据。包括收集各种分布式应用的数据，生产各种操作的集中反馈，比如报警和报告。

流式处理：比如 spark streaming 和 Flink

49.Kafka 分区的目的

Kafka 分区的目的？

参考答案：

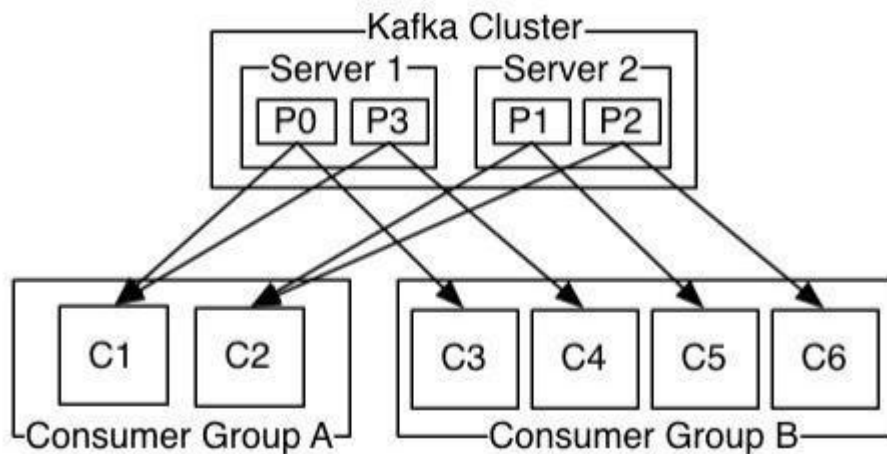
分区对于 Kafka 集群的好处是：实现负载均衡。分区对于消费者来说，可以提高并发度，提高效率。

50.消费者和消费者组有什么关系

消费者和消费者组有什么关系？

参考答案：

每个消费者从属于消费组。



51.谈一谈 Kafka 的再均衡

谈一谈 Kafka 的再均衡

参考答案：

在 Kafka 中，当有新消费者加入或者订阅的 topic 数发生变化时，会触发 Rebalance(再均衡：在同一个消费者组当中，分区的所有权从一个消费者转移到另外一个消费者)机制，Rebalance 顾名思义就是重新均衡消费者消费。Rebalance 的过程如下：

第一步：所有成员都向 coordinator 发送请求，请求入组。一旦所有成员都发送了请求，coordinator 会从中选择一个 consumer 担任 leader 的角色，并把组成员信息以及订阅信息发给 leader。

第二步：leader 开始分配消费方案，指明具体哪个 consumer 负责消费哪些 topic 的哪些 partition。一旦完成分配，leader 会将这个方案发给 coordinator。coordinator 接收到分配方案之后会把方案发给各个 consumer，这样组内的所有成员就都知道自己应该消费哪些分区了。

所以对于 Rebalance 来说，Coordinator 起着至关重要的作用

52.Kafka 监控都有哪些

Kafka 监控都有哪些？

参考答案：

- 1) Apache Kafka 监控之 KafkaOffsetMonitor
 - 2) 雅虎开源的 Kafka 集群管理器(Kafka Manager)
 - 3) Apache Kafka 监控之 Kafka Web Console
-
- 4) JMX 监控

53.如何为 Kafka 集群选择合适的 Topics/Partitions 数量

如何为 Kafka 集群选择合适的 Topics/Partitions 数量？

参考答案：

首先我们需要明白以下事实：在 kafka 中，单个 partition 是 kafka 并行操作的最小单元。在 producer 和 broker 端，向每一个分区写入数据是可以完全并行化的，此时，可以通过加大硬件资源的利用率来提升系统的吞吐量，例如对数据进行压缩。在 consumer 段，kafka 只允许单个 partition 的数据被一个 consumer 线程消费。因此，在 consumer 端，每一个 Consumer Group 内部的 consumer 并行度完全依赖于被消费的分区数量。综上所述，通常情况下，在一个 Kafka 集群中，partition 的数量越多，意味着可以到达的吞吐量越大。

1.越多的分区可以提供更高的吞吐量

我们可以粗略地通过吞吐量来计算 kafka 集群的分区数量。假设对于单个 partition，producer 端的可达吞吐量为 p ，Consumer 端的可达吞吐量为 c ，期望的目标吞吐量为 t ，那么集群所需要的 partition 数量至少为 $\max(t/p, t/c)$ 。在 producer 端，单个分区的吞吐量大小会受到批量大小、数据压缩方法、确认类型（同步/异步）、复制因子等配置参数的影响。经过测试，在 producer 端，单个 partition 的吞吐量通常是在 10MB/s 左右。在 consumer 端，单个 partition 的吞吐量依赖于 consumer 端每个消息的应用逻辑处理速度。因此，我们需要对 consumer 端的吞吐量进行测量。

2.越多的分区需要打开更多地文件句柄

在 kafka 的 broker 中，每个分区都会对照着文件系统的一个目录。在 kafka 的数据日志文件目录中，每个日志数据段都会分配两个文件，一个索引文件和一个数据文件。当前版本的 kafka，每个 broker 会为每个日志段文件打开一个 index 文件句柄和一个数据文件句柄。因此，随着 partition 的增多，需要底层操作系统配置更高的文件句柄数量限制。这更多的是一个配置问题。我们曾经见到过，在生产环境 Kafka 集群中，每个 broker 打开的文件句柄数量超过 30,000。

3.更多地分区会导致更高的不可用性

4.越多的分区可能增加端对端的延迟

5.越多的 partition 意味着需要客户端需要更多的内存

54.Kafka 有哪些缺点

Kafka 有哪些缺点？

参考答案：

- 1) 由于是批量发送，数据并非真正的实时；
- 2) 对于 mqtt 协议不支持；
- 3) 不支持物联网传感数据直接接入；
- 4) 仅支持统一分区内消息有序，无法实现全局消息有序；
- 5) 监控不完善，需要安装插件；
- 6) 依赖 zookeeper 进行元数据管理；

55.Kafka 新旧消费者的区别

Kafka 新旧消费者的区别？

参考答案：

旧的 Kafka 消费者 API 主要包括：SimpleConsumer（简单消费者）和 ZookeeperConsumerConnector（高级消费者）。SimpleConsumer 名字看起来是简单消费者，但是其实用起来很不简单，可以使用它从特定的分区和偏移量开始读取消息。高级消费者和现在新的消费者有点像，有消费者群组，有分区再均衡，不过它使用 ZK 来管理消费者群组，并不具备偏移量和再均衡的可操控性。

56.Kafka 分区数可以增加或减少吗

Kafka 分区数可以增加或减少吗？为什么？

参考答案：

我们可以使用 bin/kafka-topics.sh 命令对 Kafka 增加 Kafka 的分区数据，但是 Kafka 不支持减少分区数。

Kafka 分区数据不支持减少是由很多原因的，比如减少的分区其数据放到哪里去？是删除，还是保留？删除的话，那么这些没消费的消息不就丢了。如果保留这些消息如何放到其他分区里面？追加到其他分区后面的话那么就破坏了 Kafka 单个分区的有序性。如果要保证删除分区数据插入到其他分区保证有序性，那么实现起来逻辑就会非常复杂。

57.谈谈你对 Kafka 幂等的了解

谈谈你对 Kafka 幂等的了解？

参考答案：

Kafka 旧版本只能支持两种语义：At most once 和 At least once，Kafka 在 0.11.0.0 版本支持增加了对幂等的支持。幂等是针对生产者角度的特性。幂等可以保证生产者发送的消息，不会丢失，而且不会重复

Kafka 幂等性实现原理：

为了实现 Producer 的幂等性，Kafka 引入了 Producer ID（即 PID）和 Sequence Number。

1) PID。每个新的 Producer 在初始化的时候会被分配一个唯一的 PID，这个 PID 对用户是不可见的。

2) Sequence Number。（对于每个 PID，该 Producer 发送数据的每个<Topic, Partition>都对应一个从 0 开始单调递增的 Sequence Number

Kafka 可能存在多个生产者，会同时产生消息，但对 Kafka 来说，只需要保证每个生产者内部的消息幂等就可以了，所有引入了 PID 来标识不同的生产者。

对于 Kafka 来说，要解决的是生产者发送消息的幂等问题。也即需要区分每条消息是否重复。Kafka 通过为每条消息增加一个 Sequence Number，通过 Sequence Number 来区分每条消息。每条消息对应一个分区，不同的分区产生的消息不可能重复。所有 Sequence Number 对应每个分区

Broker 端在缓存中保存了这 seq number，对于接收的每条消息，如果其序号比 Broker 缓存中序号大 1 则接受它，否则将其丢弃。这样就可以实现了消息重复提交了。但是，只能保证单个 Producer 对于同一个<Topic, Partition>的 Exactly Once 语义。不能保证同一个 Producer 一个 topic 不同的 partition 幂等。

13.Spark 面试

1.Spark 数据倾斜

对于 Spark 中的数据倾斜问题你有什么好的方案？

参考答案：

1.定位数据倾斜，看是 OOM 原因，还是任务执行缓慢的原因，看日志、Web UI。

2.解决方案

1) 避免不必要的 shuffle，如使用广播小表的方式，将 reduce-side-join 提升为 map-side-join。

2) 分拆发生数据倾斜的记录，分成几个部分进行，然后合并 join 后的结果。

3) 改变并行度，可能并行度太少了，导致个别 task 数据压力大。

4) 两阶段聚合，先局部聚合，再全局聚合。

5) 自定义 partitioner，分散 key 的分布，使其更加均匀。

2.RDD 工作原理

1. Spark RDD 是什么？

2. RDD 是如何提高数据处理的效率的？

3. 画出 RDD 的工作原理流程？

参考答案：

1. Spark 编程模型是弹性分布式数据集（Resilient Distributed Dataset，RDD），它是 MapReduce 模型的扩展和延伸，但它解决了 MapReduce 的缺陷：在并行计算阶段高效地进行数据共享。

一个 RDD 就是一个分布式对象集合，本质上是一个只读的分区记录集合，每个 RDD 可分成多个分区，每个分区就是一个数据集片段，并且一个 RDD 的不同分区可以被保存到集群中不同的节点上，从而可以在集群中的不同节点上进行并行计算。RDD 提供了一种高度受限的共享内存模型，即 RDD 是只读的记录分区的集合，不能直接修改，只能基于稳定的物理存储中的数据集创建 RDD，或者通过在其他 RDD 上执行确定的转换操作（如 map、join 和 group by）而创建得到新的 RDD。

2. RDD 可以包含多个分区，进行并行计算；RDD 有很多缓存级别；RDD 窄依赖可以进行流水线执行，提高执行效率。

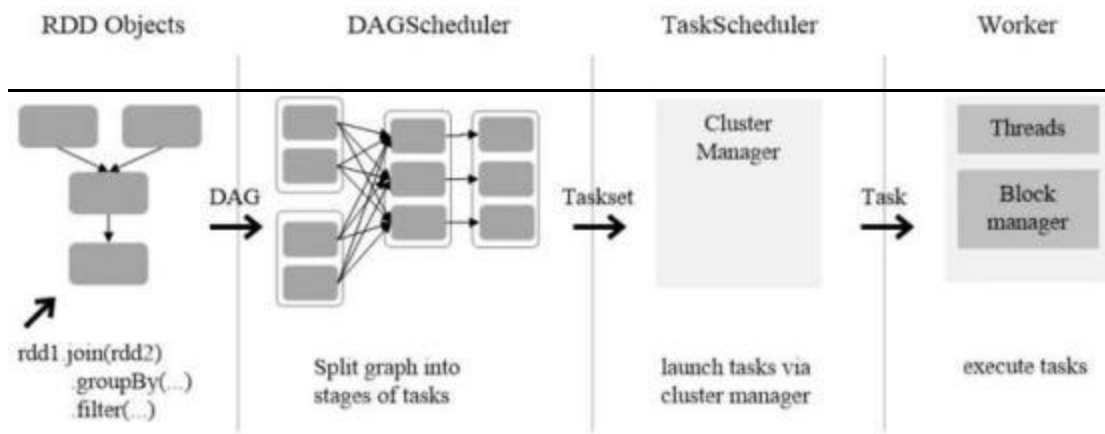
3.

RDD 典型的执行过程如下：

- 1) RDD 读入外部数据源进行创建，利于使用 `textFile` 函数加载本地数据；
- 2) RDD 经过一系列的转换（Transformation）操作，每一次都会产生不同的 RDD，供给狭义转换作使用，这里的 Transformation 操作就是 `filter` 函数；
- 3) 最后一个 RDD 经过“动作”（Action）操作进行转换，并输出到外部数据源。

RDD 在 Spark 架构中的运行过程：

- 1) 创建 RDD 对象；
- 2) `SparkContext` 负责计算 RDD 之间的依赖关系，构建 DAG；
- 3) `DAGScheduler` 负责把 DAG 图分解成多个 Stage，每个 Stage 中包含了多个 Task，每个 Task 会被 `TaskScheduler` 分发给各个 `WorkerNode` 上的 `Executor` 去执行。



3.reduceByKey 和 groupbyKey 的区别

Spark 中的 `reduceByKey` 和 `groupbyKey` 的区别？

参考答案：

reduceByKey: `reduceByKey` 会在结果发送至 reducer 之前会对每个 mapper 在本地进行 merge，有点类似于在 MapReduce 中的 combiner。这样做的好处在于，在 map 端进行一次 reduce 之后，数据量会大幅度减小，从而减小传输，保证 reduce 端能够更快的进行结果计算。

groupByKey: `groupByKey` 会对每一个 RDD 中的 value 值进行聚合形成一个序列(iterator)，此操作发生在 reduce 端，所以势必会将所有的数据通过网络进行传输，造成不必要的浪费。同时如果数据量十分大，可能还会造成 `OutOfMemoryError`。

通过以上对比可以发现在进行大量数据的 reduce 操作时候建议使用 `reduceByKey`。不仅可以提高速度，还是可以防止使用 `groupByKey` 造成的内存溢出问题。

4.RDD 数据丢失如何重算

RDD 做如下转换流程：RDD A——>RDD B——>RDD C——>RDD D

如果 RDD D 中的分区数据丢失，是只需要在 RDD C 的分区上重算？还是需要从 RDD A 开始从头重新计算？

参考答案：

如果 RDD D 中的分区数据丢失，在 B 或 C 都没有做 cache 的情况下会从头开始计算，也就是从 A 开始计算。如果 C 做了 cache，那么就会从 C 开始计算。

RDD 不做 cache 的话可以这么理解：变换前后的新旧 RDD 的分区，在物理上可能是同一块内存存储，所以后面的 RDD 会覆盖前面 RDD 的内存，所以 RDD 是个瞬时状态，生成新的 RDD 后它自己就会消失，所以必要的时候可以对 RDD 做 cache 持久化。

5.谈谈 spark 中的宽窄依赖

谈谈 spark 中的宽窄依赖？

参考答案：

1.宽依赖和窄依赖概述

窄依赖就是一个父 RDD 分区对应一个子 RDD 分区，如 map，filter 或者多个父 RDD 分区对应一个子 RDD 分区，如 co-partitioned join。

宽依赖是一个父 RDD 分区对应非全部的子 RDD 分区，如 groupByKey，reduceByKey 或者一个父 RDD 分区对应全部的子 RDD 分区，如未经协同划分的 join。

2.窄依赖的优势

1) 窄依赖相比宽依赖更高效资源消耗更少

允许在单个集群节点上流水线式执行，这个节点可以计算所有父级分区。例如，可以逐个元素地依次执行 filter 操作和 map 操作（多步操作在同一个节点可以执行，否则第一步和第二步不在一个节点操作）。

而宽依赖则需要首先计算好所有父分区数据，然后在节点之间进行 Shuffle，这与 MapReduce 类似（map 都计算完毕，才计算 reduce）。

2) 在窄依赖中，节点失败后的恢复更加高效。

因为只有丢失的父级分区需要重新计算，并且这些丢失的父级分区可以并行地在不同节点上重新计算。

而对于一个宽依赖关系的 Lineage 图，单个节点失效导致这个 RDD 丢失部分分区，可能所有祖先 RDD 因此需要整体重新计算。

6.哪些 spark 算子会有 shuffle

哪些 spark 算子会有 shuffle？

参考答案：

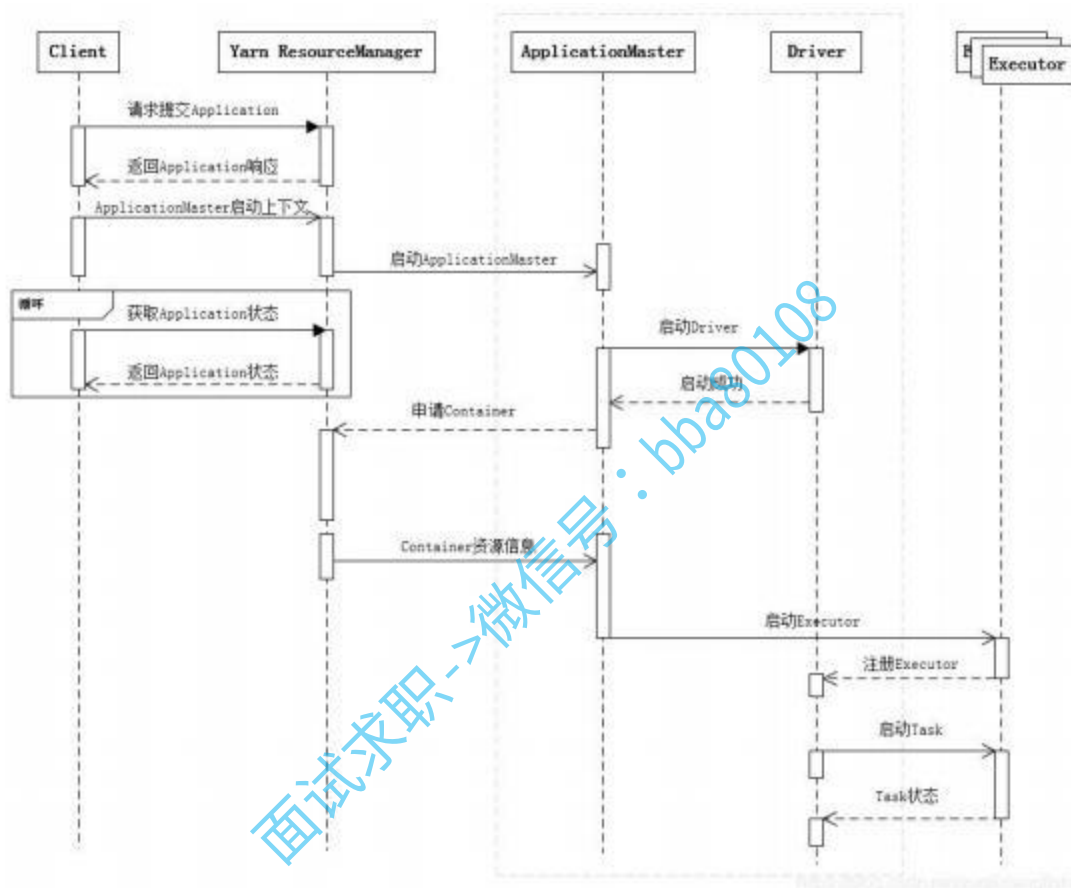
以下 Spark 算子会出现 shuffle 过程：

- 1) 去重, distinct
- 2) 排序, groupByKey, reduceByKey 等
- 3) 重分区, repartition, coalesce
- 4) 集合或者表操作, interection, join

7.谈一谈 spark (ON YARN) scheduler (任务调度)

谈一谈 spark (ON YARN) scheduler (任务调度) ?

参考答案:



1.提交一个 Spark 应用程序, 首先通过 Client 向 ResourceManager 请求启动一个 Application, 同时检查是否有足够的资源满足 Application 的需求, 如果资源条件满足, 则准备 ApplicationMaster 的启动上下文, 交给 ResourceManager, 并循环监控 Application 状态。

2.当提交的资源队列中有资源时, ResourceManager 会在某个 NodeManager 上启动 ApplicationMaster 进程, ApplicationMaster 会单独启动 Driver 后台线程, 当 Driver 启动后, ApplicationMaster 会通过本地的 RPC 连接 Driver, 并开始向 ResourceManager 申请 Container 资源运行 Executor 进程 (一个 Executor 对应与一个 Container), 当 ResourceManager 返回 Container 资源, 则在对应的 Container 上启动 Executor。

3.Driver 线程主要是初始化 SparkContext 对象，准备运行所需的上下文，然后一方面保持与 ApplicationMaster 的 RPC 连接，通过 ApplicationMaster 申请资源，另一方面根据用户业务逻辑开始调度任务，将任务下发到已有的空闲 Executor 上。

5. 当 ResourceManager 向 ApplicationMaster 返回 Container 资源时，ApplicationMaster 就尝试在对应的 Container 上启动 Executor 进程，Executor 进程起来后，会向 Driver 注册，注册成功后保持与 Driver 的心跳，同时等待 Driver 分发任务，当分发的任务执行完毕后，将任务状态上报给 Driver。

Driver 把资源申请的逻辑给抽象出来，以适配不同的资源管理系统，所以才间接地通过 ApplicationMaster 去和 Yarn 打交道。Client 只管提交 Application 并监控 Application 的状态。对于 Spark 的任务调度主要是集中在两个方面：资源申请和任务分发，其主要是通过 ApplicationMaster、Driver 以及 Executor 之间来完成。

8.讲一下 Spark Stage 是如何划分的

讲一下 Spark Stage 是如何划分的？

参考答案：

1) 一个 spark 应用包含很多个 job，Spark 所有操作是针对 RDD，对 RDD 可以有两种计算操作算子：Transformation（转换）和 Action（行动），Action 算子会触发 Spark 提交作业（Job），每个 action 的计算都会生成一个 job。

2) 用户提交的 Job 会提交给 DAGScheduler，Job 会被分解成 Stage 和 Task。

3) 一个 Job 会被拆分为多组 Task，每组任务被称为一个 Stage。Stage 的划分是以 shuffle 和 result 这两种类型来划分。

在 Spark 中有两类 task，一类是 shuffleMapTask，一类是 resultTask，第一类 task 的输出是 shuffle 所需数据，第二类 task 的输出是 result，stage 的划分也以此为依据，shuffle 之前的所有变换是一个 stage，shuffle 之后的操作是另一个 stage。

9.RDD 弹性及缺陷是什么

RDD 是弹性数据集，“弹性”体现在哪里呢？你觉得 RDD 有哪些缺陷？

参考答案：

1.RDD 弹性

1) RDD 自动进行内存和磁盘切换

2) RDD 基于 lineage 的高效容错

3) 如果子 RDD 计算失败或者丢失, 会根据父 RDD 重新计算, 而且只会计算失败或者丢失的分区

4) 数据调度弹性: DAG task 和资源管理无关

5) 数据分片的高度弹性 repartition

2.RDD 缺陷

中间数据默认不会保存, 每次动作操作都会对数据重复计算, 某些计算量比较大的操作可能会影响到系统的运算效率。

10.Groupbykey 和 reducebykey 哪个性能更高

Groupbykey 和 reducebykey 哪个性能更高, 为什么?

参考答案:

reduceByKey: reduceByKey 会在结果发送至 reducer 之前会对每个 mapper 在本地进行 merge, 有点类似于在 MapReduce 中的 combiner。这样做的好处在于, 在 map 端进行一次 reduce 之后, 数据量会大幅度减小, 从而减小传输, 保证 reduce 端能够更快的进行结果计算。

groupByKey: groupByKey 会对每一个 RDD 中的 value 值进行聚合形成一个序列(iterator), 此操作发生在 reduce 端, 所以势必会将所有的数据通过网络进行传输, 造成不必要的浪费。同时如果数据量十分大, 可能还会造成 OutOfMemoryError。

通过以上对比可以发现在进行大量数据的 reduce 操作时候建议使用 reduceByKey。不仅可以提高速度, 还是可以防止使用 groupByKey 造成的内存溢出问题。

结论: reduceByKey 性能高与 groupbykey, 更适合大数据集。

11.说说 spark 比 hadoop 有哪些优势

说说 spark 比 hadoop 有哪些优势?

参考答案:

从以下角度分析

- 1、内存计算 vs 离线计算
- 2、DAG 执行优化
- 3、线程模型与进程模型

- 4、延时调度

比较 Hadoop 与 Spark

1. Hadoop

- 1) Hadoop 的 MapReduce 支持离线计算
- 2) Hadoop 不支持实时计算
- 3) Hadoop 有机器学习库 Mahout
- 4) Hadoop 中的 hive 支持 SQL 操作
- 5) Hadoop 不支持图计算
- 6) MapReduce 计算需要不断读写磁盘，计算模型比较单一，仅仅包含 map reduce 模型
- 7) Hadoop 包含 hdfs 存储和 YARN 资源调度

2. Spark

- 1) Spark core 支持离线计算
- 2) Spark Streaming 支持实时计算
- 3) Spark 有机器学习库 ML
- 4) Spark DataFrame 支持 SQL
- 5) Spark 支持图计算
- 6) Spark 支持内存计算，包含很多丰富的算子，而且延迟计算，提供优化空间，提高计算效率
- 7) 生产环境中 Spark ON YARN 模式运行

12.RDD 特性

spark 中的 RDD 是什么，有哪些特性？

参考答案：

1. RDD (Resilient Distributed Dataset) 叫做分布式数据集，是 spark 中最基本的数据抽象，它代表一个不可变，可分区，里面的元素可以并行计算的集合。

2. RDD 五大特性

A list of partitions: RDD 是一个由多个 partition 组成的 List

A function for computing each split: RDD 的每个 partition 上面都会有 function，也就是函数应用，其作用是实现 RDD 之间 partition 的转换。

A list of dependencies on other RDDs: RDD 会记录它的依赖，为了容错，也就是说在内存中的 RDD 操作时出错或丢失会进行重算。

Optionally, a Partitioner for Key-value RDDs: 可选项，如果 RDD 里面存的数据是 key-value 形式，则可以传递一个自定义的 Partitioner 进行重新分区。

Optionally, a list of preferred locations to compute each split on: 最优的位置去计算，也就是保持数据的本地性。

13.概述一下 spark 中的常用算子区别

概述一下 spark 中的常用算子区别（map, mapPartitions, foreach, foreachPartition）？

参考答案：

map：用于遍历 RDD，将函数应用于每一个元素，返回新的 RDD（transformation 算子）

foreach：用于遍历 RDD，将函数应用于每一个元素，无返回值（action 算子）

mapPartitions：用于遍历操作 RDD 中的每一个分区，返回生成一个新的 RDD（transformation 算子）

foreachPartition：用于遍历操作 RDD 中的每一个分区，无返回值（action 算子）

总结：一般使用 mapPartitions 和 foreachPartition 算子比 map 和 foreach 更加高效，推荐使用。

14.Transformation 和 Action 算子有什么区别

Transformation 和 Action 算子有什么区别？举例说明

参考答案：

Transformation 变换/转换：这种变换并不触发提交作业，完成作业中间过程处理。Transformation 操作是延迟计算的，也就是说从一个 RDD 转换生成另一个 RDD 的转换操作不是马上执行，需要等到有 Action 操作的时候才会真正触发运算。比如 map, filter 等算子。

Action 行动算子：这类算子会触发 SparkContext 提交 Job 作业。Action 算子会触发 Spark 提交作业（Job）。比如 print, count, saveAsText 等算子。

15.spark 中 cache 和 persist 的区别与联系

spark 中 cache 和 persist 的区别与联系？

参考答案：

联系：cache 和 persist 都是用于将一个 RDD 进行缓存的，这样在之后使用的过程中就不需要重新计算了，可以大大节省程序运行时间。

区别: cache 调用了 persist 方法, cache 只有一个默认的缓存级别 MEMORY_ONLY , 而 persist 可以根据情况设置多种缓存级别。

16.RDD 有多少种持久化方式

RDD 有多少种持久化方式? memory_only 持久化方式, 如果内存存储不了, 会怎么操作?

参考答案:

1. persist 操作, 可以将 RDD 持久化到不同层次的存储介质, 以便后续操作重复使用。

1) cache:RDD[T] (调用了 persist 方法)

2) persist:RDD[T]

3) persist(level:StorageLevel):RDD[T]

首次使用 RDD 的时候, 我们可以选择对 RDD 进行持久化, 当再次使用 RDD 是就可以直接从之前的缓存中获取而无需再次进行计算。对于需要反复使用的 RDD 会带来很大的性能改善。

StorageLevel 存储级别包含如下类别:

级别	说明
MEMORY_ONLY	默认级别, 非序列化方式存储在jvm堆内存中, 如果无法内存里放不下, 则部分partition将不会缓存, 会在每次需要时重新计算
MEMORY_AND_DISK	非序列化方式存储到jvm堆内存, 内存不够时缓存到磁盘
MEMORY_ONLY_SER	以序列化方式存储到jvm堆内存, 占用空间小, 但读取时需要耗费cpu来反序列化
MEMORY_AND_DISK_SER	与MEMORY_ONLY_SER类似, 不同的是他会把内存无法容纳的数据写到磁盘
DISK_ONLY	只缓存到磁盘
MEMORY_ONLY_2, MEMORY_AND_DISK_2, etc.	与上述对应的持久化方式相同, 不同的是会为分区创建副本以保持两份拷贝

2. memory_only 默认级别, 非序列化方式存储在 jvm 堆内存中, 如果无法内存里放不下, 则部分 partition 将不会缓存, 会在每次需要时重新计算。

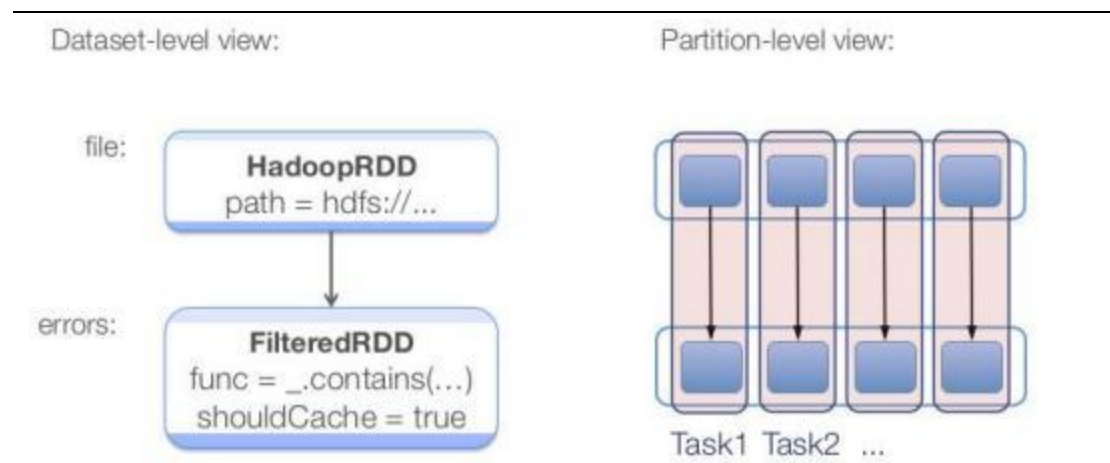
17.Spark RDD 分区和数据块 (block) 有啥联系

Spark RDD 分区和数据块 (block) 有啥联系?

参考答案:

RDD 是 Spark 中的概念, block 是 hdfs 中的概念。

这里 hdfs 有 4 个数据块，以下图为例



当 Spark 读取 hdfs 文件系统的数据时，4 个 block 一般对应 4 个分片，每个分片形成一个 partition，也就是一个 RDD 包含 4 个 partition。

18. Spark cache 一定能提升计算性能么

Spark cache 一定能提升计算性能么？说明原因？

参考答案：

不一定，cache 是将数据缓存到内存里，当小数据量的时候是能提升效率，但数据大的时候内存放不下就会报溢出。

19. Spark 为什么要对数据进行序列化

Spark 为什么要对数据进行序列化，有什么优缺点？

参考答案：

优点：序列化可以减少数据的体积，减少存储空间，高效存储和传输数据。

缺点：使用的时候要反序列化，非常消耗 CPU。

20. Spark 为什么要持久化

Spark 为什么要持久化，一般什么场景下要进行 persist 操作？

参考答案：

Spark RDD 中间不产生临时数据，但分布式系统风险很高，所以容易出错，rdd 出错或者分片可以根据血统算出来，如果没有对父 rdd 进行 persist 或者 cache 的化，就需要重头做。

以下场景会使用 persist

- 1) 某个步骤计算非常耗时，需要进行 persist 持久化。
- 2) 计算链条非常长，重新恢复要算很多步骤，很好使，persist。
- 3) checkpoint 所在的 rdd 要持久化 persist。
- 4) shuffle 之后要 persist，shuffle 要进行网络传输，风险很大，数据丢失重来，恢复代价很大。
- 5) shuffle 之前进行 persist，框架默认将数据持久化到磁盘，这个是框架自动做的。

21. Spark 怎么修改默认 task 执行个数

Spark 程序执行，有时候默认为什么会产生很多 task，怎么修改默认 task 执行个数？

参考答案：

1) 因为输入数据有很多 task，尤其是有很多小文件的时候，有多少个输入 block 就会有多少个 task 启动；

2) spark 中有 partition 的概念，每个 partition 都会对应一个 task，task 越多，在处理大规模数据的时候，就会越有效率。不过 task 并不是越多越好，如果平时测试，或者数据量没有那么大，则没有必要 task 数量太多。

3) 参数可以通过 spark_home/conf/spark-default.conf 配置文件设置：

```
#针对 spark sql 的 task 数量
spark.sql.shuffle.partitions 50
#非 spark sql 程序设置生效
spark.default.parallelism 10
```

22.Spark collect 功能是什么

Spark collect 功能是什么，其底层是怎么实现的？

参考答案：

driver 通过 collect 把集群中各个节点的内容收集过来汇总成结果，collect 返回结果是 Array 类型的。collect 把各个节点上的数据抓过来，抓过来数据是 Array 型，collect 对 Array 抓过来的结果进行合并，合并后 Array 中只有一个元素，是 tuple 类型（KV 类型的）的。

23.Spark 的数据本地性有哪几种

Spark 的数据本地性有哪几种？

参考答案：

Spark 中的数据本地性有三种

- 1) PROCESS_LOCAL 是指读取缓存在本地节点的数据
- 2) NODE_LOCAL 是指读取本地节点硬盘数据
- 3) ANY 是指读取非本地节点数据

通常读取数据 PROCESS_LOCAL>NODE_LOCAL>ANY，尽量使数据以 PROCESS_LOCAL 或 NODE_LOCAL 方式读取。其中 PROCESS_LOCAL 还和 cache 有关，如果 RDD 经常用的话将该 RDD cache 到内存中，注意，由于 cache 是 lazy 的，所以必须通过一个 action 的触发，才能真正的将该 RDD cache 到内存中。

24.Spark 中数据的位置是被谁管理的

Spark 中数据的位置是被谁管理的？

参考答案：

每个数据分片都对应具体物理位置，数据的位置，无论数据是在磁盘，还是内存，都是由 blockManager 管理。

25.Spark 并行度怎么设置比较合适

Spark 并行度怎么设置比较合适？

参考答案：

spark 并行度，每个 core 承载 2~4 个 partition，如有有 32 个 core，那么就是 64~128 之间的并行度，也就是设置 64~128 个 partition，并行度和数据规模无关，只和内存和 cpu 有关。

26. Spark 如何从 Kafka 中获取数据

Spark 如何从 Kafka 中获取数据？

参考答案：

1) 基于 Receiver 的方式

这种方式使用 Receiver 来获取数据。Receiver 是使用 Kafka 的高层次 Consumer API 来实现的。receiver 从 Kafka 中获取的数据都是存储在 Spark Executor 的内存中的，然后 Spark Streaming 启动的 job 会去处理那些数据。

2) 基于 Direct 的方式

这种新的不基于 Receiver 的直接方式，是在 Spark 1.3 中引入的，从而能够确保更加健壮的机制。替代掉使用 Receiver 来接收数据后，这种方式会周期性地查询 Kafka，来获得每个 topic+partition 的最新的 offset，从而定义每个 batch 的 offset 的范围。当处理数据的 job 启动时，就会使用 Kafka 的简单 consumer api 来获取 Kafka 指定 offset 范围的数据。

备注：spark0.10 之后，Spark 基于 Direct 方式使用新消费者 api 消费 kafka 数据。

27. 说说 Spark 的特点，相对于 MR 来说

说说 Spark 的特点，相对于 MR 来说？

参考答案：

1. 减少磁盘 I/O，MR 会把 map 端将中间输出和结果存储在磁盘中，reduce 端又需要从磁盘读写中间结果，势必造成磁盘 I/O 称为瓶颈。Spark 允许将 map 端的中间结果输出和结果存储在内存中，reduce 端在拉取中间结果的时候避免了大量的磁盘 I/O。

2. 增加并行度，由于把中间结果写到磁盘与从磁盘读取中间结果属于不同的缓解，Hadoop 将他们简单地通过串行执行衔接起来，Spark 则把不同的环节抽象成为 Stage，允许多个 Stage

既可以串行又可以并行执行。

3.避免重新计算，当 Stage 中某个分区的 Task 执行失败后，会重新对此 Stage 调度，但在重新调度的时候会过滤已经执行成功的分区任务，所以不会造成重复计算和资源浪费。

4.可选的 Shuffle 排序，MR 在 Shuffle 之前有着固定的排序操作，而 Spark 则可以根据不同场景选择在 map 端排序还是 reduce 排序。

5.灵活的内存管理策略，Spark 将内存分为堆上的存储内存、堆外的存储内存，堆上的执行内存，堆外的执行内存 4 个部分。

28.groupByKey 和 reduceByKey 是属于哪种算子

groupByKey 和 reduceByKey 是属于 Transformation 还是 Action?

参考答案:

前者，因为 Action 输出的不再是 RDD 了，也就意味着输出不是分布式的，而是回送到 Driver 程序。以上两种操作都是返回 RDD，所以应该属于 Transformation。

常见的 Transformation 和 Action 操作

Transformation: Map, Filter, FlatMap, Sample, GroupByKey, ReduceByKey, Union, Join, Cogroup, MapValues, Sort, PartitionBy

Action: Collect, Reduce, Lookup, Save

29.Spark 为什么快，Spark SQL 一定比 Hive 快吗

Spark 为什么快，Spark SQL 一定比 Hive 快吗?

参考答案:

Spark SQL 比 Hadoop Hive 快，是有一定条件的，而且不是 Spark SQL 的引擎比 Hive 的引擎快，相反，Hive 的 HQL 引擎还比 Spark SQL 的引擎更快。其实，关键还是在于 Spark 本身快。

消除了冗余的 HDFS 读写: Hadoop 每次 shuffle 操作后，必须写到磁盘，而 Spark 在 shuffle 后不一定落盘，可以 cache 到内存中，以便迭代时使用。如果操作复杂，很多的 shuffle 操作，那么 Hadoop 的读写 IO 时间会大大增加，也是 Hive 更慢的主要原因了。

消除了冗余的 MapReduce 阶段: Hadoop 的 shuffle 操作一定连着完整的 MapReduce 操作, 冗余繁琐。而 Spark 基于 RDD 提供了丰富的算子操作, 且 reduce 操作产生 shuffle 数据, 可以缓存在内存中。

JVM 的优化: Hadoop 每次 MapReduce 操作, 启动一个 Task 便会启动一次 JVM, 基于进程的操作。而 Spark 每次 MapReduce 操作是基于线程的, 只在启动 Executor 是启动一次 JVM, 内存的 Task 操作是在线程复用的。每次启动 JVM 的时间可能就需要几秒甚至十几秒, 那么当 Task 多了, 这个时间 Hadoop 不知道比 Spark 慢了多少。

30.说说 SparkContext 和 SparkSession 有什么区别

说说 SparkContext 和 SparkSession 有什么区别?

参考答案:

SparkContext: 整个应用的上下文, 控制应用的生命周期。

SparkSession 是在 Spark 2.0 中引入的, 它使开发人员可以轻松地使用它, 这样我们就不用担心不同的上下文, 并简化了对不同上下文的访问。通过访问 SparkSession, 我们可以自动访问 SparkContext。

31.driver 的功能是什么

driver 的功能是什么?

参考答案:

1) 一个 Spark 作业运行时包括一个 Driver 进程, 也是作业的主进程, 具有 main 函数, 并且有 SparkContext 的实例, 是程序的入口点;

2) 功能: 负责向集群申请资源, 向 master 注册信息, 负责了作业的调度, 负责作业的解析、生成 Stage 并调度 Task 到 Executor 上。包括 DAGScheduler, TaskScheduler。

32.Spark 为什么比 mapreduce 快

Spark 为什么比 mapreduce 快?

参考答案:

1) 基于内存计算, 减少低效的磁盘交互;

2) 高效的调度算法, 基于 DAG;

3) 容错机制 Linage, 精华部分就是 DAG 和 Linage

33.sparkStreaming 如何调优

sparkStreaming 如何调优?

参考答案:

1.资源参数调优

2.增加并行度 parallelism:增加 spark partition 数量

3.设置合理的批处理时间和 Kafka 数据拉取速率 maxRatePerPartition

4.使用 Kryo 序列化

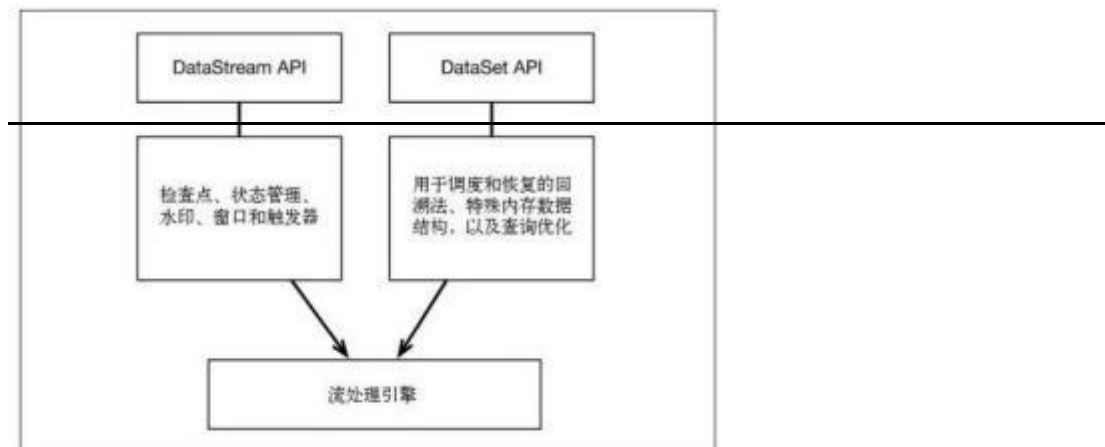
5.缓存需要经常使用的数据

6.spark 优化之广播变量

14.Flink 面试

1.Flink 是如何支持批流一体的?

Flink 通过一个底层引擎同时支持流处理和批处理.



在流处理引擎之上，Flink 有以下机制：

- 1.检查点机制和状态机制：用于实现容错、有状态的处理；
- 2.水印机制：用于实现事件时钟；
- 3.窗口和触发器：用于限制计算范围，并定义呈现结果的时间。

在同一个流处理引擎之上，Flink 还存在另一套机制，用于实现高效的批处理。

- 1.用于调度和恢复的回溯法：由 Microsoft Dryad 引入，现在几乎用于所有批处理器；
- 2.用于散列和排序的特殊内存数据结构：可以在需要时，将一部分数据从内存溢出到硬盘上；
- 3.优化器：尽可能地缩短生成结果的时间。

2.Flink 是如何做容错的？

Flink 实现容错主要靠强大的 CheckPoint 机制和 State 机制。

Checkpoint 负责定时制作分布式快照、对程序中的状态进行备份；State 用来存储计算过程中的中间状态。

3.Flink 分布式快照的原理是什么？

消息准确一次(exactly once)的语义保证是实时计算的关键特性之一，这要求作业从失败恢复后的状态以及管道中的数据流要和失败时一致，通常这是通过定期对作业状态和数据流进行快照实现的。

然而这种方式主要有两点不足：

- 1) 快照进行期间常常要暂停数据流的摄入，造成额外延迟和吞吐量下降；
- 2) 快照会过度谨慎地将管道里正在计算的数据也随着状态保存下来，导致快照过于庞大。

针对以上两个问题，Flink 引入了异步数据栅栏快照 (Asynchronous Barrier Snapshot, ABS)。

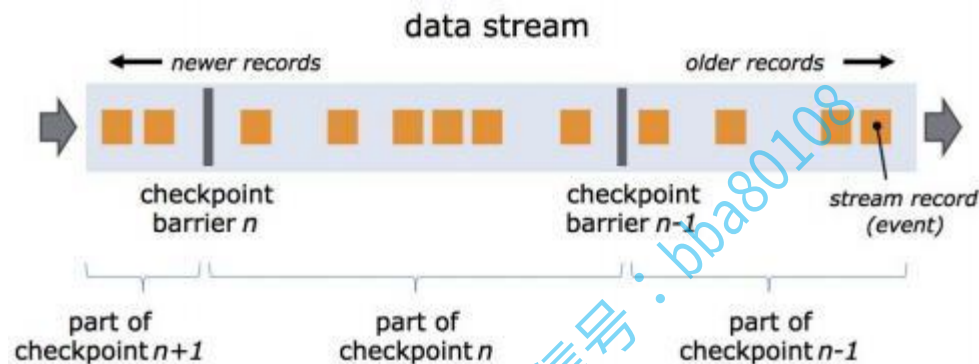
ABS 是一种轻量级的快照技术，能以低成本备份 DAG (有向无环图) 或 DCG (有向有环图)。

计算作业的状态，这使得计算作业可以频繁进行快照并且不会对性能产生明显影响。ABS 核心思想是通过数据栅栏(barrier)来标记触发快照的时间点和对应的数据，从而将数据流和快照时间解耦以实现异步快照操作，同时也大大降低了对管道数据的依赖（对 DAG 类作业甚至完全不依赖），减小了随之而来的快照大小。

1.barrier 机制

Flink 使用 checkpoint 做快照，借助于 barrier（数据栅栏）。

- 1) barrier（数据栅栏）是一种特殊的内部消息，用于将数据流从时间上切分为多个窗口，每个窗口对应一系列连续的快照中的一个。
- 2) barrier 由 JobManager 定时广播给计算任务所有的 Source，其后伴随数据流一起流至下游。
- 3) 每个 barrier 是属于当前快照的数据与属于下个快照的数据的分割点。



每当接收到 barrier，算子便会将对当前的状态做一次快照，成功后将 barrier 以广播形式传给下游。最后 barrier 会流至作业的 Sink，Sink 接受到 barrier 后即向 JobManager 确认，JobManager 收到所有 Sink 的确认标志着一个完整快照的生成。

快照完成后，当前的快照窗口随之关闭，算子不会再向上游请求早于这个时间的数据，因为这些数据已经流过整个作业，被转化为作业状态的一部分。

4.Flink 是如何保证 Exactly-once 语义的？

Exactly Once 语义：

Exactly-Once：指的是每个输入的事件只影响最终结果一次。即使机器或软件出现故障，既没有重复数据，也不会丢数据。

1.Flink 应用程序中的 Exactly-Once 语义

Flink 利用 checkpoint 对从 source—>operator—>sink 提供一次完整的容错。

Flink checkpoint 快照包含如下内容：

- 1) 对于并行输入数据源：快照创建时数据流中的位置偏移
- 2) 对于 operator：存储在快照中的状态指针

Flink 可以配置一个固定的时间点，定期产生 checkpoint，将 checkpoint 的数据写入持久存储系统（HDFS）。将 checkpoint 数据写入持久存储是异步发生的，这意味着 Flink 应用程序在 checkpoint 过程中可以继续处理数据。

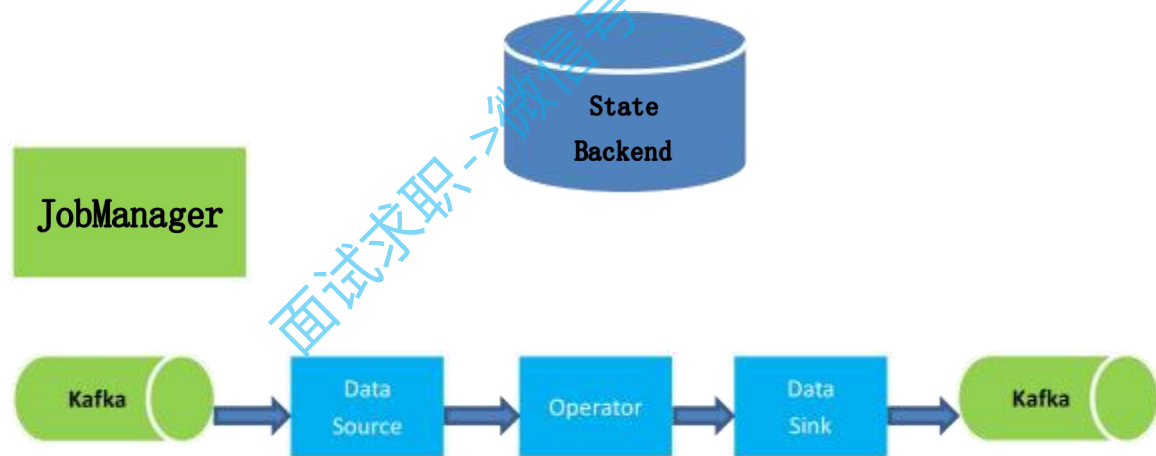
如果发生机器或软件故障，重新启动后，Flink 应用程序将从最新的 checkpoint 点恢复处理；Flink 会恢复应用程序状态，将输入流回滚到上次 checkpoint 保存的位置（比如 kafka offset），然后重新开始运行。这意味着 Flink 可以像从未发生过故障一样计算结果。

总结：在 Flink 应用程序内部，利用 checkpoint 提供了 Exactly Once。

2. Flink 应用程序端到端的 Exactly-Once 语义

通过 Flink 的 TwoPhaseCommitSinkFunction 两阶段提交协议能支持端到端（KafkaSource，KafkaSink）的 Exactly-Once 语义。

Exactly Once two-phase commit（两阶段提交）



从 Kafka 读取的数据源——>转换操作——>将数据写回 Kafka 的数据输出端

要使数据输出端提供 Exactly-Once 保证，它必须将所有数据通过一个事务提交给 Kafka。提交捆绑了两个 checkpoint 之间的所有要写入的数据。这可确保在发生故障时能回滚写入的数

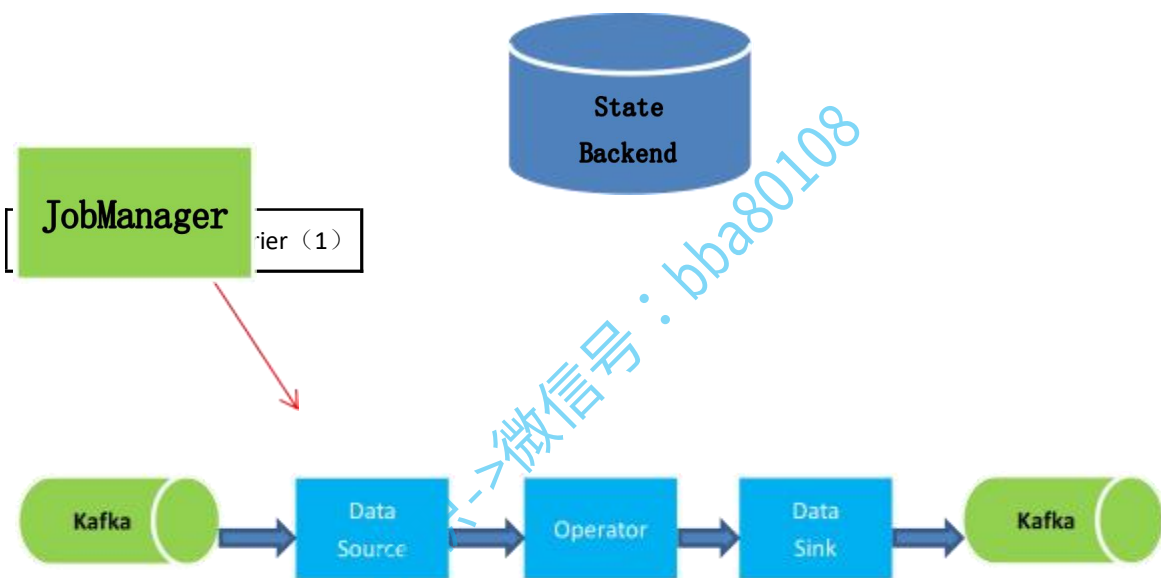
据。Flink 使用两阶段提交协议及预提交阶段来解决这个问题。

Pre-commit （预提交阶段）

在 checkpoint 开始的时候(starting Checkpoint)，即两阶段提交协议的“**预提交**”阶段。当 checkpoint 开始时，Flink 的 JobManager 会将 checkpoint barrier（将数据流中的记录分为进入当前 checkpoint 与进入下一个 checkpoint）注入数据流。

barrier 在 operator 之间传递。对于每一个 operator，它触发 operator 的状态快照写入到 state backend。

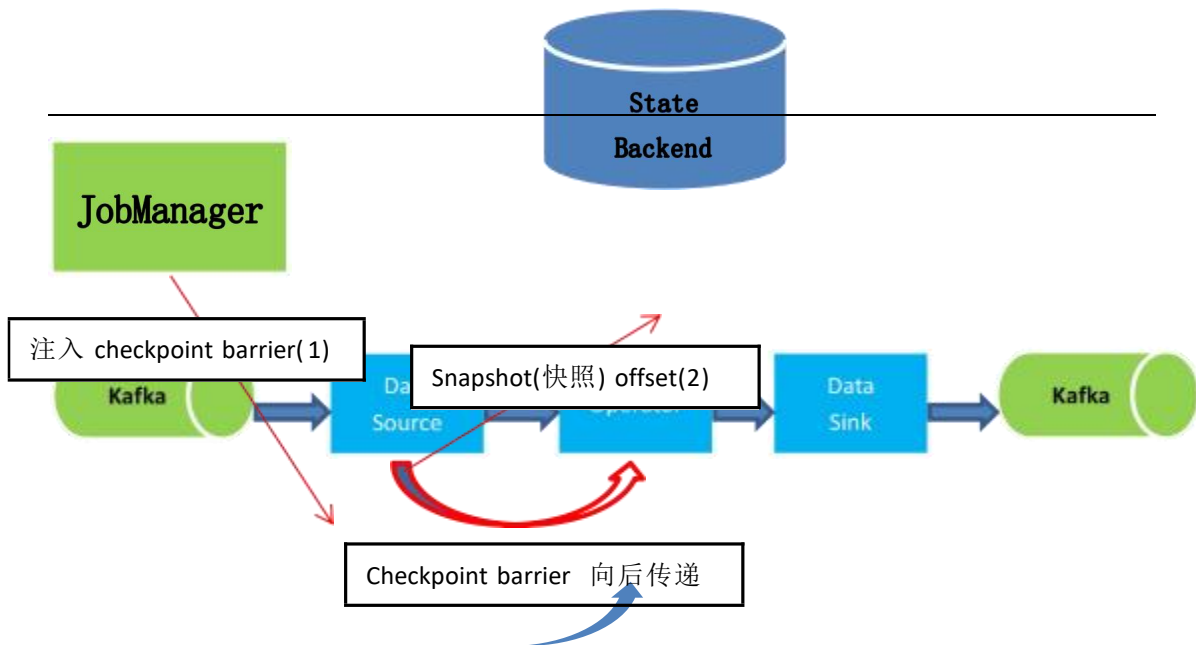
Pre-commit(Checkpoint starts)



数据源保存了消费 Kafka 的偏移量(offset)，之后将 checkpoint barrier 传递给下一个 operator。

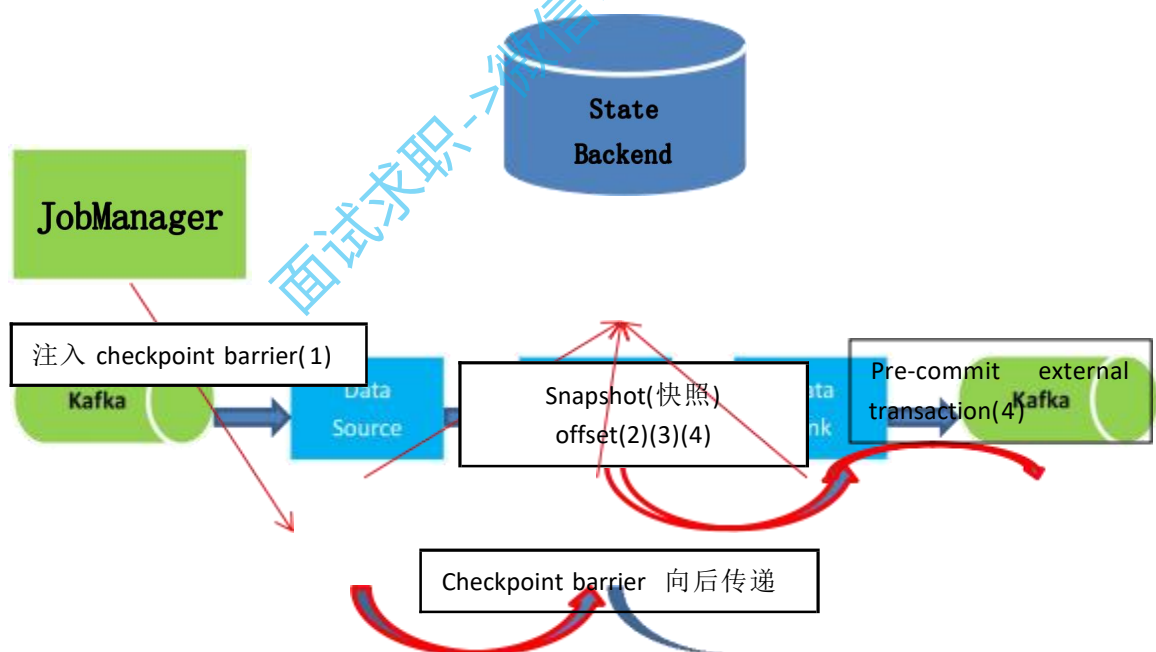
在 operator 过程中，比如聚合计算得到了 sum 值，此时表明 operator 具有内部状态，那么需要将 sum 值写入 state Backend，Flink 负责在 checkpoint 成功的情况下正确提交这些写入的数据到 JobManager，或者在出现故障时中止这些写入的数据 JobManager。但在 checkpoint 成功之前，不需要在预提交阶段执行任何其他操作。

Pre-commit without external state （Checkpoint in Progress）



当输出端（DataSink）有输出数据（写入 Kafka），就具有了外部状态，需要做些额外的处理，为了提供 Exactly-Once 保证，在预提交阶段，除了将其状态写入 state backend 之外，数据输出端还必须预先提交其外部事务，这样才能和两阶段提交协议集成。

Pre-commit with external state in data sink



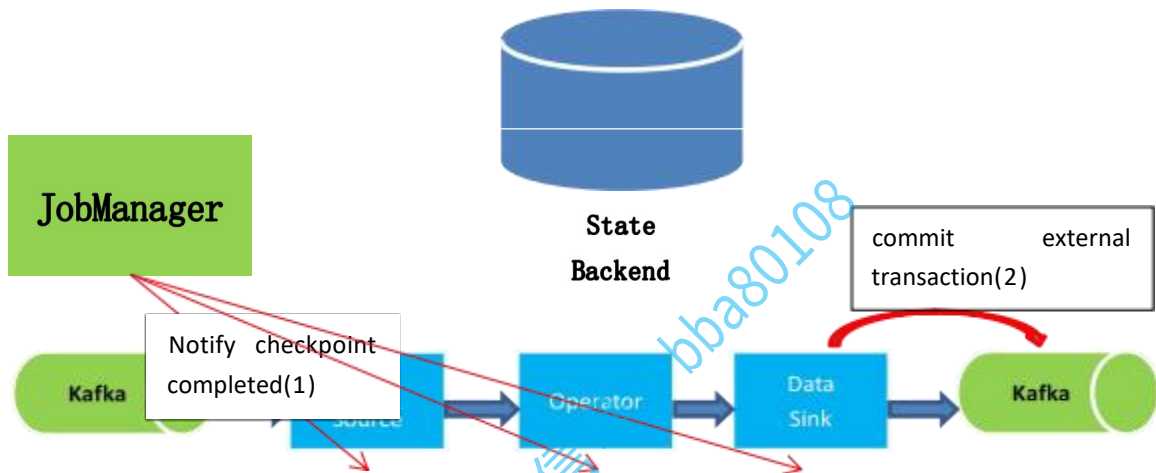
当 checkpoint barrier 在所有 operator 都传递了一遍，并且触发的 checkpoint 回调成功完成时，预提交阶段就结束了。所有触发的状态快照都被视为该 checkpoint 的一部分。checkpoint 是

整个应用程序状态的快照，包括预先提交的外部状态。如果发生故障，我们可以回滚到上次成功完成快照的时间点。

commit （提交阶段）

Job Manager 然后通知所有操作（包括 DataSource、Operator、DataSink），checkpoint 已经成功了。这是两阶段提交协议的提交阶段，Job Manager 为应用程序中的每个 operator 发出 checkpoint 已完成的回调。

数据源（Data Source）和 Operator 没有外部状态，因此在提交阶段，这些 operator 不必执行任何操作。但是，数据输出端（Data Sink）拥有外部状态，此时应该提交外部事务。



总结：

- 1) 一旦所有 operator 完成预提交（Pre-commit），才会提交一个 commit。
- 2) 如果至少有一个预提交（Pre-commit）失败，则所有其他提交都将中止，我们将回滚到上一个成功完成的 checkpoint。
- 3) 在预提交（Pre-commit）成功之后，提交的 commit 需要保证最终成功（operator 和外部系统都需要保障这点）。如果 commit 失败（比如，由于间歇性网络问题），整个 Flink 应用程序将失败，应用程序将根据用户的重启策略重新启动（Flink 会将 operator 的状态恢复到已经预提交，但尚未真正提交的状态。），还会尝试再提交（commit）。这个过程至关重要，因为如果 commit 最终没有成功，将会导致数据丢失。

5.说说 Flink 的序列化如何做的?

Java 本身自带的序列化和反序列化的功能，但是辅助信息占用空间比较大，在序列化对象时记录了过多的类信息。

Apache Flink 摒弃了 Java 原生的序列化方法，以独特的方式处理数据类型和序列化，包含自己的类型描述符，泛型类型提取和类型序列化框架。

TypeInformation 是所有类型描述符的基类。它揭示了该类型的一些基本属性，并且可以生成序列化器。

TypeInformation 支持以下几种类型：

BasicTypeInfo：任意 Java 基本类型或 String 类型

BasicArrayTypeInfo：任意 Java 基本类型数组或 String 数组

WritableTypeInfo：任意 Hadoop Writable 接口的实现类

TupleTypeInfo：任意的 Flink Tuple 类型(支持 Tuple1 to Tuple25)。Flink tuples 是固定长度固定类型的 Java Tuple 实现

CaseClassTypeInfo：任意的 Scala CaseClass(包括 Scala tuples)

PojoTypeInfo：任意的 POJO (Java or Scala)，例如，Java 对象的所有成员变量，要么是 public 修饰符定义，要么有 getter/setter 方法

GenericTypeInfo：任意无法匹配之前几种类型的类

针对前六种类型数据集，Flink 皆可以自动生成对应的 TypeSerializer，能非常高效地对数据集进行序列化和反序列化。

6.Flink 中的 Window 出现了数据倾斜，你有什么解决办法？

window 产生数据倾斜指的是数据在不同的窗口内堆积的数据量相差过多。本质上产生这种情况的原因是数据源头发送的数据量速度不同导致的。出现这种情况一般通过两种方式来解决：

- (1) 在数据进入窗口前做预聚合
- (2) 重新设计窗口聚合的 key

7.Flink 中在使用聚合函数 GroupBy、Distinct、KeyBy 等函数时出现数据热点该如何解决？

参考答案：

数据倾斜和数据热点是所有大数据框架绕不过去的问题。处理这类问题主要从 3 个方面入手：

- (1) 在业务上规避这类问题

例如一个假设订单场景，北京和上海两个城市订单量增长几十倍，其余城市的数据量不变。

这时候我们在进行聚合的时候，北京和上海就会出现数据堆积，我们可以单独处理北京和上海的数据。

(2) Key 的设计上把热 key 进行拆分，比如上个例子中的北京和上海，可以把北京和上海按照地区进行拆分聚合。

(3) 参数设置

Flink 1.9.0 SQL(Blink Planner) 性能优化中一项重要的改进就是升级了微批模型，即 MiniBatch。原理是缓存一定的数据后再触发处理，以减少对 State 的访问，从而提升吞吐和减少数据的输出量。

8.Flink 任务延迟高，想解决这个问题，你会如何入手？

在 Flink 的后台任务管理中，我们可以看到 Flink 的哪个算子和 task 出现了反压。最主要的手段是资源调优和算子调优。

资源调优即是对作业中的 Operator 的并发数 (parallelism)、CPU (core)、堆内存 (heap_memory) 等参数进行调优。

作业参数调优包括：并行度的设置，State 的设置，checkpoint 的设置。

9.Flink 是如何处理反压的？

Flink 内部是基于 producer-consumer 模型来进行消息传递的，Flink 的反压设计也是基于这个模型。Flink 使用了高效有界的分布式阻塞队列，就像 Java 通用的阻塞队列 (BlockingQueue) 一样。下游消费者消费变慢，上游就会受到阻塞。

Flink 什么情况下才会把 Operator chain 在一起形成算子链？

参考答案：

两个 operator chain 在一起的条件：

- (1) 上下游的并行度一致
- (2) 下游节点的入度为 1 (也就是说下游节点没有来自其他节点的输入)
- (3) 上下游节点都在同一个 slot group 中 (下面会解释 slot group)
- (4) 下游节点的 chain 策略为 ALWAYS (可以与上下游链接，map、flatmap、filter 等默认是 ALWAYS)
- (5) 上游节点的 chain 策略为 ALWAYS 或 HEAD (只能与下游链接，不能与上游链接，Source 默认是 HEAD)
- (6) 两个节点间数据分区方式是 forward (参考理解数据流的分区)

(7) 用户没有禁用 chain

10.说说 Flink 资源管理中 Task Slot 的概念?

参考答案:

在 Flink 架构角色中我们提到, TaskManager 是实际负责执行计算的 Worker, TaskManager 是一个 JVM 进程, 并会以独立的线程来执行一个 task 或多个 subtask。为了控制一个 TaskManager 能接受多少个 task, Flink 提出了 Task Slot 的概念。简单的说, TaskManager 会将自己节点上管理的资源分为不同的 Slot: 固定大小的资源子集。这样就避免了不同 Job 的 Task 互相竞争内存资源, 但是需要注意的是, Slot 只会做内存的隔离。没有做 CPU 的隔离。

11.说说 Flink1.9 的新特性?

- 1.支持 hive 读写, 支持 UDF
- 2.Flink SQL TopN 和 GroupBy 等优化
- 3.Checkpoint 与 savepoint 针对实际业务场景做了优化
- 4.支持 Flink state 查询

12.消费 kafka 数据的时候, 如何处理脏数据?

可以在处理前加一个 filter 算子, 将不符合规则的数据过滤出去。

13.Flink 的反压和 storm 有哪些不同?

Storm 是通过监控 Bolt 中的接收队列负载情况, 如果超过高水位值就会将反压信息写到 Zookeeper, Zookeeper 上的 watch 会通知该拓扑的所有 Worker 都进入反压状态, 最后 Spout 停止发送 tuple。

Flink 中的反压使用了高效有界的分布式阻塞队列, 下游消费变慢会导致发送端阻塞。

二者最大的区别是 Flink 是逐级反压, 而 Storm 是直接从源头降速。

14.Flink 中水印是什么概念, 起到什么作用?

Watermark 是 Flink 为了处理 EventTime 窗口计算提出的一种机制, 本质上是一种时间戳。一般来讲 Watermark 经常和 Window 一起被用来处理乱序事件。

15. Flink 中的时间有哪几类？

Flink 中的时间和其他流式计算系统的时间一样分为三类：
事件时间，摄入时间，处理时间三种。

如果以 EventTime 为基准来定义时间窗口将形成 EventTimeWindow,要求消息本身就应该携带 EventTime。

如果以 IngestingTime 为基准来定义时间窗口将形成 IngestingTimeWindow,以 source 的 systemTime 为准。

如果以 ProcessingTime 基准来定义时间窗口将形成 ProcessingTimeWindow,以 operator 的 systemTime 为准。

16. 说说 Flink 中的状态存储？

Flink 在做计算的过程中经常需要存储中间状态，来避免数据丢失和状态恢复。选择的状态存储策略不同，会影响状态持久化如何和 checkpoint 交互。

Flink 提供了三种状态存储方式：MemoryStateBackend、FsStateBackend、RocksDBStateBackend。

17. Flink 有没有重启策略？说说有哪几种？

Flink 实现了多种重启策略。

1. 固定延迟重启策略（Fixed Delay Restart Strategy）
2. 故障率重启策略（Failure Rate Restart Strategy）
3. 没有重启策略（No Restart Strategy）

18. Flink 面对数据高峰期时如何处理？

使用 Kafka 消息队列实现数据消峰，然后再使用 Flink 消费数据。

19. Flink 的 checkpoint 机制对比 spark 有什么不同和优势？

spark streaming 的 checkpoint 仅仅是针对 driver 的故障恢复做了数据和元数据的 checkpoint。而 flink 的 checkpoint 机制要复杂了很多，它采用的是

轻量级的分布式快照，实现了每个算子的快照，及流动中的数据的快照。

20.大数据，海量数据去重问题？

订单业务，总量：30 多亿数据，几十 T 的量（阿里子公司解决方案）

1) 实时去重

数据——》ETL 处理——》redis（可结合 bloomfilter 使用）去重（设置过期时间淘汰数据，公司保留 7 天）——》Spark/Flink 实时消费。

2) 离线去重

直接使用 hive 去重，跑 hive shell 脚本即可。

21.为什么使用 Flink 替代 Spark？

主要考虑的是 flink 的低延迟、高吞吐量和对流式数据应用场景更好的支持；另外，flink 可以很好地处理乱序数据，而且可以保证 exactly-once 的状态一致性。

Flink 与 Spark 对比：

1.抽象

在 Spark 中，对于批处理，有 RDD 抽象，而 DStream 用于流式传输，DStream 内部仍然是 RDD。因此，在 Spark 下面代表的所有数据都使用 RDD 抽象来表示。

在 Flink 中，为批处理数据集提供了 DataSet，为流应用程序提供了 DataStream。

2. 内存管理

直到 Spark 1.5，Spark 使用 Java 堆来缓存数据。虽然项目开始时更容易，但它导致了内存不足（OOM）问题和垃圾收集（gc）暂停。因此，从 1.5 开始，Spark 进入定制内存管理，称为 project tungsten。

Flink 从第一天起就开始定制内存管理。实际上，这是 Spark 向这个方向发展的灵感之一。Flink 不仅将数据存储在其自定义二进制布局中，它还可以直接对二进制数据进行操作。

在 JVM 上执行自定义内存管理可以提高性能并提高资源利用率。

3. 实现语言

Spark 在 Scala 中实现。它提供其他语言的 API，如 Java，Python 和 R。

Flink 是用 Java 实现的。它提供其他语言的 API，如 Scala，Python 和 R。

4. API

Spark 和 Flink 都模仿 scala 集合 API。所以从表面来看，两者的 API 看起来非常相似。

5. 流

Apache Spark 将流式处理视为快速批处理。 Apache Flink 将批处理视为流处理的特殊情况。这两种方法都具有令人着迷的含义。

6. SQL 界面

截至目前，最活跃的 Spark 库之一是 spark-sql。 Spark 提供了像 Hive 一样的查询语言和像 DSL 这样的 Dataframe 来查询结构化数据。它是成熟的 API 并且在批处理中广泛使用并且很快将在流媒体世界中使用。

目前，Flink Table API 还不太成熟和完善，并且仍处于测试阶段。

总结

与 Flink 相比，Spark 是一个非常成熟和完整的框架，但 Flink 确实带来了非常有趣的想法，如自定义内存管理，数据集 API 等。 Spark 社区正在认识它并将这些想法融入到 Spark 中。所以从这个意义上来说，Flink 正在将大数据处理完全提升到下一个层次。

15.数仓面试

1.Hive 与 HBase 的区别？

参考答案：

1.使用方面的区别

Hive

- 1)Hive 是一个构建在 Hadoop 平台上的数据仓库。
- 2)Hive 数据存放在 HDFS 之上。
- 3)Hive 数据查询最终被转化为 MapReduce。

HBase

- 1)HBase 是一个基于 HDFS 平台的 NoSQL 数据库。
- 2)HBase 数据存储在 HDFS 之上。
- 3)HBase 是基于数据库本身的实时查询，而非运行 MapReduce。

2.特点的区别

Hive

- 1)Hive 方便熟悉 SQL 的人，快速上手大数据分析。

- 2)Hive 默认计算引擎是 MapReduce，所以查询时间比较长。
- 3)Hive 底层计算引擎也可以更换为 Spark/Tez。
- 4)Hive 中的表是纯逻辑表，只是表的定义，本身不存储和计算，完全依赖 HDFS/MapReduce

HBase

- 1)HBase 本身不支持 SQL，需要通过集成 Phoenix/Hive 才支持 SQL。
- 2)HBase 基于一级索引 Rowkey 进行数据查询，所以查询速度比较快。
- 3)HBase 底层基于 Scan 进行数据扫描，而非基于 MapReduce
- 4)HBase 中的表是物理表，有独立的物理数据结构，查询时可以将数据加载到内存，提升后续的查询效率

3.局限性的区别

Hive

- 1)Hive 目前仅支持 ORCFile 文件格式的数据更新操作，还需要提前开启事务支持。
- 2)Hive 的运行依赖 HDFS 进行数据存储，默认依赖 MapReduce 进行数据计算。

HBase

- 1)HBase 本身不支持 SQL 查询功能，需要通过与 Phoenix 集成实现。
- 2)HBase 的运行依赖 Zookeeper 提供协调服务(配置服务、维护元数据、命名空间服务)，依赖 HDFS 数据存储。

4.应用场景的区别

Hive

- 1) Hive 主要用于构建基于 Hadoop 平台的数据仓库，离线处理海量数据。
- 2) Hive 提供完整的 SQL 实现，一般用于历史数据的分析、挖掘。

HBase

- 1) HBase 适合用了大数据的实时查询、海量数据的存储。
- 2) HBase 是近实时数据库，支持线上业务的实时查询。

2.手写拉链表 SQL，并分析有多少个 job?

参考答案：

1.拉链表定义

维护历史状态，以及最新状态数据的一种表，拉链表根据拉链粒度的不同，实际上相当于快照，只不过做了优化，去除了一部分不变的记录而已,通过拉链表可以很方便的还原出拉链时点的客户记录。

2.用户表

用户表中的数据每日既有可能新增，也有可能修改，但修改频率并不高，属于缓慢变化维度，此处采用拉链表存储用户维度数据。

3.拉链表实现原理

第一步：比如在 2020-01-01 这一天，我们初始化用户数据到数据仓库 user 表，user 表包含 start_time 和 end_time 字段用来标识该条数据的生命周期，明细数据如下所示。

用户编号	用户姓名	用户性别	开始日期	结束日期
101	张三	男	2020-01-01	9999-99-99
102	李四	男	2020-01-01	9999-99-99
103	王五	男	2020-01-01	9999-99-99
104	赵六	女	2020-01-01	9999-99-99
105	小七	女	2020-01-01	9999-99-99

第二步：在 2020-01-02 这一天，104 用户被删除，同时增加了 106 和 107 用户，103 用户的名称改成了小吴，明细数据如下所示。

用户编号	用户姓名	用户性别	开始日期	结束日期
101	张三	男	2020-01-01	9999-99-99
102	李四	男	2020-01-01	9999-99-99
103	王五	男	2020-01-01	2020-01-02
103	小吴	男	2020-01-02	9999-99-99
104	赵六	女	2020-01-01	2020-01-02
105	小七	女	2020-01-01	9999-99-99
106	八爷	男	2020-01-02	9999-99-99
107	九爷	男	2020-01-02	9999-99-99

第三步：在 2020-01-03 这一天，107 用户被删除，同时 106 用户名称改为八王爷，103 用户名称改为吴王，并增加了 108 用户，明细数据如下所示。

用户编号	用户姓名	用户性别	开始日期	结束日期
101	张三	男	2020-01-01	9999-99-99
102	李四	男	2020-01-01	9999-99-99
103	王五	男	2020-01-01	2020-01-02
103	小吴	男	2020-01-02	2020-01-03
103	吴王	男	2020-01-03	9999-99-99
104	赵六	女	2020-01-01	2020-01-02
105	小七	女	2020-01-01	9999-99-99
106	八爷	男	2020-01-02	2020-01-03
106	八王爷	男	2020-01-03	9999-99-99
107	九爷	男	2020-01-02	9999-99-99
108	十爷	男	2020-01-03	9999-99-99

4. 拉链表使用场景

数据量有点大，表中某些字段有变化，但是变化的频率不是很高（数据缓慢变化），业务需求又需要统计这种变化状态。但每天全量一份数据，有点不太现实，不仅浪费了存储空间，有时可能业务统计也有点麻烦。这时，拉链表的作用就体现出来了，既节省空间，又满足了需求。比如用户表，用户量 1 亿，一年 365 天，每天存储一份全量用户数据，效率非常低，此时就可以使用用户拉链表。

5. 拉链表实现步骤

(1) 最初识的用户表为 2020-01-01 日期的全量用户表 (2) 初识的拉链表为 2020-01-01 日期的用户全量表

用户编号	用户姓名	用户性别
101	张三	男
102	李四	男
103	王五	男
104	赵六	女
105	小七	女

用户编号	用户姓名	用户性别	开始日期	结束日期
101	张三	男	2020-01-01	9999-99-99
102	李四	男	2020-01-01	9999-99-99
103	王五	男	2020-01-01	9999-99-99
104	赵六	女	2020-01-01	9999-99-99
105	小七	女	2020-01-01	9999-99-99

(3) 2020-01-02 修改和新增用户数据 (4) 用户变化数据与现有拉链表合并结果

用户编号	用户姓名	用户性别
103	王五	男
103	小吴	男
104	赵六	女
106	八哥	男
107	九哥	男

用户编号	用户姓名	用户性别	开始日期	结束日期
101	张三	男	2020-01-01	9999-99-99
102	李四	男	2020-01-01	9999-99-99
103	王五	男	2020-01-01	2020-01-02
103	小吴	男	2020-01-02	9999-99-99
104	赵六	女	2020-01-01	2020-01-02
105	小七	女	2020-01-01	9999-99-99
106	八哥	男	2020-01-02	9999-99-99
107	九哥	男	2020-01-02	9999-99-99

1)原始业主用户表

```
drop table if exists o_sca_owner_d;
create external table o_sca_owner_d (
  `id` string COMMENT '业主 id',
  `roomId` string COMMENT '业主房间号',
  `ownerName` string COMMENT '业主姓名',
  `ownerSex` string COMMENT '业主性别',
  `ownerAge` string COMMENT '业主年龄',
  `ownerIdcard` string COMMENT '业主身份证号',
  `ownerTel` string COMMENT '业主联系电话',
  `ownerHomeplace` string COMMENT '业主籍贯',
  `ownerWorkplace` string COMMENT '业主工作地',
  `create_time` string COMMENT '创建时间',
  `operator_time` string COMMENT '操作时间'
) COMMENT '业主表'
PARTITIONED BY (`dt` string)
row format delimited fields terminated by '\t'
STORED AS textfile
location '/user/hive/warehouse/sca.db/o_sca_owner_d/';
```

2)初始化拉链表

```
drop table if exists dim_owner_l;
create external table dim_owner_l (
  `id` string COMMENT '业主 id',
  `roomId` string COMMENT '业主房间号',
  `ownerName` string COMMENT '业主姓名',
  `ownerSex` string COMMENT '业主性别',
  `ownerAge` string COMMENT '业主年龄',
  `ownerIdcard` string COMMENT '业主身份证号',
  `ownerTel` string COMMENT '业主联系电话',
```

```
`ownerHomeplace` string COMMENT '业主籍贯',
`ownerWorkplace` string COMMENT '业主工作地',
`create_time` string COMMENT '创建时间',
`operator_time` string COMMENT '操作时间',
`start_time` string COMMENT '有效开始日期',
`end_time` string COMMENT '有效结束日期'
) COMMENT '业主拉链表表'
stored as parquet
location '/user/hive/warehouse/sca.db/dim_owner_l/';
```

3)制作临时拉链表

```
drop table if exists dim_owner__l_tmp;
create external table dim_owner__l_tmp (
`id` string COMMENT '业主 id',
`roomId` string COMMENT '业主房间号',
`ownerName` string COMMENT '业主姓名',
`ownerSex` string COMMENT '业主性别',
`ownerAge` string COMMENT '业主年龄',
`ownerIdcard` string COMMENT '业主身份证号',
`ownerTel` string COMMENT '业主联系电话',
`ownerHomeplace` string COMMENT '业主籍贯',
`ownerWorkplace` string COMMENT '业主工作地',
`create_time` string COMMENT '创建时间',
`operator_time` string COMMENT '操作时间',
`start_time` string COMMENT '有效开始日期',
`end_time` string COMMENT '有效结束日期'
) COMMENT '业主拉链表表'
stored as parquet
location '/user/hive/warehouse/sca.db/dim_owner__l_tmp/';
```

4)拉链表具体 sql 实现

```
insert overwrite table dim_owner__l_tmp
select * from
(select
id,
roomId,
ownerName,
ownerSex,
ownerAge,
ownerIdcard,
ownerTel,
ownerHomeplace,
ownerWorkplace,
```

```

create_time,
operator_time,
'2021-05-21' start_time,
'9999-99-99' end_time
from o_sca_owner_d where dt='2021-05-21'
union all
select
ol.id,
ol.roomId,
ol.ownerName,
ol.ownerSex,
ol.ownerAge,
ol.ownerIdcard,
ol.ownerTel,
ol.ownerHomeplace,
ol.ownerWorkplace,
ol.create_time,
ol.operator_time,
ol.start_time,
if(oso.id is not null and ol.end_time='9999-99-99', date_add(oso.dt,-1),
ol.end_time) end_time
from dim_owner_l ol left join
(
select
*
from o_sca_owner_d
where dt='2021-05-21'
) oso on ol.id=oso.id
)olt
order by olt.id, olt.start_time;

```

5) Hive 中 stage 与 job 的关联

- (1) 通过 explain 可以清晰的看到 stage 划分。
- (2) 经过自动优化之后，有些 stage 并不会执行，这些 stage 就是空的 stage。
- (3) 基于 stage 划分，如何确定 job 数量，主要是看哪些 stage 会执行。只有需要执行的 stage 才会提交到 yarn，然后生成具体的 job application。

6) 生成最终拉链表

临时拉链表导入最终拉链表

```
insert overwrite table dim_owner_l select * from dim_owner_l_tmp;
```

7) 查询最终拉链表数据

```
select id, ownerName,start_time, end_time from dim_owner_l limit 10;
```

3.手写连续登入 7 日的用户 SQL?

参考答案:

1.数据结构

1) 数据格式

第一列(uid): 用户 id

第二列(dt): 登录时间

第三列(status): 用户状态

2) 数据示例

1,2021-05-11,1

1,2021-05-12,1

1,2021-05-13,1

1,2021-05-14,1

1,2021-05-15,1

1,2021-05-16,1

1,2021-05-17,1

1,2021-05-18,1

2.创建表

```
create table ulogin(  
uidint, dt date, status int )  
row format delimited fields terminated by ','  
stored as textfile  
location '/user/hive/warehouse/sca.db/ulogin/';
```

3.加载数据

```
load data local inpath '/home/hadoop/shell/data/userlogin.txt' into table ulogin;
```

4.具体实现

1) 使用 row_number()函数在组内给每行数据编号

使用的 row_number 给日期排序

```
select uid, dt, row_number() over (partition by uid order by dt) gid from ulogin where status=1;
```

2) 日期-rownum = gid, 得到结果可以作为连续登录的依据

通过 date_sub 函数来获取连续登录的日期

```
select uid, dt, date_sub(dt, row_number() over (partition by uid order by dt)) gid from ulogin  
where status=1;
```

3) 根据求得的 gid 作为分组条件, 统计最终结果

通过 group by count having 分组统计, 即可获取连续登入 7 日的用户

```
select uid,count(gid) from (select uid, dt, date_sub(dt, row_number() over (partition by uid order by dt)) gid from ulogin where status=1) as u group by uid,gid having count(gid)>=7;
```

4.Hive 架构原理?

参考答案:

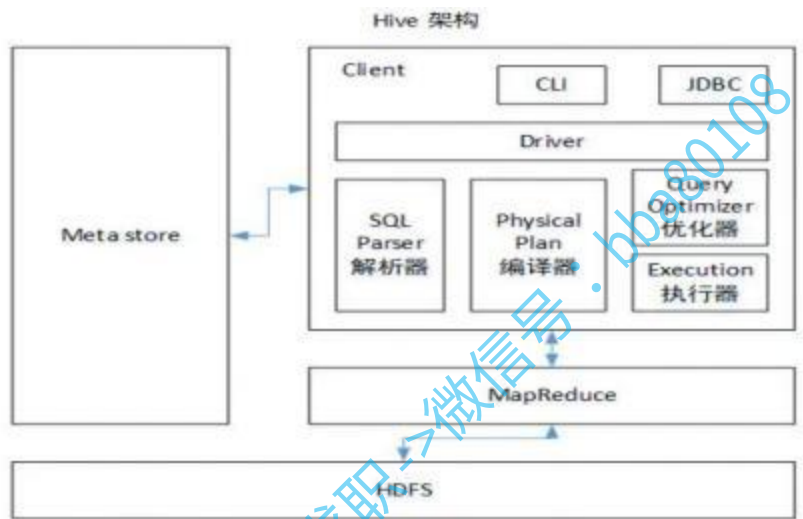
1.Hive 是什么

1) Hive 是由 Facebook 开源用于解决海量结构化日志的数据统计

2) Hive 是基于 Hadoop 的一个数据仓库工具, 可以将结构化的数据文件映射 成一张表

3) Hive 供类了 SQL 查询功能, 底层计算引擎默认为 MapReduce, 存储为 HDFS

2.Hive 架构



3.Hive 优点

1) 操作接口采用类 SQL 语法, 提供快速开发的能力;

2) 避免了编写 MapReduce, 减少开发人员的学习成本;

4) 易扩展(HDFS+MapReduce: 可以扩展集群规模; 支持自定义函数);

4.Hive 使用场景

1) 数据的离线处理: 比如, 日志分析, 海量结构化数据离线分析;

2) Hive 的执行延迟比较高, 因此 hive 常用于数据分析的对实时性要求不高的场合;

3) Hive 优势在于处理大数据, 对于处理小数据没有优势, 因为 Hive 的执行延迟比较高。

5.Hive 的执行流程

1) Hive 通过用户即可 (如 CLI/JDBC) 向 Hive Driver 发送执行命令, 目的是获取查询结果;

2) Driver 拿到执行命令后, 会向 Compiler (编译器) 获取查询计划;

3) Compiler 生成查询计划, 首先要向 MetaStore 获取相关的元数据信息;

4) Driver 获取返回的查询计划并进行优化, 然后将执行计划提交给 Hadoop 运行;

- 5) Hadoop 将执行的结果返回给 Driver;
- 6) Driver 将执行结果返回给客户端。

6.Hive SQL 转换成 MapReduce 的过程

- 1) Parser: 将 HQL 语句转换成抽象语法树 (AST: Abstract Syntax Tree);
- 2) Semantic Analyzer: 将抽象语法树转换成查询块;
- 3) Logic Plan Generator: 将查询块转换成逻辑查询计划;
- 4) Logic Optimizer: 重写逻辑查询计划, 优化逻辑执行计划;
- 5) Physical Plan Generator: 将逻辑计划转化成物理计划 (MapReduce Jobs);
- 6) Physical Optimizer: 选择最佳的 Join 策略, 优化物理执行计划。

5.Hive 内外表的区分方法与差异?

参考答案:

1.Hive 内外表区分方法

- 1) 查看 hive 元数据: 进入 mysql 中 hive 元数据库, 查看 TBLS 表, 查看对应的表名和表类型;
- 2) 在 hive-cli 界面: desc extended tablename, 查看 TableType, 一般在展示结果的最后显示;
- 3) 在 hive-cli 界面: desc formatted tablename, 一个格式化的信息, 查看 TableType;

2.Hive 内外表差异

(1) 内部表

- 1) hive 创建表的时候默认是 MANAGED_TABLE(内部表)
 - 2) hive 创建表的时候如果指定 location, 当进行 load 操作的时候, hdfs 上的原始数据会迁移到 location 指定的目录下;
 - 3) 删除 hive 内部表的时候, 会将 hdfs 目录的数据和元数据信息全部删除;
- ```
create table if not exists temperature
(id string comment '气象站 id', year string comment '年', temperature int comment '气温')
comment '天气表'
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE ;
```

load data inpath '/weather/temperature' overwrite into table temperature ;

#### (2) 外部表

- 1) 删除 hive 表的时候只会将元数据信息删除, 并不会删除 HDFS 上的数据;
  - 2) 生产环境中, 建议使用 hive 外部表;
- ```
create external table if not exists external_temperature
(id string comment '气象站 id', year string comment '年', temperature int comment '气温')
comment '天气表'
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
```

STORED AS TEXTFILE LOCATION '/user/hive/warehouse/myoutdb/external_temperature';

load data inpath '/weather/temperature' overwrite into table external_temperature;

6.Hive 的数据模型及应用场景?

参考答案:

1.Hive 的数据模型

Hive 的数据模型主要有: database、table、partition、bucket 四部分;

2.数据模型之 database

database: 相当于关系型数据库中的命名空间, 作用是将数据库应用隔离到不同的数据库模式中, hive 提供了 create database dbname 语句来建库;

3.数据模型之 table

table: 表是由存储的数据以及描述表的一些元数据组成。数据存储在 HDFS 分布式文件系统中, 元数据存储在 MySQL 关系型数据库中;

hive 常见表分类:

MANGED_TABLE :内部表

EXTERNAL_TABLE: 外部表

4.数据模型之 partition

hive 的分区是根据某列的值进行粗略的划分, 每个分区对应 HDFS 上的一个目录;

5.数据模型之 bucket

要使用 hive 的分桶功能, 首先需要打开 hive 对桶的开关 set hive.enforce.bucketing=true;

bucket 是按照指定值进行 hash, 每个桶就是表目录里的一个文件。

7.Kylin 与 ClickHouse 全面比较?

参考答案:

对比项	ClickHouse	Kylin
核心理念	MPP 在线计算，向量化执行	Cub 预计算 + MPP 现场计算
数据存储原理	存储明细数据，可选物化视图进行优化	存储聚合数据，一个模型中数百上千聚合表 (cuboid)
数据 Layout	分区、分片	分区、分片
索引方式	稀疏主键索引 + 二级索引 (可选)	HBase RowKey 主键索引 / Parquet 稀疏索引
编程语言	C++ 编写，自成一套，专有	Java 编写，Hadoop/Spark 生态
数据存储方式	Shared Nothing；本地磁盘存储	Shared storage；HDFS 分布式存储
引擎支持	多样索引引擎支持多种场景	支持批、流和 RDMS 数据源
数据库完整性	完整的 DBMS，支持 DDL，DML	只支持 SELECT；元数据同步自 Hive 或 RDMS
数据更新	支持更新和删除，后台异步合并	数据按分区刷新来修改，任务引擎异步执行
数据类型	支持复杂数据类型 (map, array, 嵌套等)	支持常见的简单数据类型
适用场景	TB 到 PB 之间的灵活场景分析	百 TB 到数十 PB 的相对灵活场景分析

8.数据倾斜产生原因及处理方式？

参考答案：

1.数据倾斜定义

任务进度长时间维持在 99%，查看任务监控页面，发现只有少量 1 个或几个 reduce 子任务未完成。因为其处理的数据量和其他 reduce 差异过大。单一 reduce 的记录数与平均记录数差异过大，通常可能达到 3 倍甚至更多，最长时远远大于平均时长。

2.数据倾斜原因

map 输出数据按 key Hash 的分配到 reduce 中，由于 key 分布不均匀、业务数据本身的特性、建表时考虑不周、某些 SQL 语句本身就有数据倾斜等原因，造成的 reduce 上的数据量差异过大，所以如何将数据均匀的分配到各个 reduce 中，就是解决数据倾斜的根本所在。

3.key 为空引起数据倾斜

(1) 倾斜原因

join 的 key 值发生倾斜，key 值包含很多空值或是异常值。

(2) 解决方案

对值为空的 key 进行打散，为空 key 赋一个随机的值，使得 key 值为空的数据随机均匀地分布到不同的 reducer 上。

(3) 测试案例

设置多个 reduce 格式

```
set mapreduce.job.reduces = 5;
```

两张大表 join，做全连接

```
select t.id,t.year,t.temperature,s.state from temperature t full join station s on
nvl(t.id,rand())=s.id limit 10;
```

备注：nvl()为空值转换函数，rand()为随机函数

4.group by 引起数据倾斜

(1) 倾斜原因

默认情况下，Map 阶段同一 Key 数据分发给一个 reduce，当一个 key 数据过大时就倾斜了。

(2) 解决方案

然而并不是所有的聚合操作都需要在 Reduce 端完成，很多聚合操作都可以先在 Map 端进行部分聚合，最后在 Reduce 端得出最终结果。



(3) 实现方式 1

1) 是否在 Map 端进行聚合，默认为 True (使用 Combiner 局部合并)

```
set hive.map.aggr = true;
```

2) 设置 map 端预聚合的行数阈值

```
set hive.groupby.mapaggr.checkinterval = 100000;
```

(4) 实现方式 2

1) 有数据倾斜的时候进行负载均衡 (默认是 false)

```
set hive.groupby.skewindata = true;
```

当遇到数据倾斜时，group by 会启动两个 MR job。第一个 job 会将 map 端数据随机输入 reducer，每个 reducer 做部分聚合，相同的 key 就会分布在不同的 reducer 中。第二个 job 再将前面预处理过的数据按 key 聚合并输出结果，这样就起到了均衡的效果。

2) 测试案例

```
set hive.map.aggr = true;
set hive.groupby.mapaggr.checkinterval = 100000;
set hive.groupby.skewindata = true;
select id,count(*) from temperature group by id;
```

5.count distinct 引起数据倾斜

(1) 产生原因

count distinct 聚合时存在大量特殊值，比如存在大量值为 NULL 或空的记录。

(2) 解决方式

做 `count distinct` 时，将值为空的情况单独处理。

1) 如果只是统计去重后的记录数，可以不用处理空值，先把空值过滤掉，然后在最后结果中加 1 即可

2) 如果还包含其他计算，需要进行 `group by` 操作，先将值为空的记录单独处理，然后再跟其他计算结果 `union` 操作。

6.join 操作引起数据倾斜

(1) 大表 join 小表

1) 产生原因

业务数据本身就存在 `key` 分布不均匀的情况，一般情况会产生数据倾斜

2) 解决方式

使用 `map join` 让小的维度表先进内存，在 `map` 端完成 `join`

3) 实现原理

使用 `map join`，直接在 `map` 端就完成表的 `join` 操作，进入 `map` 端的数据都是经过 `split` 得到的，没有根据 `key` 分区这一操作，所以数据都是相对均匀地分布在每个 `map task` 中的，所以就不会产生数据倾斜。

(2) 大表 join 大表

1) 产生原因

业务数据本身的特性，导致两个表都是大表。

2) 解决方式

业务消减

3) 实现原理

业务数据有数据倾斜的风险，但是这些导致数据倾斜风险的 `key` 一般都是无效的，如 `uid` 为空，因为 `uid` 为空的记录是没有意义的。

所以当业务数据很大，但是数据中的大部分（一般都是 80%）可能都是无效数据，那么就可以在 `join` 时过滤掉空值 `uid`，没有了这些无效数据，自然就不存在这么大量集中的 `key`，数据倾斜的风险就会消失。

9.统计注册用户留存天数与留存人数？

参考答案：

1. 日志表

(1) 用户日志临时表

```
CREATE TABLE `user_log_tmp`(  
    `uid` int,  
    `action` string,  
    `day` date)  
row format delimited fields terminated by ','  
STORED AS textfile  
LOCATION '/user/hive/warehouse/mydb/user_log_tmp';
```

uid 用户 id， action 用户的具体行为， day 产生日志的时间（按天分区）；

（2）用户日志表

```
CREATE TABLE `user_log`(  
    `uid` int,  
    `action` string)  
PARTITIONED BY (`day` date)  
row format delimited fields terminated by ','  
STORED AS textfile  
LOCATION '/user/hive/warehouse/mydb/user_log';
```

uid 用户 id， action 用户的具体行为， day 产生日志的时间（按天分区）；

（3）加载数据

```
load data local inpath '/home/hadoop/shell/data/userlog.txt' overwrite into table  
user_log_tmp;
```

```
INSERT OVERWRITE TABLE user_log PARTITION(day) SELECT uid,action,day FROM user_log_tmp ;
```

2.全量用户表

（1）建表

```
CREATE TABLE `all_user`(  
    `uid` int,  
    `createtime` date)  
row format delimited fields terminated by ','  
STORED AS textfile  
LOCATION '/user/hive/warehouse/mydb/all_user';
```

uid 用户 id， createTime 用户第一次有记录的时间；

3.每天去重用户表

（1）建表

```
CREATE TABLE `daily_user`(  
    `uid` int)  
PARTITIONED BY (`day` date)  
row format delimited fields terminated by ','  
STORED AS textfile  
LOCATION '/user/hive/warehouse/mydb/daily_user';
```

uid 用户 id, day 日期（按天分区）;

4.具体实现步骤

（1）统计获得每天的去重用户

```
INSERT OVERWRITE table daily_user partition(day = '2021-05-09') SELECT DISTINCT uid FROM  
user_log WHERE day = '2021-05-09';
```

（2）更新全量用户

```
INSERT INTO table all_user (uid, createtime) SELECT u.uid, '2021-05-09' AS createtime FROM  
(SELECT DISTINCT uid FROM user_log WHERE day = '2021-05-09') AS u WHERE u.uid NOT IN  
(SELECT uid FROM all_user);
```

（3）计算用户注册时间与活动时间差值

```
SELECT au.uid, au.createtime, du.day, datediff(du.day, au.createtime )  
FROM all_user au INNER JOIN daily_user du  
ON au.uid = du.uid WHERE au.createtime = '2021-05-09';
```

（4）统计留存天数与人数

```
SELECT datediff(du.day, au.createtime) AS day , COUNT(du.uid) AS total  
FROM all_user au INNER JOIN daily_user du  
ON au.uid = du.uid  
WHERE au.createtime = '2021-05-09'  
GROUP BY datediff(du.day, au.createtime);
```

10.数据库的三范式?

参考答案:

第一范式:

域都应该是原子性的, 即数据库表的每一列都是不可分割的原子数据项。

商品ID	商品名称	商家名称	用户ID
10001	100个电风扇	xxx专卖店	101



商品ID	商品名称	商品数量	商家名称	用户ID
10001	电风扇	100	xxx专卖店	101

第二范式：

在 1NF 的基础上，实体的属性完全依赖于主关键字，不能存在仅依赖主关键字一部分的属性。

学生ID	所属系	系主任	所修课程	分数
001	体育系	姚明	000001	96
001	体育系	姚明	000002	98



学生ID	所修课程	分数
001	000001	96
001	000002	98

学生ID	所属系	系主任
001	体育系	姚明

第三范式：

在 2NF 的基础上，任何非主属性不依赖于其它非主属性。只要非主键内部存在传递依赖，就不满足第三范式。

学号	姓名	系名	系主任
101	巩俐	艺术系	张艺谋
102	黄晓明	艺术系	张艺谋
103	孙俪	美术系	高希希



学号	姓名	系名
101	巩俐	艺术系
102	黄晓明	艺术系
103	孙俪	美术系

系名	系主任
艺术系	张艺谋
艺术系	张艺谋
美术系	高希希

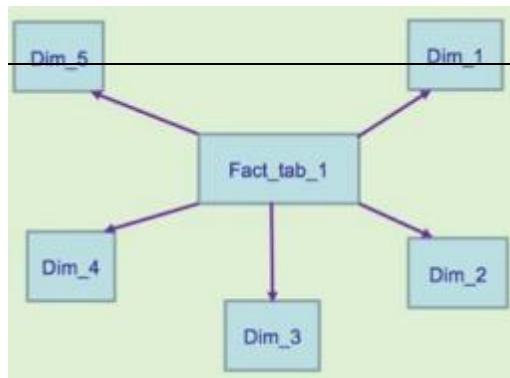
11. 维度建模常用模型？

参考答案：

星型模型

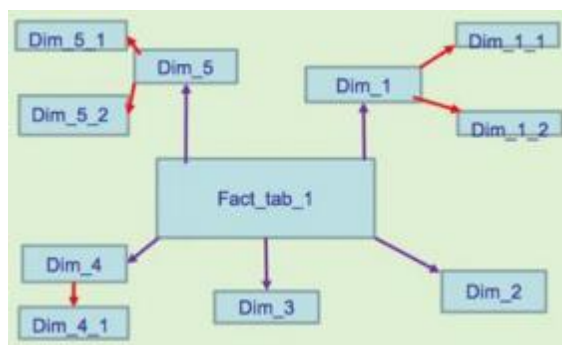
当所有维表都直接连接到 "事实表" 上时，整个图解就像星星一样，故将该模型称为星型模

型.



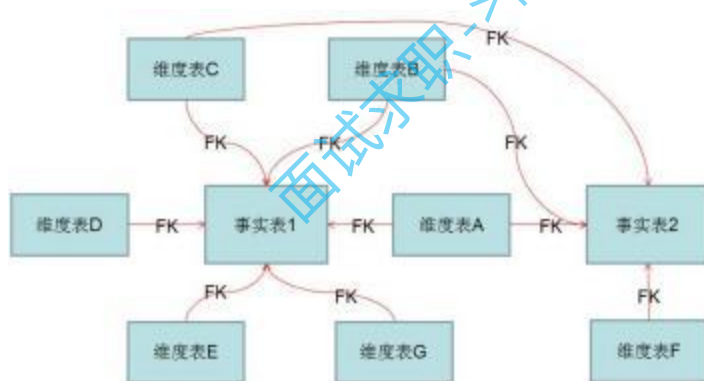
雪花模型

当有一个或多个维表没有直接连接到事实表上，而是通过其他维表连接到事实表上时，其图解就像多个雪花连接在一起，故称雪花模型。



星座模型

星座模式是由星型模式延伸而来，星型模式是基于一张事实表，而星座模式是基于多张事实表，而且共享维度表信息。



12.维度建模一般分为哪几步？

参考答案：

1.选择业务过程

通过对业务需求以及可用数据源的综合考虑，决定对哪种业务过程开展建模工作。

在业务系统中，先挑选影响大、可行性高、最感兴趣的一个或几个业务，如门禁业务和车辆进出业务，一条业务线对应一个事实表。

例如，可以选择这些业务，如门禁、车辆进出等。

2.声明粒度

数据粒度：指数据仓库中保存数据的细化程度。

声明粒度：意味着精确定义事实表中的一行数据表示什么，一般尽可能选择最小粒度，从而应对不同粒度的需求。

例如：

如在门禁表中，业主的每次开门作为门禁事实表中的一行，粒度为每次开门；每周的开门次数作为一行，粒度就是每周开门。每月的开门次数作为一行，粒度就是每月开门。

3.确定维度

事实表粒度选择完毕后，维度的选择就比较直接了，在主维度框架内，可以考虑其他维度是否可以被属性化为开门度量。如业主、日期、小区等。

维度的作用是描述业务事实，如“谁”，“何时”，“何地”，“做了什么事”。

4.确定事实

确认应该将那些事实放到事实表中，在考虑可能存在的事实时，可能会发现仍然需要调整早期的粒度声明或者维度选择。事实指的就是业务中的度量值，如订单金额，下单次数等。

事实表=维度表+度量值

	业主 (谁)	日期 (何时)	小区 (何地)	车辆 (开车)	度量值
门禁	✓	✓	✓		刷卡次数
车辆进出	✓	✓	✓	✓	进出次数

13.数据集市 vs 数据仓库？

参考答案：

数据集市：可以理解成一种微型的数据仓库。它通常包含更少的数据，更少的主题以及更少的历史数据，属于部门级的，一般只能为某个局部范围内的管理人员服务。

数据仓库：是企业级的，能为企业各个部门的运行提供决策支持手段。

14.Oracle 业务数据迁移数仓？

参考答案：

批量抽取数据库表，一般两种方式：

关系型数据库，可以通过 sqoop、datax 同步到 hdfs 中，目前几乎支持了所有的主流数据库。

用 spark、MR 等方式开发数据同步程序，目前主要采用该方案，实际采集的时候可以根据情况考虑全量或者增量；

15.事实表的概念？

参考答案：

每个数据仓库都包含一个或者多个事实数据表。

事实数据表可以包含业务销售数据，如现金登记事务所产生的数据，事实数据表通常包含大量的行。

一般事实表中只存放数字或者一些 Flag 用来统计(Count)，如收益、数量、支出等。

16.Hive 分区表 vs 分桶表？

参考答案：

1.建表方式不同

创建分区表时，分区字段不能直接写在表结构中，通过指定分区字段和类型的方式成为表结构；

创建分桶表时，分桶表的字段必须写在表结构中，再指定分桶字段。

2.数据存储方式不同：

分区表是根据不同分区以文件夹的形式存储在 HDFS 上；

分桶表是以多个数据文件的形式存储在表数据目录下。

3.使用场景不同

分区表适合用于多条数据有相同字段值的情况，比如按天分区。

分桶表适合用于数据字段没有重复值的情况，或者分区太多更不利于查询的情况，比如用户表 uid 字段。

17.数仓架构分层及每层作用？

参考答案：

1.数仓为什么要分层

（1）把复杂问题简单化，将复杂的任务分解成多层来完成，每一层只处理简单的任务，方便定位问题。

（2）减少重复开发，规范数据分层，通过中间层数据，能够减少很多重复计算工作，提高计算结果的复用性。

（3）隔离原始数据，使真实数据与统计数据隔离开，ODS 之后层级的数据处理，无论出现数据的异常还是数据的敏感性，都可以重新从 ODS 再计算。

2.数仓架构分层

2.1ODS 层

ODS(Operational Data Store) 操作性数据，即原始数据层，是数据仓库第一层数据，是最接近数据源中数据的一层，作为数据库到数据仓库的一种过渡。数据源中的数据，经过 ETL 之后装入 ODS 层，ODS 的数据结构一般与数据来源保持一致，便于减少 ETL 的工作复杂性。

2.2DWD 层

DWD(data warehouse detail) 是明细数据层，该层的表结构和粒度与原始表保持一致，不过需要对 ODS 层数据进行清洗、维度退化、脱敏等，最终得到的数据是干净的，完整的，一致的数据。

2.3DWS 层

DWS (data warehouse service)：服务数据层，基于 DWD 层上的明细数据，按天轻度汇总成某一个主题域的服务数据，一般是宽表。

DWS 层统计各个主题对象的当天行为，以及一些业务明细数据，服务于 DM 层的某一个主题域宽表。例如每日业主行为，每日车辆行为。

2.4ADS 层

ADS (Application Data Store) 层：数据应用层，也即是出报表结果。以 DW 为基础，为各种统计报表提供汇总数据，然后将分析汇总数据同步到 RDS 数据库。

RDS 是 OLTP 数据库。RDS 用于支撑日常的业务活动，市场上典型产品是 Mysql、Oracle。

2.5DM 层

数据集市层 DM(data market)，基于 DW 上的基础数据，整合汇总成分析某一个主题域的服务数据，一般是宽表，用于提供后续的业务查询，OLAP 分析，数据分发等。

数据集市，以某个业务应用为出发点而建设的局部 DW，DM 只关心自己需要的数据，不会全盘考虑企业整体的数据架构和应用，每个应用有自己的 DM。

2.6DM VS ADS

狭义上，数据集市，可以理解为数据仓库中为用户提供数据支撑的应用层，比如 ADS 层。

广义上，数据集市，所有以主题划分的数据仓库中可供查阅的都可以成为数据集市，包括 DWD,DWS,ADS 层，甚至包括从 hadoop 中同步到 RDS 的数据都可以成为数据集市。

18.数仓仓库 VS 数据库？

参考答案：

1.数据库定义

数据库是按照数据结构来组织、存储和管理数据的建立在计算机存储设备上的仓库。
数据库是长期存储在计算机内、有组织的、可共享的数据集合。

数据库中的数据指的是以一定的数据模型组织、描述和存储在一起，具有尽可能小的冗余度、较高的数据独立性和易扩展性的特点并可在一定范围内为多个用户共享。

2.数据仓库定义

数据仓库是面向主题的，集成的，相对稳定的反映历史变化的数据集合，用于支持管理决策。

面向主题的：在较高层次上将企业信息系统的数据综合归并进行分析利用的抽象的概念。每个主题基本上对应一个相应的分析领域。

集成的：企业级数据，同时数据要保持一致性、完整性、有效性、精确性。

稳定的：从某个时间段来看是保持不变的，没有更新操作、删除操作，以查询分析为主。

变化的：反应历史变化。

3.数据仓库 VS 数据库

面向业务的数据库常称作 OLTP，面向分析的数据仓库亦成为 OLAP。

比较	数据仓库	数据库
数据范围	存储历史的、完整的、反应历史变化的	当前状态数据
数据变化	可添加、无删除、无变更的、反应历史变化	支持频繁的增、删、改、查操作
应用场景	面向分析、支持战略决策	面向业务交易流程
设计理论	违范式、适当冗余	遵照范式(第一、二、三等范式)、避免冗余
处理量	非频繁、大批量、高吞吐、有延迟	频繁、小批次、高并发、低延迟

19.如何构建数据仓库？

参考答案：

1.系统分析，确定主题

通过与业务部门的交流，了解建立数仓要解决的问题，确认各个主题下的查询分析要求

2.选择满足数据仓库系统要求的软件平台。

选择合适的软件平台，包括数据库、建模工具、分析工具等

3.建立数据仓库的逻辑模型。

确定建立数据仓库逻辑模型的基本方法，基于主题视图，把主题视图中的数据定义转到逻辑数据模型中

4. 逻辑数据模型转换为数据仓库数据模型

5.数据仓库数据模型优化

随着需求和数据量的变化进行调整

6.数据清洗转换和传输

业务系统中的数据加载到数据仓库之前，必须进行数据的清洗和转换，保证数据仓库中数据的一致性。

7.开发数据仓库的分析应用

满足业务部门对数据进行分析的需求。

8.数据仓库的管理

包括数据库管理和元数据管理。

20.数据中台 VS 数据仓库 VS 大数据平台？

参考答案：

1.什么是数据中台

数据中台是指通过数据技术，对海量数据进行采集、计算、存储、加工，同时统一标准和口径。数据中台把数据统一之后，会形成标准数据，再进行存储，形成大数据资产层，进而为客户提供高效服务。

这些服务和企业的业务有较强的关联性，是企业所独有且能复用的，它是企业业务和数据的积淀，其不仅能降低重复建设，减少烟囱式协作的成本，也是差异化竞争的优势所在。

数据中台通过整合公司开发工具、打通全域数据、让数据持续为业务赋能，实现数据平台化、

数据服务化和数据价值化。数据中台更加侧重于“复用”与“业务”。

2.数据中台 VS 数据仓库 VS 大数据平台

2.1 基础能力上的区别

数据平台：提供的是计算和存储能力。

数据仓库：利用数据平台提供的计算和存储能力，在一套方法论指导下建设的一整套的数据表。

数据中台：包含了数据平台和数据仓库的所有内容，将其打包，并且以更加整合以及更加产品化的方式对外提供服务和价值。

2.2 业务能力上的区别

数据平台：为业务提供数据主要方式是提供数据集。

数据仓库：相对具体的功能概念是存储和管理一个或多个主题数据的集合，为业务提供服务的方式主要是分析报表

数据中台：企业级的逻辑概念，提供企业数据产生价值的能力，为业务提供服务的主要方式是数据 API

21.数仓最重要的作用？

参考答案：

数据集成

（1）个人认为是数据集成。

（2）企业的数据通常是存储在多个异构数据库中，要进行分析必须先要对数据进行一致性整合。

（3）集成整合后才可以对数据进行分析，挖掘数据潜在的价值。

22.缓慢变化维度的常用处理方式？

参考答案：

1.维度分类

1.1 无变化维度

比如一个人的身份证号、姓名、性别等信息属于不变的信息

1.2 缓慢变化维度（slowly changing dimensions）

比如一个人的政治面貌、婚姻状态等信息属于缓慢变化的信息

1.3 剧烈变化维度

比如一个人的工作经历、工作单位等信息属于剧烈变化的信息

2.SCD 常用的处理方式

2.1 完全不记录历史变化信息

在 ETL 将数据载入 SCD 的时候，对于会产生变化的属性值直接覆盖。

	uid	name	age	sex	position
更新前	101	张三	28	女	销售
更新后	101	张三	28	女	开发

2.2 通过添加记录将每一次变化都记录到 SCD 中

在 ETL 将数据载入 SCD 的时候，对于会产生变化的属性值保留，同时新增一条记录。

	uid	name	age	sex	position	starttime	endtime
更新前	101	张三	28	女	销售	2021-05-30	9999-99-99
更新后	101	张三	28	女	销售	2021-05-30	2021-05-31
	101	张三	28	女	开发	2021-05-31	9999-99-99

2.3 添加相应的历史字段，记录最近的变化而非全部变化

如果字段发生了变化，通过添加历史字段记录最近的变化，更久之前的变化就无法追溯了。

	uid	name	age	sex	position	old_position
更新前	101	张三	28	女	销售	null
更新后	101	张三	28	女	开发	销售

23.元数据的理解？

参考答案：

狭义来讲：用来描述数据的数据。

广义来讲：除了业务逻辑直接读写处理的业务数据，所有其他用来维护整个系统运转所需要的数据，都可以较为元数据。

定义：元数据 **metadata** 是关于数据的数据。在数仓系统中，元数据可以帮助数据仓库管理员和数据仓库开发人员方便的找到他们所关心的数据；元数据是描述数据仓库内部数据的结构和建立方法的数据。

按照用途可分为：技术元数据、业务元数据。

1.技术元数据

存储关于数据仓库技术细节的数据，用于开发和管理数据仓库使用的数据。

数据仓库结构的描述，包括数据模式、视图、维、层次结构和导出数据的定义，以及数据集市的位置和内容。

业务系统、数据仓库和数据集市的体系结构和模式。

由操作环境到数据仓库环境的映射，包括元数据和他们的内容、数据提取、转换规则和数据刷新规则、权限等。

2.业务元数据

从业务角度描述了数据仓库中的数据，他提供了介于使用者和实际系统之间的语义层，使不懂计算机技术的业务人员也能读懂数仓中的数据。

企业概念模型：表示企业数据模型的高层信息。整个企业业务概念和相互关系。以这个企业模型为基础，不懂 **sql** 的人也能做到心中有数。

多维数据模型。告诉业务分析人员在数据集市中有哪维、维的类别、数据立方体以及数据集市中的聚合规则。

业务概念模型和物理数据之间的依赖。业务视图和实际数仓的表、字段、维的对应关系也应该在元数据知识库中有所体现。

24.如何确定数仓主题域？

参考答案：

1.主题

主题是在较高层次上将数据进行综合、归类和分析利用的一个抽象概念，每一个主题基本对应一个宏观的分析领域。在逻辑意义上，它是对企业中某一宏观分析领域所涉及的分析对象。

面向主题的数据组织方式，就是在较高层次上对分析对象数据的一个完整并且一致的描述，能刻画各个分析对象所涉及的企业各项数据，以及数据之间的联系。

主题是根据分析的要求来确定的。

2.主题域

从数据角度看（集合论）

主题域通常是联系较为紧密的数据主题的集合。可以根据业务的关注点，将这些数据主题划分到不同的主题域。主题域的确定由最终用户和数仓设计人员共同完成。

从需要建设的数仓主题看（边界论）

主题域是对某个主题进行分析后确定的主题的边界。

数仓建设过程中，需要对主题进行分析，确定主题所涉及到的表、字段、维度等界限。

25.如何控制数据质量？

参考答案：

数据质量控制

校验机制：每天进行数据量的比对 `select count(*) from xxx`，早发现，早修复。

数据内容的比对：对数据抽样比对。

复盘：每月做一次全量。

26.如何做数据治理？

参考答案：

数据治理不仅需要完善的保障机制，还需要理解具体的治理内容，比如数据应该怎么进行规范，元数据该怎么来管理，每个过程需要那些系统或者工具来配合？

数据治理领域包括但不限于以下内容：数据标准、元数据、数据模型、数据分布、数据存储、数据交换、数据生命周期管理、数据质量、数据安全以及数据共享服务。

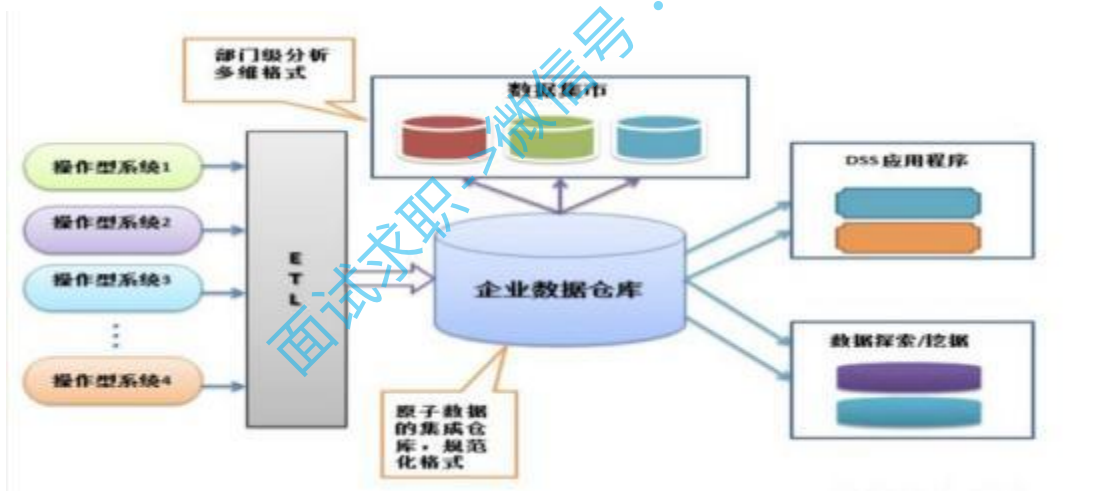


27.数仓模型设计思路？

参考答案：

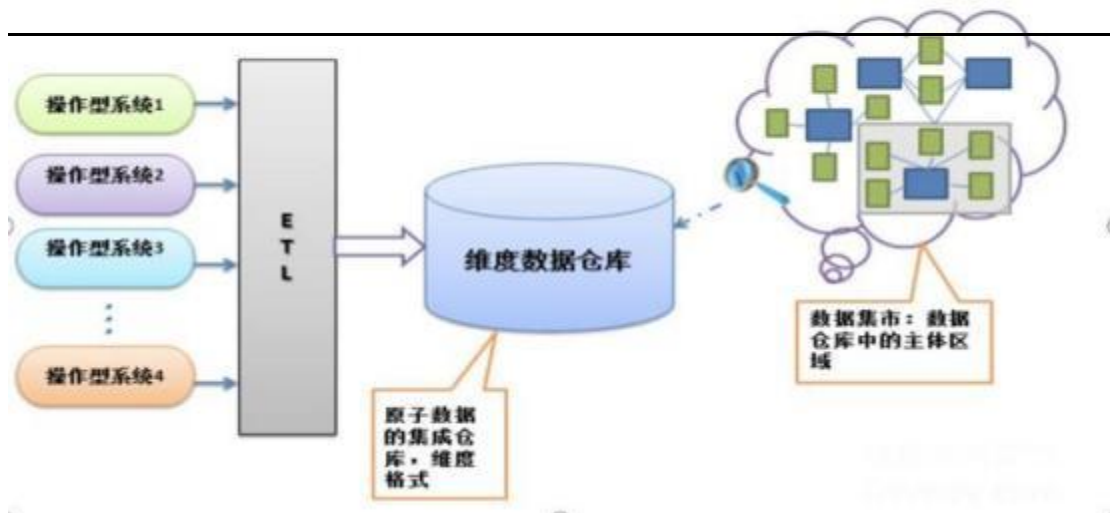
自上而下（偏数据驱动）

Bill Inmon 推崇自上而下的方式，一个企业建立唯一的数据中心，数据是经过整合、清洗、去掉脏数据、标准的、能够提供统一的视图。要从整个企业的环境入手，建立数据仓库，要做很全面的设计。



自下而上（偏业务驱动）

Ralph Kimball 推崇自下而上的方式，认为数据仓库应该按照实际的应用需求加载需要的数据，不需要的数据不要加载到数据仓库中。这种方式建设周期短，用户能很快看到结果。



28.为什么需要数仓建模？

参考答案：

数仓建模需要按照一定的数据模型，对整个企业的数据进行采集，整理，提供跨部门、完全一致的报表数据。

合适的数据模型，对于大数据处理来讲，可以获得得更好的性能、成本、效率和质量。良好的模型可以帮助我们快速查询数据，减少不必要的冗余，提高用户的使用效率。

数据仓库为企业决策，提供数据支持。基于数据仓库对数据进行分析，能够助力企业改进业务流程、控制成本、提高产品质量。

29.OLAP 和 OLTP 模型的选择？

参考答案：

OLTP 系统是操作事物型系统，主要数据操作是随机读写，主要采用满足 3NF 的实体关系模型存储数据，在事物处理中解决数据的冗余和一致性问题。

OLAP 系统是分析型系统，主要数据操作是批量读写，不需要关注事务处理的一致性，主要关注数据的整合，以及复杂大数据量的查询和处理的性能。

30.事实表的分类？

参考答案：

1.事务事实表

事务事实表记录的是事务层面的事实，保存的是最原子的数据，也称“原子事实表”。事务事实表中的数据在事务事件发生后产生，数据的粒度通常是每个事务记录一条记录。

2.周期快照事实表

以具有规律性的、可预见的时间间隔来记录事实。它统计的是间隔周期内的度量统计，每个时间段一条记录，是在事务事实表之上建立的聚集表。比如时间周期是 1 周，那么这个周期快照事实表记录着一周对于某个度量值的统计值。

3.累计快照事实表

累积快照表记录的不确定的周期的数据。代表的是完全覆盖一个事务或产品的生命周期的时间跨度，通常具有多个日期字段，用来记录整个生命周期中的关键时间点。例如订单累计快照事实表会有付款日期，发货日期，收货日期等时间点。

事务事实表中一条交易记录会每天有一条数据来记录整个交易过程；而累积快照事实表只会有一条记录，数据会一直更新直到过程结束。

31.数仓架构演进

参考答案：

- 1.经典数仓架构：使用传统工具（如 MySQL、Oracle）来建设数仓。
- 2.离线大数据架构：开始使用大数据工具（如 Hive）来替代经典数仓中的传统工具。
- 3.Lambda 架构：在离线大数据架构的基础上，使用流处理技术（如 Spark、Flink）直接完成实时性较高的指标计算。
- 4.Kappa 架构：实时处理变成了主要的部分，出现了以实时处理为核心的 kappa 架构。

32.开源 OLAP 解决方案？

参考答案：

Persto、SparkSQL、Impala 等 MPP 架构和 ROLAP 的引擎

Druid 和 Kylin 等预处理架构

ElasticSearch 搜索引擎架构

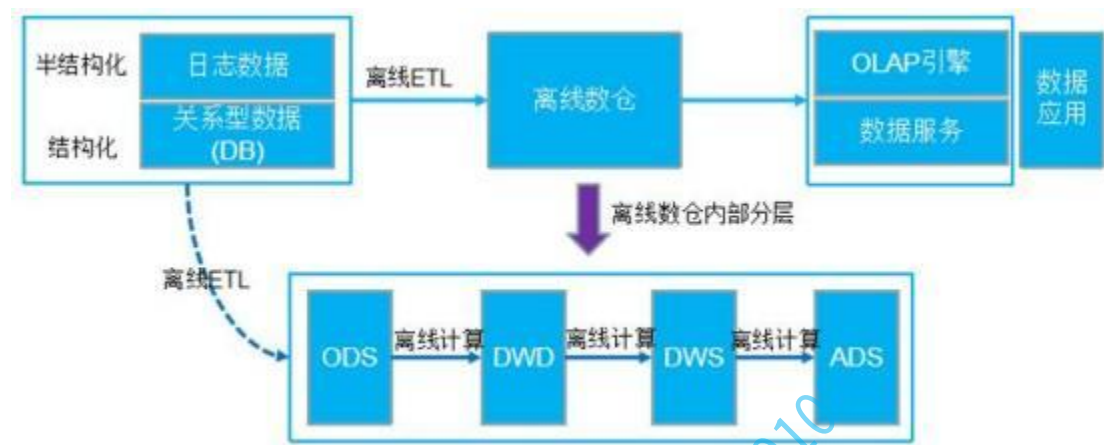
ClickHouse 列式数据库

33.离线数仓架构？

参考答案：

离线大数据架构的特点：

- (1) 数据源通过离线的方式导入到离线数仓中
- (2) 数据处理采用 MapReduce、Hive、SparkSQL 等离线计算引擎

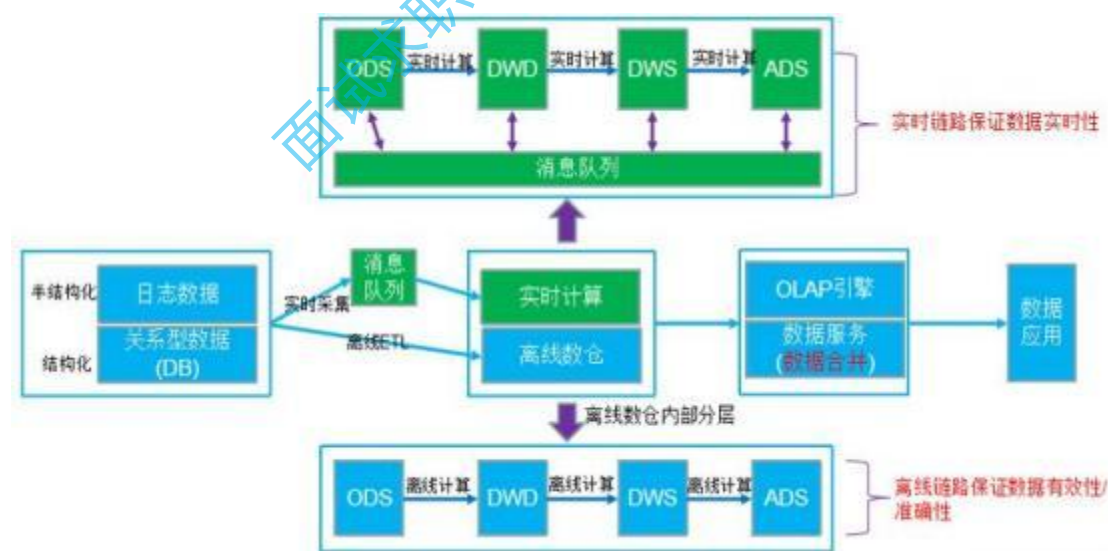


34.Lambda 数仓架构？

参考答案：

在离线数仓的基础上增加了实时计算的链路，并对数据源进行流式改造，实时计算去订阅消息队列，并推送到下游的数据服务中去。

Lambda 架构问题：同样的需求需要开发两套一样的代码；资源占用增多。

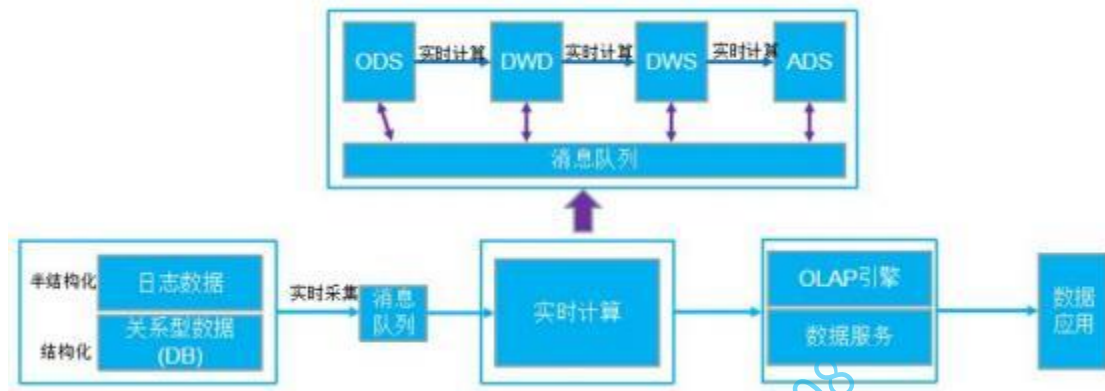


35.Kappa 数仓架构？

参考答案：

kappa 架构可以认为是 lambda 架构的简化版，移除了 lambda 架构中的批处理部分。在 kappa 架构中，需求修改或者历史数据重新处理都通过上游重放完成。

kappa 架构最大的问题是流式重新处理历史数据的吞吐能力会低于批处理，但可以通过增加计算资源来弥补。



36.数仓建模方法？

参考答案：

数仓建模模型

- (1) ER 模型
- (2) 维度模型
- (3) Data Vault 模型
- (4) Anchor 模型

数仓建模常用的是维度模型和 ER 模型。

16.运维面试

1.大数据集群节点规划

1.HDFS 集群节点规划

假如业务系统数据量每天增量 50T，保留周期为 30 天，那么 HDFS 存储容量为 $50T * 30 \text{ 天} * 3 \text{ 副本} * 2 \text{ 倍（数据源+清晰加工）} = 9000T = 8.79P$ 。

假如每个机器的磁盘是 $4T * 10 = 40T$ ，每台机器的可用存储容量为 $40T * 0.75 = 30T$ ，节点预估数量 = $9000T / 30 = 300$ 节点，所以 datanode 的节点最小数量为 300 个。

2.YARN 集群节点规划

根据任务量和性能评估 YARN 的节点数是很难的，难以评估，所以 NodeManager 节点数可以和 datanode 节点数保持一致，如果算力负载过高，根据实际情况再扩容即可。

3.HBase 集群节点规划

HBase 节点规划：一般开始搭建是根据 HDFS 存储公式计算即可，如果从增加并发的考虑，一般一个 RegionServer 并发为 5000 ~ 2 万（优化后并发更高），可以根据业务实际并发估计节点数量。

4.Kafka 集群节点规划

Kafka 节点规划：一般开始搭建是根据类似 HDFS 存储公式计算，一般 1 个 broker 并发为 5 万（优化后并发更高），可以根据业务实际并发估计节点数量。

5.Zookeeper 集群节点规划

Zookeeper 节点规划：集群开始搭建时 3 节点就够用了，如果发现 zookeeper 负载过高或有超时现象时可以考虑扩展到 5 节点。

6.大数据集群厂商选择

集群中的每个组件要做高可用，一般国企会用 CDH，互联网公司会用开源社区版演化自己平台。

2.大数据集群硬件规划

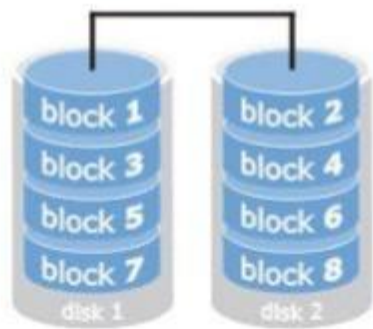
1.磁盘阵列概念

RAID：磁盘阵列（Redundant Arrays of Independent Disks，RAID），有“数块独立磁盘构成具有冗余能力的阵列”之意。

RAID0 是把多个(最少 2 个)硬盘合并成 1 个逻辑盘使用，数据读写时对各硬盘同时操作，不同硬盘写入不同数据，速度快。

RAID 0

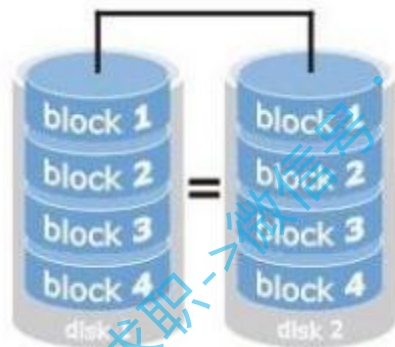
striping



RAID1 是通过磁盘数据镜像实现数据冗余，在成对的独立磁盘上产生互为备份的数据。当原始数据繁忙时，可直接从镜像拷贝中读取数据，因此 RAID 1 可以提高读取性能。RAID 1 是磁盘阵列中单位成本最高的，但提供了很高的数据安全性和可用性。当一个磁盘失效时，系统可以自动切换到镜像磁盘上读写，而不需要重组失效的数据。

RAID 1

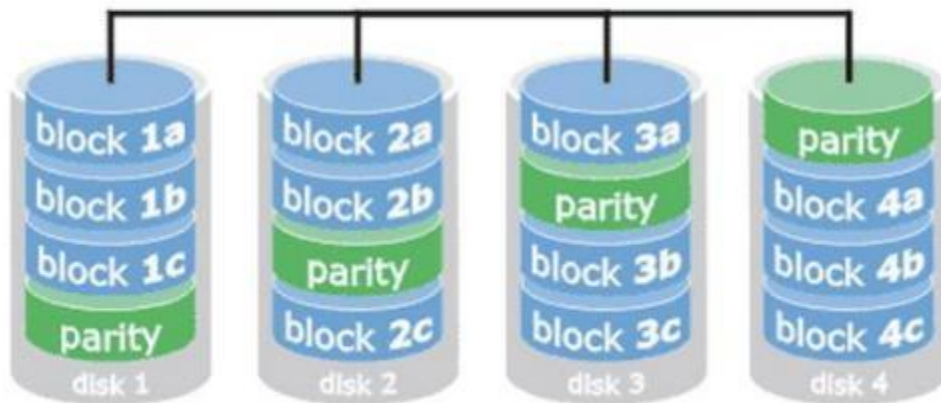
mirroring



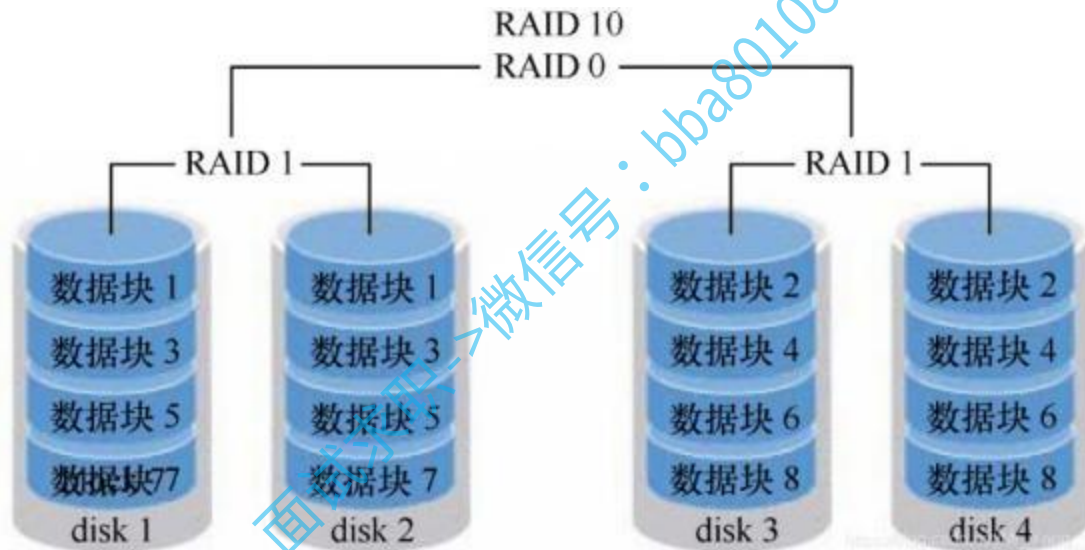
Raid5 是把硬盘设备的**数据奇偶校验信息**保存到其他硬盘设备中，“parity”部分存放的就是数据的奇偶校验信息，换句话说，Raid5 技术实际上没有备份磁盘中的真实数据，而是当硬盘设备出现问题后，通过**奇偶校验技术**来尝试重建损坏的数据。Raid5 这样的技术特性“妥协”的兼顾了硬盘设备的读写速度、数据安全性与存储成本问题。

RAID 5

parity across disks



Raid10 是 Raid1 和 Raid0 的组合体，Raid10 技术至少需要 4 块硬盘来组建，其中先分别两两制成 Raid1 磁盘阵列，以保证数据的安全性。然后再对两个 Raid1 磁盘阵列实施 Raid0 技术，进一步提高硬盘设备的读写速度。



2.HDFS 集群硬件规划

内存：NameNode 内存一般 100 万个 block 对应 1G 的堆内存，比如我们最大的一个集群的 block 达到了 9000 万，会占用内存 90G，NameNode 的内存不止存放 block，我们产线环境配置的是 200G+。

磁盘：主节点 NameNode 主要 CPU/内存配置高些，系统盘做 RAID1，hdfs 要安装在系统盘上，如果有其他的数据盘，可以做 RAID5，容量所需不大，500G~ 1T 即可。

详细信息

主机 ID	7ffffebb3-58df-40b6-96c1-ce739996522b		
IP		机架	/default
内核	24 (48 采用超线程技术)	上次更新	5.00秒 之前
CDH 版本	CDH 5	物理内存	186.2 吉字节/251.6 吉字节
分配	redhat 7.2	交换空间	0 字节/0 字节

从节点 datanode 内存/CPU/磁盘都有要求，我们产线存储每服务器 4T* 10=40T 台

状态 配置 进程 资源 组件 命令 图表库 快速链接

详细信息

主机 ID	b8463d2c-a69d-45f6-9c76-9bbd4314dd54		
IP		机架	/default
内核	12 (24 采用超线程技术)	上次更新	1.00秒 之前
CDH 版本	CDH 5	物理内存	58.1 吉字节/125.4 吉字节
分配	redhat 7.2	交换空间	0 字节/0 字节

备注：DataNode 对磁盘要求比较高。

3.YARN 集群硬件配置

主节点 ResourceManager 主要 CPU/内存配置高些，系统盘做 RAID1，yarn 要安装在系统盘上，如果有其他的数据盘，可以做 RAID5，容量所需不大，500G~1T 即可。

状态 配置 进程 资源 组件 命令 图表库 快速链接 ▾

详细信息

主机 ID	06bd819f-2ce7-4fec-933f-58c68b84ef2d		
IP		机架	/default
内核	24 (48 采用超线程技术)	上次更新	0毫秒 之前
CDH 版本	CDH 5	物理内存	44.0 吉字节/251.6 吉字节
分配	redhat 7.2	交换空间	0 字节/0 字节

备注：因为 ResourceManager 要做调度，所以 CPU 可以分配多一点，内存不用太大。

从节点 NodeManager 对 CPU 和内存都有要求。

状态 配置 进程 资源 组件 命令 图表库 快速链接

详细信息

主机 ID	b8463d2c-a69d-45f6-9c76-9bbd4314dd54		
IP		机架	/default
内核	12 (24 采用超线程技术)	上次更新	1.00秒 之前
CDH 版本	CDH 5	物理内存	58.1 吉字节/125.4 吉字节
分配	redhat 7.2	交换空间	0 字节/0 字节

备注：NodeManager 一般与 DataNode 共用节点，对内存要求不是很高，主要是计算。

4.HBase 集群硬件配置

主节点 Master CPU 内存中配就行。

详细信息

主机 ID	f9642202-1109-4fbe-85b4-5d5a7e831c24		
IP		机架	/lf419/rack1
内核	12 (24 采用超线程技术)	上次更新	4.00秒 之前
CDH 版本	CDH 5	物理内存	<u>112.3 吉字节/125.4 吉字节</u>
分配	redhat 7.2	交换空间	<u>0 字节/0 字节</u>

从节点 RegionServer 内存可以大些。

详细信息

主机 ID	3b521ed8-3758-4993-8a21-844a17bb6fe1		
IP		机架	/lf419/rack2
内核	12 (24 采用超线程技术)	上次更新	6.00秒 之前
CDH 版本	CDH 5	物理内存	<u>109.9 吉字节/125.4 吉字节</u>
分配	redhat 7.2	交换空间	<u>0 字节/0 字节</u>

备注：RegionServer 对内存要求较高，blockcache 和 Memstore 都需要占用内存。

5.Kafka 集群硬件配置

状态	配置	进程	资源	组件	命令	图表库	快速链接
详细信息							
主机 ID	ba7db532-bd77-4236-b947-84db1558e8f0						
IP		机架	/default				
内核	12 (24 采用超线程技术)		上次更新	2.00秒 之前			
CDH 版本	CDH 5		物理内存	17.1 吉字节/125.4 吉字节			
分配	redhat 7.2		交换空间	0 字节/0 字节			

3.大数据集群目录规划

1.管理节点的系统盘与数据盘

OS 磁盘建议 RAID1，建议 200G 以上，并且做 LVM（逻辑卷），这样可以动态调整 OS 空间大小，安装包需要安装在 OS 盘，namenode 的 fsimage/editlog、journalnode 的 /editlog、zookeeper 的数据日志都放在 OS 盘（RAID1）。

管理节点的数据盘做 RAID5，管理节点的数据盘如下所示：

磁盘	装入点	使用情况
cm_processes	/opt/cm-5.13.1/run/cloudera-scm-agent/process	14.0 吉字节/125.8 吉字节
tmpfs	/run/user/1009	0 字节/25.2 吉字节
tmpfs	/run/user/1007	0 字节/25.2 吉字节
/dev/sdb1	/data	19.3 吉字节/837.4 吉字节
/dev/mapper/VolGroup00-LogVol01	/	112.8 吉字节/823.5 吉字节
/dev/sda5	/boot/efi	0 字节/498.7 兆字节
tmpfs	/run	4.1 吉字节/125.8 吉字节
tmpfs	/run/user/0	0 字节/25.2 吉字节
/dev/sda2	/boot	132.5 兆字节/1,014.0 兆字节
tmpfs	/dev/shm	0 字节/125.8 吉字节

2. 数据节点的数据盘

数据节点的数据盘做 RAID0（一块盘做 RAID0，硬件 RAID）作为数据盘，文件格式为 xfs，并配置 noatime，不做 LVM，最好是同构。数据节点的数据盘：

磁盘	装入点	使用情况
/dev/sdj1	/mnt/sd09	2.1 太字节/3.6 太字节
/dev/sdi1	/mnt/sd08	2.1 太字节/3.6 太字节
/dev/mapper /VolGroup00-LogVol02	/mnt/sd11	1.8 太字节/3.3 太字节
cm_processes	/opt/cm-5.13.1/run/cloudera- scm-agent/process	101.6 兆字节/62.7 吉字节
/dev/sdb1	/mnt/sd01	2.1 太字节/3.6 太字节
/dev/sdk1	/mnt/sd10	2.0 太字节/3.6 太字节
/dev/sdc1	/mnt/sd02	2.1 太字节/3.6 太字节
/dev/sdf1	/mnt/sd05	2.1 太字节/3.6 太字节
/dev/sdh1	/mnt/sd07	2.1 太字节/3.6 太字节
/dev/sde1	/mnt/sd04	2.1 太字节/3.6 太字节
tmpfs	/run	762.8 兆字节/62.7 吉字节

```

NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda                  8:0    0 446.1G  0 disk
├─sda1               8:1    0    2M  0 part
├─sda2               8:2    0 500M  0 part /boot
├─sda3               8:3    0 400.5G  0 part
│   ├─centos-root    253:0   0 400G  0 lvm  /
│   └─centos-swap    253:1   0 500M  0 lvm  [SWAP]
sdb                  8:16   0 3.7T  0 disk /mnt/sata01
sdc                  8:32   0 3.7T  0 disk /mnt/sata02
sdd                  8:48   0 3.7T  0 disk /mnt/sata03
sde                  8:64   0 3.7T  0 disk /mnt/sata04
sdf                  8:80   0 3.7T  0 disk /mnt/sata05
sdg                  8:96   0 3.7T  0 disk /mnt/sata06
sdh                  8:112  0 3.7T  0 disk /mnt/sata07
sdi                  8:128  0 3.7T  0 disk /mnt/sata08
sdj                  8:144  0 3.7T  0 disk /mnt/sata09
sdk                  8:160  0 3.7T  0 disk /mnt/sata10

```

备注：数据盘大小建议一致，避免数据不均衡；磁盘不能做 LVM，避免影响存储性能。

4.Kafka Topic 分区规划

1.Kafka 磁盘规划

步骤 1：评估数据量：要求研发提前评估所有 topic 一个周期全量的数据大小。

步骤 2：计算磁盘总存储：如一块盘 825g，一个节点 20 块盘，10 个节点。那么磁盘总存储就是 165000g。

步骤 3：预估实际数据存储占比：

Topic:一个周期全量数据大小占磁盘总存储的百分比，超过百分之六十，即要求研发减少存储周期。

步骤 4：计算磁盘总块数：一个节点 20 块盘，10 个节点，总磁盘块数 200 个。

2.Kafka Topic 分区方式 1

合理预分区：所有分区数量为磁盘总数的整数倍。如所有的 topic 总数据量

为 50000g，磁盘个数为 200，那么就可以设置总分区数为 200/400/600.具体多少分区

数视业务决定。若分区数为 400，那么一个分区的大小约 125g。

案例 1：

某一个 topic：cbss001 的预估数据量是 210g，那么通过计算（每个分区：125g）可以将其分成两个分区。这样根据 kafka 副本落盘策略，各个主机磁盘就能保证最大限度的存储均衡。

案例 2：

如果我们建立了一个非常大的 topic（100 个分区），那么此 topic 的分区数量应该是磁盘数的整数倍；如果说我们还有建立一个小的 topic（如 10 个分区），那么此 topic 的每个分区大小应该跟之前的大的 topic 分区的大小保持一致。

3.Kafka Topic 分区方式 2

确定分区数量的方法：

- （1）创建一个只有 1 个分区的 topic

- (2) 测试这个 topic 的 producer 吞吐量和 consumer 吞吐量。
 - (3) 假设他们的值分别是 T_p 和 T_c ,单位可以是 MB/s.
 - (4) 然后假设总的目标吞吐量是 T_t ,那么分区数= $T_t/\min(T_p, T_c)$
-

kafka 性能基准测试:

(1) Kafka Producer 压力测试

```
bin/kafka-producer-perf-test.sh --topic test --record-size 100 --num-records 100000  
--throughput -1 --producer-props bootstrap.servers=hadoop1:9092,hadoop2:9092,hadoop3:9092
```

说明:

record-size 是一条信息有多大,单位是字节。

num-records 是总共发送多少条信息。

throughput 是每秒多少条信息, 设成-1, 表示不限流, 可测出生产者最大吞吐量。

(2) Kafka Consumer 压力测试

```
bin/kafka-consumer-perf-test.sh --broker-list hadoop1:9092,hadoop2:9092,hadoop3:9092 --topic  
test --fetch-size 10000 --messages 10000000 --threads 1
```

参数说明:

--zookeeper 指定 zookeeper 的链接信息

--topic 指定 topic 的名称

--fetch-size 指定每次 fetch 的数据的大小

--messages 总共要消费的消息个数

5.HBase 表 Region 规划

1.业务场景

每天写入 5 亿+条数据

每天写入 176G+数据量

region 分裂大小为 10G

TTL 为 20 天

rowkey 为手机号

2.预建分区个数

预建分区的个数=最终保有的数据量/每个 region 配置的大小

- (1) Region 最大值推荐 10-20Gb, 最优值推荐 5-10Gb。

- (2) 数据总量=176*20=3520G

3.预分区实现

方法一:

```
admin.createTable(tableDesc, startkey, endkey, numRegion)
```

方法二:

```
admin.createTable(tableDesc, splitKeys);
```

6.集群扩容

Q1: 集群线上扩容如何达到自动化? 线上扩容, 规模正常都以 10+ 以上, 如果都人肉操作, 时间投入与产出比不匹配, 人力造成很大的不必要浪费, 想学习线上集群实际扩容的方案。

A1: 产线环境扩容数量少则几十多则上百, 人肉肯定不行, 我们公司的运维分为 IAAS 运维、大数据运维、应用运维, 大数据运维工程师在扩容集群时, 需要向 IAAS 运维工程师以工单的方式申请服务器并提出自己的需求, IAAS 运维工程师在提供服务器时不管是新采购服务器还是其他集群退役的服务器都需要重装系统, 重装系统的镜像是针对大数据定制的, 镜像包含了大数据运维工程师的通用需求以及安装操作系统后的网络、磁盘以及其他设置, 比如关闭防火墙、时钟同步、同构磁盘挂载、关闭大透明页、关闭 SWAP、公用 YUM 源、禁用 SELinux 等模式化操作, 大数据运维工程师收到服务器后准备工作基本准备完毕了, 进行少量准备工作就可以直接进入了 CM 可视化批量安装模式, 比如脚本批量设置 hostname、脚本同步/etc/hosts 文件等; 当然如上所说的 IAAS 操作, 如关闭防火墙、时钟同步、同构磁盘挂载、关闭大透明页、关闭 SWAP、公用 YUM 源、禁用 SELinux 都可以脚本化, 无非就是使用 SSH 通讯方式设置, 这就需要大数据运维同学熟练使用 shell 了。

7.具体目录下的文件存储在哪些 DataNode

Q2: 已知一个 HDFS 的目录, 想知道此目录下的文件数, 而且存储于哪些 DataNode 节点上。

A2:

1. 查看文件数使用 count 命令, 如下查看 /ops 目录, 目录数为 9606, 文件数为 1353 万+, 总文件大小为 9.7P

```
[ops_ai_pro@work1 ~]$ hdfs dfs -count /ops
9606 13532136 9744893453648 /ops
[ops_ai_pro@work1 ~]$
```

2. 查看 /ops/test 目录下的文件存储的位置

hdfs fsck /ops/test -files -blocks -locations -racks

```
[ops_ai_pro@work1 ~]$ hdfs fsck /ops/test -files -blocks -locations -racks
Connecting to namenode via http://10.10.10.1:50070/fsck?ugi=ops_ai_pro&files=1&blocks=1&locations=1&racks=1&path=%2Fops%2Ftest
FSCK started by ops_ai_pro (auth:SIMPLE) from /10.10.10.1:20.232 for path /ops/test at Wed Mar 04 20:53:40 CST 2020
/ops/test <dir>
/ops/test/input <dir>
/ops/test/input/MRtext.txt 334 bytes, 1 block(s): OK
0. BP-1698834824-10.10.10.1:21.6-1536323326497:blk_1073752335_11511 len=334 Live_rep1=3 [/default-rack/10.10.10.1:21.9:50010, /default-rack/10.10.10.1:21.6:50010, /default-rack/10.10.10.1:21.8:50010]
/ops/test/output <dir>
/ops/test/output-libjars <dir>
/ops/test/output-libjars/_SUCCESS 0 bytes, 0 block(s): OK
/ops/test/output-libjars/a21_path <dir>
/ops/test/output-libjars/a21_path/test3 <dir>
/ops/test/output-libjars/a21_path/test3/-r-000000 17 bytes, 1 block(s): OK
0. BP-1698834824-10.10.10.1:21.6-1536323326497:blk_1073752595_11771 len=17 Live_rep1=3 [/default-rack/10.10.10.1:21.9:50010, /default-rack/10.10.10.1:21.6:50010, /default-rack/10.10.10.1:21.8:50010]
```

```

Status: HEALTHY
Total size: 748 B
Total dirs: 11
Total files: 7
Total symlinks: 0
Total blocks (validated): 6 (avg. block size 124 B)
Minimally replicated blocks: 6 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 2
Average block replication: 3.0
Corrupt blocks: 0
Missing replicas: 0 (0.0 %)
Number of data-nodes: 5
Number of racks: 1
FSCK ended at Wed Mar 04 20:53:40 CST 2020 in 1 milliseconds

```

8. 内存与硬件之间有啥关系

Q3:DataNode 单节点，存储容量大小与当前节点上的 CPU，内存硬件之间有什么样的关系，磁盘扩容很容易，但这不意味着单节点上数据盘就可以无限地扩容，因此在这方面，有什么实际线上的经验分享，即这三者间可以遵循怎样合适的关系

A3: 从我们内部实践来说没有发现三者之间的规律，因为大数据业务每个公司都不一样，有的计算是 CPU 密集型的，有的计算是内存密集型的、有的计算是 IO 密集型的；我们的服务器都是中高配置，每个服务器的磁盘是 4Tx10、CPU 24core、128G 内存。

9.产线环境是单集群还是多集群

Q4: 老师线上 6000+规模集群，抛开一些其它组件集群，仅讨论 HDFS 数据存储集群，保守理应在有 3000 节点左右，想知道这些存储节点只是单集群 NameNode 集群来维护吗？还是多集群独立维护的，如果是多集群存储数据，它们之间数据是如何打通进行关联的？如果仅是一个集群的话，我想知道这个集群的 NameNode 上的配置，是如何 Hold 住这些数据 DataNode 的？

A4:

我们的集群也是用的 Cloudera 公司的，我们付了费用，Cloudera 没有保障说 CM 能够纳管 1000 节点以上的集群，再加上我们的机房有容量限制，所以我们是建了多套集群，最大集群有 800+节点，最小集群也有 300+节点；800+datanode 下，namenode 是可以支撑的，像一些互联网大厂，字节跳动、京东等他们的单集群规模有上万节点，此时就需要对 hdfs 进行深度定制了，他们改了很多源码，而且有 200+高级技术维护人员；

多集群建设要考虑业务情况，比如我们公司有 10 个以上业务，5 大核心集群，按业务相关情况划分集群，不过也难免有跨集群的作业，目前公司内部自主研发了大数据采集交换平台，你也可以使用 distcp 进行数据对拷，目前我们也正在准备自主研发多集群并行混算平台。

10.hdfs 存储压缩格式如何选取

Q5: HDFS 存储节点上的数据，存储压缩格式是如何选取的，默认采用哪种文件存储类型与

存储格式，冷热数据如何界定的？

A5:

我们产线环境用了 `gz` 和 `snappy` 压缩格式，`gz` 用于不常用的冷数据，`snappy` 用于热数据；冷热数据是跟业务相关的。

11.DataNode 与 HBase 一起部署合理嘛？

Q6: 课堂上看到 HDFS 集群的 DataNode 与 HBASE 是集成在一起部署的，我好奇，HBase 面向的都是高频率读写的业务，老师确定部署一起后，线上业务没有出现过问题吗？

A6:

你应该是理解错了，HBase 不能跟 YARN 在一起部署；HBase 需要跟 HDFS 部署在一起的，这样可以利用数据本地性提升 IO 性能，并且可以降低网络延迟，降低带宽负载。

12.yarn 集群资源队列如何划分

Q7: Yarn 集群计算层，目前集群面向用户使用群体少，故白天仅是数据写入，集群 CPU 资源都较为存在大量冗余状态，但对于凌晨执行集中高频提交跑批作业计算任务时，集群的 CPU 资源明显不够，甚至引起节点 CPU 负载过高导致节点的宕机，从而造成集群雪崩。简单来说，白天集群资源使用率在 5%，凌晨执行高频跑批作业任务时，资源使用率上到 100%，引起集群宕机。在不增加资源的前提下，资源老师会如何划分队列？

A7:

这个划分队列无法解决根本问题，建议将业务划分优先级，错开时间执行。

13.磁盘损坏了如何处理？

Q8. 某一台 CDH 物理机 12 块 RAID0 硬盘，如果其中有 4 块 RAID0 硬盘同时损坏，请问接下来大数据运维人员的详细处理流程是什么？

A8:

HDFS 有个参数 `dfs.datanode.failed.volumes.tolerated`，值为 0 的时候表示当有任何盘损坏后 `datanode` 则停止服务，如果 4 块盘同时损坏了，此时 `datanode` 进程已经停止了，你可以修改 `hdfs-site.xml` 配置文件将损坏的磁盘路径去掉，重启 `datanode` 即可，然后进入磁盘报修流程，待磁盘更换后，停止 `datanode`，在 `hdfs-site.xml` 配置文件中假如新的磁盘，启动 `datanode` 即可。

14.开发人员可以使用 beeline 访问 Hive 服务嘛？

Q9. CDH 如果为开发人员提供 HIVESERVER2 服务，可以让开发人员通过 `beeline` 命令访问？

A9: 是的, beeline 使用 JDBC 协议来连接 HIVESERVER2

15.DataNode 节点不稳定该如何处理?

Q10.在晚高峰期 HDFS 集群会出现某些 datanode 不稳定的情况, 频繁有 datanode 脱离节点, 该如何处理呢?

A10: 需要找到 datanode 的具体原因, datanode 相对来说还是比较稳定的, 需要看下是否是 GC 问题, 如果是的话适当调大内存, 再看下最大的打开文件数或进程数的限制是否太小.

16.如何处理 HDFS 节点数据不均衡问题?

Q11.CDH 集群扩容 10 台机器后, 新加入的 Datanode 角色数据相对较少, 如何处理 HDFS 的数据分布不均衡的现象呢?

A11:

在内存占用较低的节点上启动 balancer 脚本, 将 HDFS 中所有节点的存储值中的最低值和平均值的差值设置为 5%。

命令: `./start-balancer.sh -threshold 5`

17.生产环境大数据集群如何监控?

Q12.CDH 监控项从某种角度来说并不是很细粒度, 请问您线上是如何监控的呢, 能将详细步骤给我们剖析一下吗?

A12: 目前我们产线环境还是以 CDH 监控为主, CDH 的指标还是挺多的, 只不过保留的周期不长, 对于排障来说基本够用了, 当然我们会推进一些监控工具, 如通过 Flink 采集集群指标进行深度监控。

18.生产环境如何解决小文件问题?

Q13.如何基于 CDH 集群监控大量的小文件的呢? 衡量小文件标准, 以及出现大量小文件在生产环境该如何解决呢?

A13: CM 解决不了大量小文件的监控, 需要额外做其他工作, 这块在集群治理里面我们会详细讲; 对于怎么衡量小文件的标准, 你可以简单认为小于 block size 的文件就是小文件, 但是在企业真实情况下小文件问题可能更加严重, 比如大量 10M、几十 M 以下的文件, 单纯技术无法解决问题, 需要组织协同。

19.hdfs 目录的配额有限制嘛？

Q:CDH 如何对 HDFS 各级目录做权限管理，目录的配额(即使用 HDFS 的容量限制)限制呢？

A: HDFS 可通过 ACL 精细控制目标权限，除了 ACL 后续我们也会讲 sentry；目前我们产线环境没有做容量配额限制，怕影响生产，我们通过集群治理来解决容量问题。

20.大数据各组件高可用原理

Q: 除了 hdfs，像 yarn，spark，flink，hbase，kafka（controller）这类集群的高可用原理都是一样的吧

A:

hdfs 的高可用指的是 namenode 的高可用，通过 zookeeper 来选举，通过抢占 zk 的节点来实现。

Yarn 的高可用指的是 resourcemanager 的高可用，通过 zookeeper 来选举，通过抢占 zk 的节点来实现。

hbase 高可用指的 hmaster 的高可用，通过 zookeeper 来选举，通过抢占 zk 的节点来实现。

kafka（controller）：通过 zookeeper 来选举，通过抢占 zk 的节点来实现。

spark，flink:一般运行时在 on yarn 模式，yarn 会提供容错机制

21.datanode 上的坏块是如何产生的？

参考答案：

当正在写数据时，意外断电，可能会导致数据块损坏或丢失；

大于副本数的 datanode 个数同时停止、宕机时会出现数据块损坏或丢失，如果 datanode 恢复后警告会消失；

注：所谓坏块就是此 block 的所有副本都没有了，但是 namenode 的元数据中还有此 block 的信息。

22.集群出现故障如何实现自动修复呢？

Q:

集群出现故障如何实现自动修复呢？比如一个 DataNode 在凌晨 2 点半突

然挂了 4 个节点，那时候咱们运维人员已经进入深度睡眠状态，如何让其自动将该节点进程自动拉起呢(此处我们假设进程存放则被成功启动)？

A:

我们采集了 CDH 的严重事件，更加严重事件盘点是否有 datanode 挂掉

了，如果有则调研 CDH API 启动 datanode。此外我们每天都有人轮流值班的。

23.大数据安全，权限管理，审计是否有一套完整的解决方案呢？

参考答案：

目前老师所知，没有一套完整方案，可能商业化产品有，大数据安全可以大概有边界网关的安全，比如云桌面、VPN 等，其次是大数据组件自己的安全。

24.如何避免开发人员误删数据自己背锅？

Q:

HDFS 的数据被删除没法直接被监控，企业中该如何避免开发人员误删除数据免得咱们运维背锅呢？

A: 打开审计日志，并使用 filebeat 采集然后写入 ES 中，可以实时查询所有数据的操作。

25.开发人员在 hive 中创建的元数据表信息该如何实时监控呢？

参考答案：

可以使用阿里开源的 Canal 来实时读取 Hive 的元数据库 mysql 的 binlog 数据，达到实时监控表的变更操作。

推荐：使用 Flink CDC 来实时获取 Hive 的元数据库 mysql 的变化。

26.ElasticSearch VS HDFS 应用场景

Q: ElasticSearch 和 HDFS 都是分布式文件系统，也都可以做数据存储和检索功能，也都是 JAVA 开源产品，为什么在大数据领域中 HDFS 比 ES 更火呢？

A:

这两个组件没有可比性，应对的场景不一样，HDFS 应用于海量数据存储，ES 应用于全文搜索，在电商和搜索引擎用的多。

27.DataNode 节点间数据均衡问题？

Q:

DataNode 节点间数据均衡这个场景，大规模集群下必定会存在低配置节点，如：CPU/内存/数据盘，这里就仅聊数据盘这个，如：A 节点 100G，B 节点 500G，这种情况下，是怎样均衡节点数据的？5%值吗？这里怎么均衡都是无法达到预期的均衡值呀，是直接将 A 节点下线吗？又比如说同一节点 C，挂载了 3 块数据盘，分别为，/mnt/a/10G,/mnt/b/100G,/mnt/c/500G,这种 DataNode 节点的均衡数据又应该如何解决的，

1000+规模的 HDFS 应该也不会是 3.x 版本吧？对于不支持 Disk Balancer 的 2.X 版本，解决方案有吗？

A: 首先异构磁盘确实会带来问题，没有绝对的均衡，节点间存储比率不超过 5%算是均衡了；HDFS 在写入时有两种选择卷（磁盘）的策略，一是基于轮询的策略（RoundRobinVolumeChoosingPolicy），二是基于可用空间的策略（AvailableSpaceVolumeChoosingPolicy）

将这个配置 `dfs.datanode.fsdataset.volume.choosing.policy` 为 `org.apache.hadoop.hdfs.server.datanode.fsdataset.AvailableSpaceVolumeChoosingPolicy`
对于不支持 Disk Balancer 的 2.X 版本，可以将 datanode 退役，清理磁盘后再服役。

28.数据仓库搭建属于运维的工作么？

Q: 数据仓库到底是个什么东西呢？能给我们举个例子说明数仓的角色定位及功能吗？还有就是数据仓库是大数据运维工程师搭建还是大数据开发它们自建搭建呢？

A:

数据仓库主要是大数据开发搭建的，数据仓库主要是根据采集的数据进行清洗、加工、汇总生成多维度的报表数据，给高层领导看到，运维人员不参与数据清洗、加工等工作，只负责 hadoop 平台搭建和运维；数据仓库是基于 hadoop 之上建立的。

29.跨集群数据交互如何处理？

Q: 线上集群规模是以集群为粒度划分业务线，跨集群间数据也是需要交互，这种业务是如何根据集群来划分？还是说，它们之前数据的交互实时性不高？跨了集群的数据具体是如何能打通的？DistCp 跨集群间的数据迁移也觉得它慢。根本无法满足要求。它可配置的参数也配置了。

A: 实时性要求不高，我们机房有万兆带宽的专线，跨集群同步数据不会同步全量数据，在源端集群会先经过数据过滤，只传输需要并进行清洗过滤后的数据。

30.非开发人员如何监控 hdfs 文件信息？

Q:

HDFS SHELL 命令方式去监控 HDFS 文件信息这些，如前边就提问的查询文件数，Block 块等，对于专业人士偶尔查询，还行，但对于一个外行的人，连如何登录服务器都不知道，但他就是想知道集群的使用情况，老师是如何将这种资讯汇报他们的，或者是收集方案？

A: 我们将收集的数据写入 ES，可以使用 kibana 制作报表。

31.CM 收费之后，企业主流如何进行集群监控？

Q: CDH 是开源的产品，但是 CM 是付费的呀，我们在生产环境中使用 CM 没有向 cloudera 公司交钱是否会收到律师函呢？但是不用 CM 我们又不知道如何管理(搭建，扩容，监控等)CDH 集群了，如果公司不愿意交钱的话我们大数据运维该如何处理呢？

A:

以后都没有免费的了，不原因交钱的话可以使用开源社区版，节点不多的话可以人肉运维，自搭建监控平台，节点多的话自己可以写脚本或借助自助化工具运维，比如 ansible 等。

32.MapReduce 作业卡住是什么原因？

Q: 您遇到 YARN 资源还剩很多(还剩下 50%左右)，但是 MapReduce 任务就是卡着不动的场景吗？我把 job 并发度降低就好了。但原因在哪我依旧不知道...

A:

这个要具体详细查看原因了，卡着不动不代表是整个集群资源的问题，可能是任务本身的数据倾斜，也可能是 GC 问题，也可能是任务所在的服务器负载过高，也可能是所在队列的资源问题。

YARN 作业慢问题排查思路：

- 1.首先看作业所处的资源队列有没有资源
- 2.查看作业的哪些 task 运行慢：如果说某个 task 运行慢

- 1) 是否是数据倾斜，如果有就是数据倾斜引起的，此时如果是 hive sql，需求调整 sql 语句，避免数据倾斜
- 2) 查看 task 所在机器的系统负载是否很高，如果是则进去这台主机查看高的原因，原因可能是此机器上运行了大量的 task 导致系统资源负载过高，还有就是某个 task 的 jvm 频繁进行垃圾回收导致系统负载过高

3.硬件问题

- 4.查看最近集群有没有变更（如：我们生产上遇到一个问题，大面积作业运行缓慢，因为最近扩容了 50 台集群，但是其他集群某些机器的 host 文件里面配置的这 50 台集群的 host 有问题）

- 5.查看 RPC 请求是否过慢